COURSE SLIDES

Certified Kubernetes Application Developer (CKAD)

WSQ Course Code: TGS-2025053212

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version 5.0 · 27 July 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.

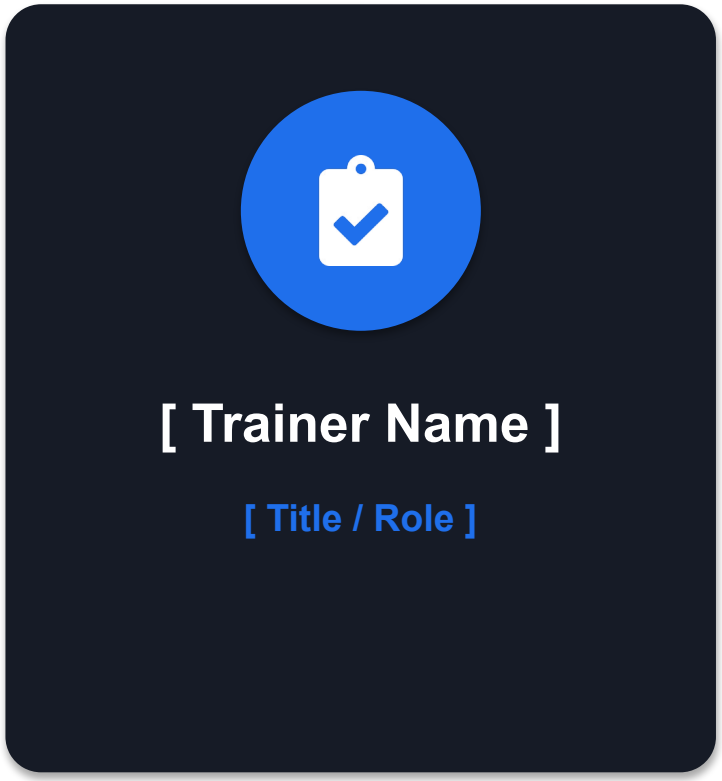


Minimum 75% attendance required

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Qualifications:

Expertise:

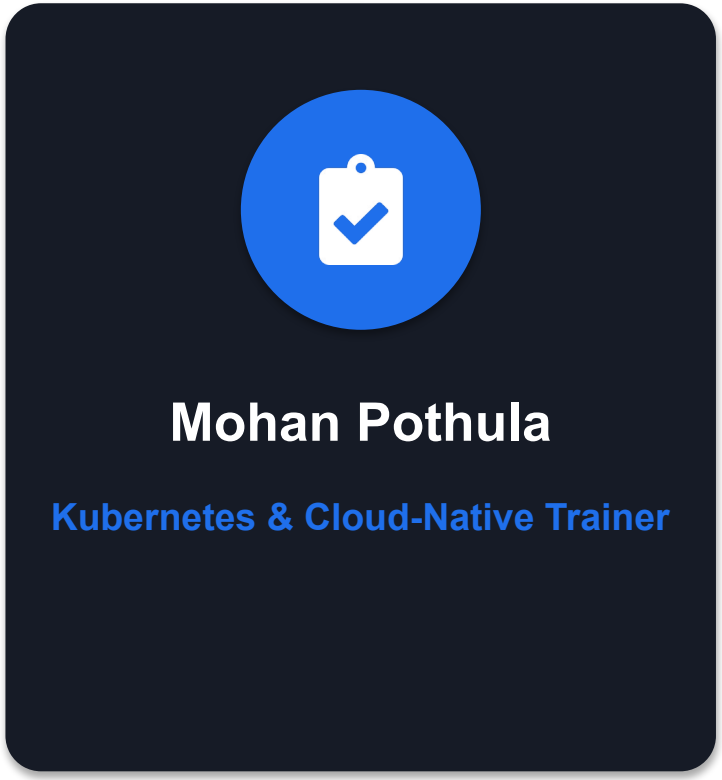
Experience:

Contact / Profile:

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Role
Principal Kubernetes & DevOps trainer at Tertiary Infotech Academy.

Qualifications
Certified Kubernetes Application Developer (CKAD) · CKA · Docker Certified Associate · AWS Solutions Architect.

Expertise
Kubernetes · Docker · CI/CD · Cloud-native platform engineering.

Experience
20+ years across DBS Bank, SingTel, Mediacorp and SPH Media.

© 2026 Tertiary Infotech Academy Pte Ltd

HOUSE RULES

Ground Rules

Phones on silent

Set your phone to silent mode.

Participate actively

No question is too small — ask freely.

Mutual respect

One conversation at a time.

Be punctual

Return from breaks on time.

Step out quietly

For calls or short breaks.

75% attendance

Required for funding eligibility.

© 2026 Tertiary Infotech Academy Pte Ltd

SCHEDULE

Lesson Plan — 3½ Days, 8 hours/day

1 Day 1 · Domain 1

Application Design & Build

· Container images & multi-stage builds (Labs 1–2)
· Pods, Jobs & CronJobs (Labs 3–5)
· Multi-container Pods & Volumes (Labs 6–8)

2 Day 2 · Domain 2

Application Deployment

· Deployments, ReplicaSets & rollouts (Labs 9–10)
· Blue/green & canary (Labs 11–12)
· Helm, Kustomize, Daemon/StatefulSet (Labs 13–15)

3 Day 3 · Domains 3 & 4

Observability + Config & Security

· Probes, logging, metrics, debug (Labs 16–20)
· ConfigMaps, Secrets, SecurityContext (Labs 21–23)
· ServiceAccounts, RBAC, Quotas (Labs 24–26)

4 Day 4 (½) · Domain 5

Services & Networking + Assessment

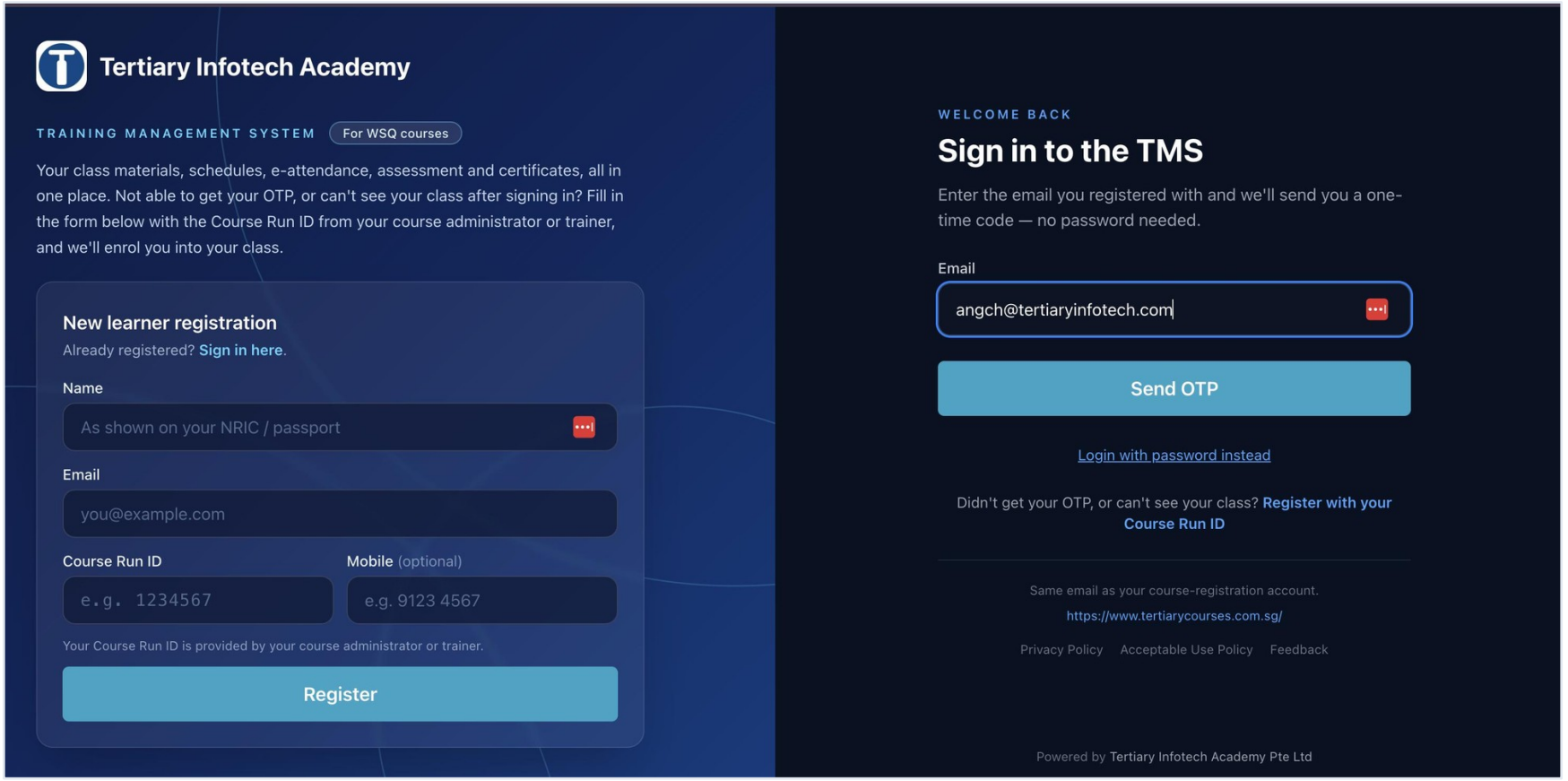
· Services & Service DNS (Labs 27–28)
· Ingress with TLS & NetworkPolicy (Labs 29–30)
· 1:30 PM mock-exam practice, then final assessment from 2:00 PM

Daily timing: 9:30am – 6:30pm · 1-hour lunch · short tea breaks within each day

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE PORTAL

Login to LMS and Download Course Material



Log in at lms-tms.tertiaryinfotech.com with your registered email and the OTP sent to your inbox, then open this course to download the slides and Learner Guide.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL ACHIEVE

Learning Outcomes

1. **LO1** Build and optimise container images with single- and multi-stage Dockerfiles.
2. **LO2** Create and manage Pods imperatively and declaratively with kubectl and --dry-run.
3. **LO3** Run batch workloads with Jobs and scheduled workloads with CronJobs.
4. **LO4** Design multi-container Pods using the sidecar and init-container patterns.
5. **LO5** Persist and share Pod data with emptyDir, tmpfs and hostPath volumes.
6. **LO6** Deploy, scale, observe, secure and network applications across the five CKAD domains.

© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT RULES

Briefing for Assessment

Clear your space

Keep only approved materials on the table.

No recording

No photos of the assessment.

Work individually

No discussion during the assessment.

Use the LMS

The assessment is delivered online.

Open book

Slides, Learner Guide & kubernetes.io are allowed.

Time's up

Submit when time is up.

© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT FLOW

How the Assessment Runs

1 · Digital Attendance
Scan the SSG QR

2 · Briefing
Rules, timing, open book

3 · WA (SAQ)
Written · 1 hour

4 · PP
Practical · 1 hour

5 · Submit on LMS
Before time is up

SIGN-OFF PATH

Trainer marks

Against the WSQ marking guide for every K and A criterion.

Result recorded

Competent / Not Yet Competent entered on the LMS.

Learner signs

You sign the Assessment Summary Record.

Assessor signs off

Trainer signs and submits the record to SSG.

Assessed as Competent = minimum 75% attendance · pass the WA (SAQ) · pass the Practical Performance

© 2026 Tertiary Infotech Academy Pte Ltd

FINAL ASSESSMENT

Assessment

Final Assessment

- Written Assessment (SAQ) 1 hour + Practical Performance 1 hour
- Mock-exam practice from 1:30 PM; final assessment from 2:00 PM
- Covers all five CKAD domains and the hands-on labs
- Open book — slides, Learner Guide & kubernetes.io
- Must be attempted for SSG funding

Funding & Competency

Minimum 75% attendance

Based on SSG digital attendance records.

Assessed as Competent

Pass the Written Assessment (SAQ) and the Practical Performance.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE ADMINISTRATION

Welcome & Housekeeping

© 2026 Tertiary Infotech Academy Pte Ltd

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.

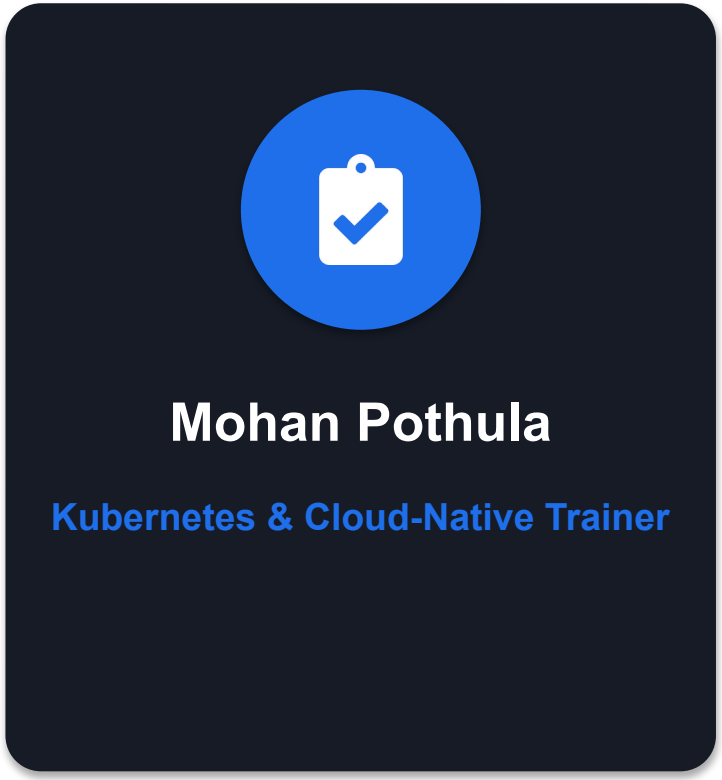


Minimum 75% attendance required

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Role
Principal Kubernetes & DevOps trainer at Tertiary Infotech Academy.

Qualifications
Certified Kubernetes Application Developer (CKAD) · CKA · Docker Certified Associate · AWS Solutions Architect.

Expertise
Kubernetes · Docker · CI/CD · Cloud-native platform engineering.

Experience
20+ years across DBS Bank, SingTel, Mediacorp and SPH Media.

© 2026 Tertiary Infotech Academy Pte Ltd

SCHEDULE

Lesson Plan — 3½ Days, 8 hours/day

1 Day 1 · Domain 1

Application Design & Build

· Container images & multi-stage builds (Labs 1–2)
· Pods, Jobs & CronJobs (Labs 3–5)
· Multi-container Pods & Volumes (Labs 6–8)

2 Day 2 · Domain 2

Application Deployment

· Deployments, ReplicaSets & rollouts (Labs 9–10)
· Blue/green & canary (Labs 11–12)
· Helm, Kustomize, Daemon/StatefulSet (Labs 13–15)

3 Day 3 · Domains 3 & 4

Observability + Config & Security

· Probes, logging, metrics, debug (Labs 16–20)
· ConfigMaps, Secrets, SecurityContext (Labs 21–23)
· ServiceAccounts, RBAC, Quotas (Labs 24–26)

4 Day 4 (½) · Domain 5

Services & Networking + Assessment

· Services & Service DNS (Labs 27–28)
· Ingress with TLS & NetworkPolicy (Labs 29–30)
· 1:00 PM mock-exam practice, then final assessment from 2:00 PM

Daily timing: 9:00am – 6:00pm · 1-hour lunch · short tea breaks within each day

© 2026 Tertiary Infotech Academy Pte Ltd

FINAL ASSESSMENT

Assessment

Final Assessment

- Written Assessment (SAQ) 1 hour + Practical Performance 1 hour
- Mock-exam practice from 1:30 PM; final assessment from 2:00 PM
- Covers all five CKAD domains and the hands-on labs
- Open book — slides, Learner Guide & kubernetes.io
- Must be attempted for SSG funding

Funding & Competency

Minimum 75% attendance

Based on SSG digital attendance records.

Assessed as Competent

Pass the Written Assessment (SAQ) and the Practical Performance.

© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT RULES

Briefing for Assessment

Clear your space

Keep only approved materials on the table.

No recording

No photos of the assessment.

Work individually

No discussion during the assessment.

Use the LMS

The assessment is delivered online.

Open book

Slides, Learner Guide & kubernetes.io are allowed.

Time's up

Submit when time is up.

© 2026 Tertiary Infotech Academy Pte Ltd

ICE-BREAKER

Let's Know Each Other

Take a minute to introduce yourself to the class:



Your role

Your name and organisation / role.



Your experience

Any experience with Docker, Kubernetes or the cloud.



Your goal

Whether you are targeting the CKAD exam or production skills.

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL ACHIEVE

Learning Outcomes

1. **LO1** Build and optimise container images with single- and multi-stage Dockerfiles.
2. **LO2** Create and manage Pods imperatively and declaratively with kubectl and --dry-run.
3. **LO3** Run batch workloads with Jobs and scheduled workloads with CronJobs.
4. **LO4** Design multi-container Pods using the sidecar and init-container patterns.
5. **LO5** Persist and share Pod data with emptyDir, tmpfs and hostPath volumes.
6. **LO6** Deploy, scale, observe, secure and network applications across the five CKAD domains.

© 2026 Tertiary Infotech Academy Pte Ltd

01

DAY 1

Application Design and Build

Images · Pods · Jobs · CronJobs · Multi-container · Volumes (CKAD Domain 1 — 20%)

© 2026 Tertiary Infotech Academy Pte Ltd

OVERVIEW

Why Container Images?

- An image is a read-only blueprint that packages your app with all its dependencies.
- A container is a running instance of an image — it starts in milliseconds.
- Kubernetes never builds images; it pulls them from a registry by name:tag and runs them.
- CKAD expects you to read, write and fix Dockerfiles under time pressure.

© 2026 Tertiary Infotech Academy Pte Ltd

ONE INSTRUCTION = ONE LAYER

Anatomy of a Dockerfile

Base & setup

- FROM — pick a small, pinned base
- WORKDIR — set the working dir
- COPY / ADD — bring files in

Build steps

- RUN — install dependencies
- Each line = one cached layer
- Order steps for cache reuse

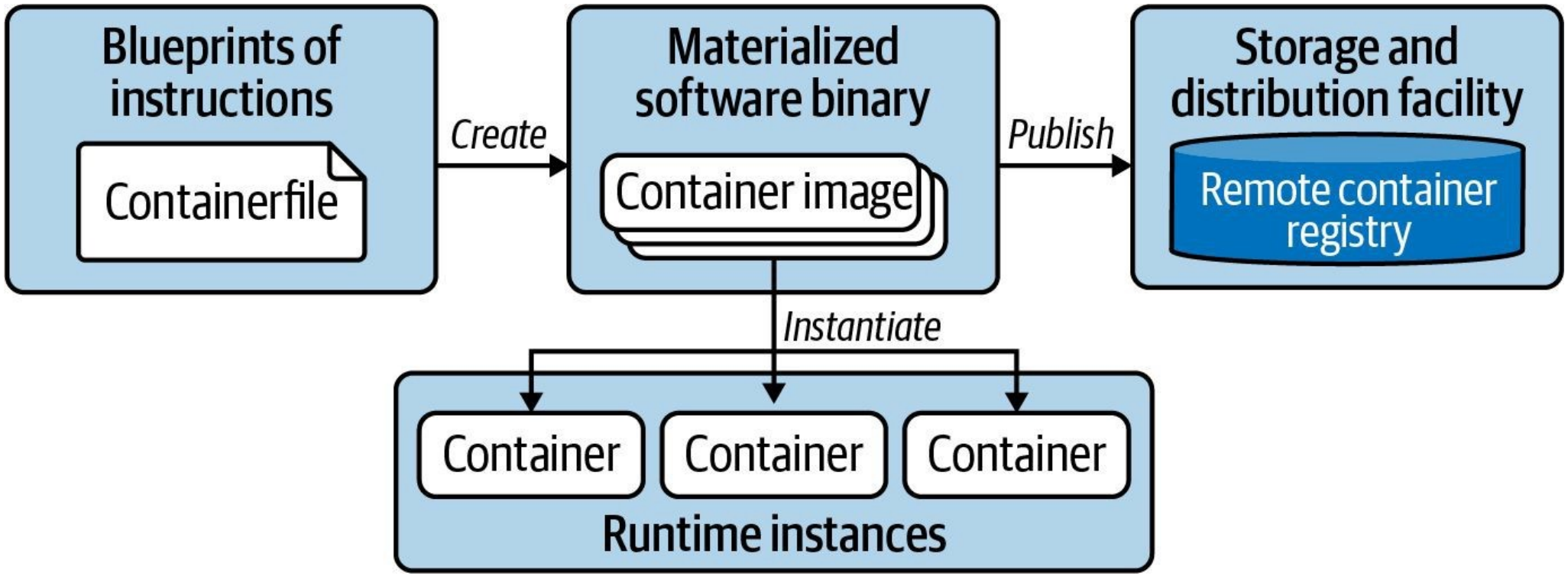
Runtime

- EXPOSE — document the port
- ENV — runtime configuration
- CMD / ENTRYPOINT — what runs

© 2026 Tertiary Infotech Academy Pte Ltd

Containerization Process

Building and publishing an image, running image in container



SHRINK THE IMAGE

Single-Stage vs Multi-Stage Builds

Single-stage

- One FROM — build + runtime together
- Ships the whole toolchain
- Large image (~800 MB for Go)
- Bigger attack surface

Multi-stage

- Multiple FROM stages with AS <name>
- COPY --from=builder copies only the binary
- Tiny image (~8 MB, distroless)
- No shell, no package manager

© 2026 Tertiary Infotech Academy Pte Ltd

Multi-Stage Build — Shrink the Image

Stage 1 — builder

FROM golang:1.22
WORKDIR /src
COPY . .
RUN go build -o /server

Ships compiler + source + modules

~800 MB final image

COPY --from=builder



only the binary

Stage 2 — runtime

FROM gcr.io/distroless
COPY --from=builder /server /
ENTRYPOINT ["/server"]

No shell · no package manager
Just your binary

~8 MB final image

© 2026 Tertiary Infotech Academy Pte Ltd

SECTION

Exam Objectives and Overview

Kubernetes Certification Path

Entry level certifications for beginners, hands-on certification for practitioners

- Entry Level
- Practitioner
- Practitioner

Exam Curriculum Details

Each domain is broken up into subtopics (find the latest outline on GitHub)

Application Design
and Build — 20%

Application
Deployment — 20%

Observability &
Maintenance — 15%

Environment, Config
& Security — 25%

Services and
Networking — 20%

Total
100%

CONTAINER IMAGES

Exam Domains

High-level topics and their weight that contribute to the total score

Application Design
and Build — 20%

Application
Deployment — 20%

Observability &
Maintenance — 15%

Environment, Config
& Security — 25%

Services and
Networking — 20%

Pass mark
66%

SECTION

Intro to Kubernetes

CONTAINER IMAGES

Containers

Packages to get software to run reliably

- App Libraries
- Payment App and
- Dependencies

Why use “Containers”?

Containers allow standardization, reduction of resource utilization, fault isolation and

- immutability
- Benefits of using Containers
- Standardization
- • Self-contained Fault Isolation
- • Portable
- • Platform Agnostic
- Reduction
- (Resource Utilization)
- Immutable
- • Lightweight
- • No Guest OS

SECTION

Virtual Machines and Containers

Container Orchestration Tool

Orchestration

- DB DB DB DB
- App App App App
- Docker Host Docker Host Docker Host Docker Host

SECTION

What is Kubernetes?

The founding of “κυβερνήτης”

History

- • Greek for “Helmsman”
- • Founded by Joe Beda, Brendan Burns and
- Craig McLuckie in Google
- • First announced to public in mid-2014
- • Version 1 released on July 2015
- • We call it k8s for short

Orchestration tools

Kubernetes became the market leader after mass support from the community and support

- from key market players in the cloud space
- Docker Swarm Kubernetes Mesos

CONTAINER IMAGES

Kubernetes (k8s) - Key Benefits

Portable Scalable Highly Available

Portable
Runs anywhere

Scalable
Scale on demand

Highly available
Self-healing

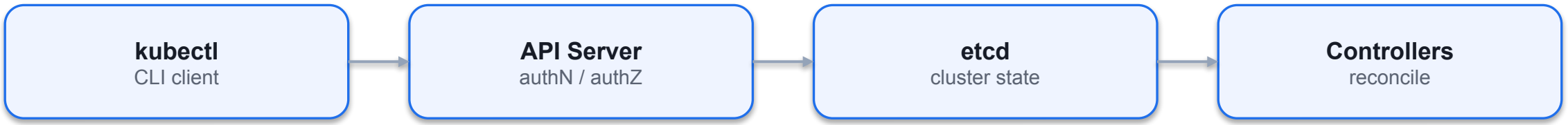
Declarative
Desired state

Extensible
CRDs & operators

Open source
CNCF governed

Using kubectl

Primary tool to interact with the Kubernetes clusters from the CLI



Master Node

- API Server (kube-apiserver): Orchestrating all

- operations within the cluster
- • Scheduler (kube-scheduler): Selects optimal node
- to run pods
- • Controller Manager (kube-controller-manager):
- Different controllers with different functions
- • Etcd: Stores data in key-value format

Worker Nodes

- Kubelet: Primary agent that communicates with

- kube-apiserver
- • Kube-Proxy: Network proxy that enables
- communication between every node and pod
- • Container Runtime (Docker): Software to run
- containers
- • Workloads (Pods): Workloads are scheduled by
- scheduler to run on worker nodes

SECTION

Kubernetes Cluster A set of nodes grouped together

Imperative Commands

"Execute Hazelcast and configure it using parameters"

```
$ kubectl run hazelcast
 --image=hazelcast/hazelcast
 --restart=Never
 --port=5701
 --env="DNS_DOMAIN=cluster"
 --labels="app=hazelcast,env=prod"
pod/hazelcast created
```

Build a Container Image with Docker

LAB 1 Container images

Write a Dockerfile from scratch, build a tagged image, run it locally and inspect its layers. CKAD expects you to read and write Dockerfiles confidently — you may be asked to fix a broken one or produce one under time pressure.

You'll build

Write a Dockerfile, build and tag an image, run it, then inspect its layers with docker history.

Key commands: FROM python:3.12-slim · » `EXPOSE` · docker build · docker run · docker history

https://killercoda.com/playgrounds/scenario/ubuntu

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · COMMANDS

Build a Container Image with Docker

Step 1 — Write the Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
EXPOSE 8080
CMD ["python", "app.py"]
```

Note EXPOSE is documentation only — it does not publish the port. -p host:container is what actually opens it.

Step 2 — Build and tag the image

```
docker build -t ckad/hello:1.0 .
docker images ckad/hello
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · COMMANDS

Build a Container Image with Docker (cont.)

Step 3 — Run and test the container

```
docker run -d --name hello -p 8080:8080 ckad/hello:1.0
curl http://localhost:8080
```

Step 4 — Inspect the image layers

```
docker history ckad/hello:1.0
docker rm -f hello && docker rmi ckad/hello:1.0
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · TEST IT

Build a Container Image with Docker

✅ Test it

curl http://localhost:8080 returns hello from CKAD 2026 lab 1, and docker history shows a distinct layer for each Dockerfile instruction.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

Multi-Stage Dockerfile

LAB 2

Container images

Separate the build stage from the runtime stage to produce images that are 10x smaller and contain no compiler toolchain. Multi-stage builds are a CKAD staple — you must be able to write one from scratch and explain why it shrinks the image.

You'll build

Build a single-stage baseline, then a multi-stage image with COPY --from and distroless, and compare sizes.

Key commands: FROM golang:1.22 · docker build · » Only

https://killercoda.com/playgrounds/scenario/ubuntu

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · COMMANDS

Multi-Stage Dockerfile

Step 1 — Single-stage baseline (large image)

```
FROM golang:1.22
WORKDIR /src
COPY main.go .
RUN go mod init demo && go build -o app main.go
CMD ["./app"]
```

Step 2 — Multi-stage Dockerfile (small image)

```
# Stage 1: compile
FROM golang:1.22 AS builder
WORKDIR /src
COPY main.go .
RUN go mod init demo && CGO_ENABLED=0 go build -o /out/app main.go

# Stage 2: runtime only
FROM gcr.io/distroless/static-debian12
COPY --from=builder /out/app /app
ENTRYPOINT ["/app"]
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · COMMANDS

Multi-Stage Dockerfile (cont.)

Step 3 — Build both and compare sizes

```
docker build -f Dockerfile.single -t demo:single .
docker build -f Dockerfile.multi  -t demo:multi .
docker images | grep demo        # ~800MB  vs  ~8MB
```

Note Only the last FROM stage ends up in the final image. CGO_ENABLED=0 produces a static binary that runs in distroless/scratch — no shell, smaller attack surface.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · TEST IT

Multi-Stage Dockerfile

✅ Test it

docker images | grep demo shows demo:multi roughly 100x smaller than demo:single, and demo:multi still serves hello from multi-stage build.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

CONTAINER IMAGES

Certified

Kubernetes

- Application
- Developer
- (CKAD)

SECTION

Kubernetes High-Level Architecture

Declarative Configurations

- YAML

- • Lines and white spaces
- delimiters

GENERATE → EDIT → APPLY

The CKAD kubectl Workflow

Generate

- kubectl run / create
- with --dry-run=client -o yaml
- Scaffold a manifest fast

Edit

- Redirect to a .yaml file
- Tweak labels, image, command
- Version-controllable

Apply

- kubectl apply -f file.yaml
- Idempotent & repeatable
- kubectl describe to verify

© 2026 Tertiary Infotech Academy Pte Ltd

SET THESE ON EXAM DAY

Exam-Speed kubectl (kubernetes.io quick reference)

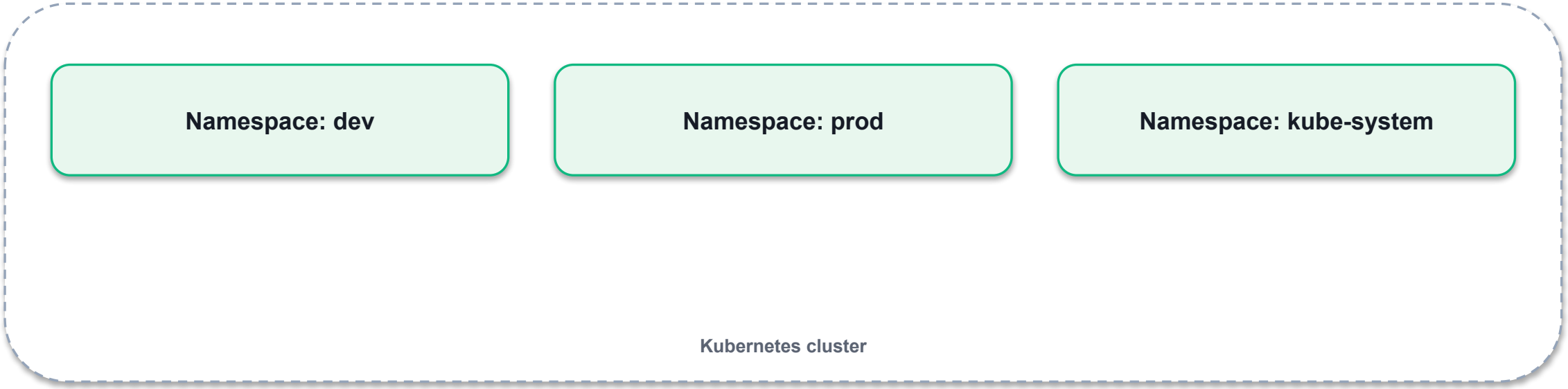
- alias k=kubectl — set this first; saves keystrokes on every command.
- export do="--dry-run=client -o yaml" — scaffold any manifest without creating it.
- source <(kubectl completion bash); complete -o default -F __start_kubectl k — tab-completion.
- kubectl get -o wide / -o yaml / -o jsonpath · kubectl describe · kubectl explain — inspect & learn.

© 2026 Tertiary Infotech Academy Pte Ltd

PODS

Pods and Namespaces

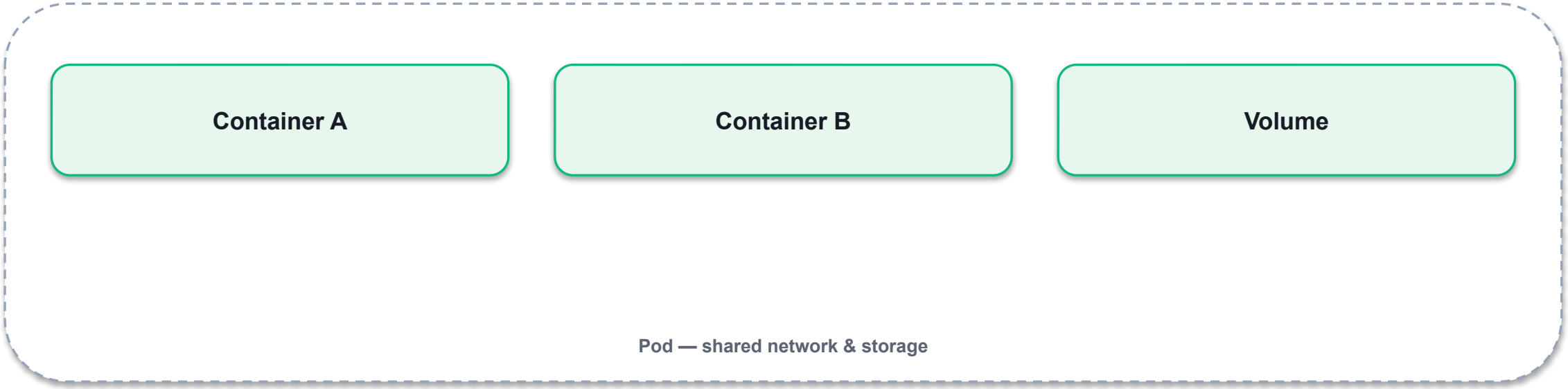
Running and inspecting workload, organizing objects



PODS

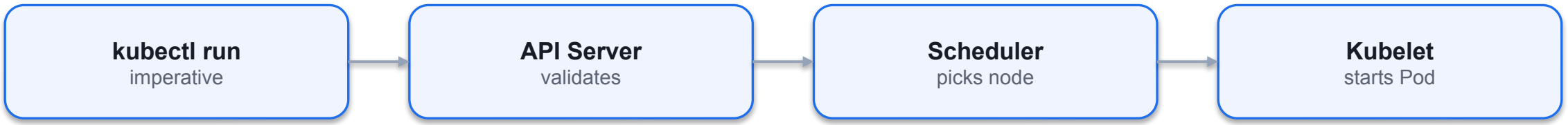
What is a Pod?

The smallest, most basic deployable objects in Kubernetes.



Creating a Pod with Imperative Command

Declarative configurations are defined using YAML file



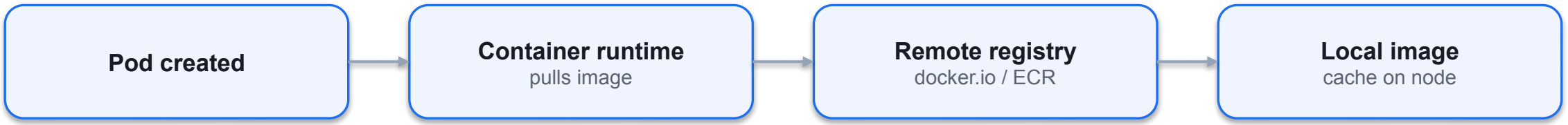
SECTION

Pods - Declarative

PODS

Container Image Retrieval

Downloads container image from registry upon Pod creation



SECTION

Pod Life Cycle Phases Indicates operational status of Pod

PODS

Troubleshooting Pods and Containers

Techniques for identifying the root cause of a failure

kubectl get pod
status & restarts

kubectl describe
events

kubectl logs
stdout / stderr

kubectl logs -p
previous container

kubectl exec
shell into it

kubectl events
cluster-wide

Retrieving High-Level Pod Information

Check the status of a Pod first

```
$ kubectl get pods -n t67
NAME READY   STATUS   RESTARTS AGE
misbehaving-pod 0/1 ErrImagePull   0   2s
$ kubectl get pods -n k31
NAME READY   STATUS   RESTARTS AGE
 Failed to pull
successful-pod   1/1 Running   0   5s
 container image
 Looks successful on
 the surface-level
```

PODS

Common Pod Error Statuses

Poorly-documented in Kubernetes docs, encountered often in practice

- Status Root Cause Potential Fix
- ImagePullBackOff or Image could not be Check correct image name, check that image name exists
- ErrImagePull pulled from in registry, verify network access from node to registry,
- registry. ensure proper authentication.
- CrashLoopBackOff Application or Check command executed in container, ensure that
- command run in image can properly execute (e.g., by creating a container
- container crashes. with Docker Engine).
- CreateContainerConfigError ConfigMap or Check correct name of the configuration object, verify the
- Secret referenced existence of the configuration object in the namespace.
- by container
- cannot be found.

Inspecting Events of a Specific Pod

Detailed event information can be found in Events section

```
$ kubectl describe pod myapp
...
Events:
 Type    Reason    Age    From    Message
 ----    ------    ----    ----    -------
 Normal    Scheduled    <unknown>    default-scheduler Successfully
 assigned default/secret-pod to minikube
 Warning    FailedMount    3m15s    kubelet, minikube    Unable to attach or
 mount volumes: unmounted volumes=[mysecret], unattached
 volumes=[default-token-bf8rh mysecret]: timed out waiting for the condition
 Warning    FailedMount    68s (x10 over 5m18s)    kubelet, minikube
 MountVolume.SetUp failed for volume "mysecret" : secret "mysecret" not found
 ...
```

Inspecting Events Across all Pods

Lists events for all Pods in the given namespace

```
$ kubectl get events
LAST SEEN TYPE REASON   OBJECT   MESSAGE
3m14s   Warning   BackOff   pod/custom-cmd   Back-off restarting
 failed container
 cronjob/google-ping
2s   Warning   FailedNeedsStart   Cannot determine if
 job needs to be
 started: too many
 missed start time
 (> 100). Set or
 decrease .spec.
 startingDeadline
 Seconds or check
 clock skew
```

PODS

Inspecting Container Logs

Application failure may not necessarily surface on the Kubernetes-level

```
$ kubectl create -f crash-loop-backoff.yaml
pod/incorrect-cmd-pod created
$ kubectl get pods incorrect-cmd-pod
NAME   READY   STATUS   RESTARTS   AGE 5
incorrect-cmd-pod   0/1   CrashLoopBackOff
3m20s
$ kubectl logs incorrect-cmd-pod
/bin/sh: unknown: not found
```

Opening an Interactive Shell to a Container

Allows for exploring container file system and processes

```
$ kubectl logs failing-pod
/bin/sh: can't create /root/tmp/curr-date.txt: nonexistent directory
$ kubectl exec failing-pod -it -- /bin/sh
# mkdir -p
~/tmp # cd
~/tmp
# ls
-l
total
-rw-r--r-- 1 root root 112 May 9 23:52
curr-date.txt # exit
```

Distroless Containers

Some container images do not provide a shell

```
$ kubectl run minimal-pod --image=k8s.gcr.io/pause:3.1
pod/minimal-pod created
$ kubectl exec minimal-pod -it -- /bin/sh
OCI runtime exec failed: exec failed: container_linux.go:349:
Starting container process caused "exec: \"/bin/sh\": stat
/bin/sh: no such file or directory": unknown
command terminated with exit code 126
```

PODS

Debugging Using an Ephemeral Container

Using a disposable container for troubleshooting distroless containers

```
  $ kubectl debug -it minimal-pod --image=busybox
  Defaulting debug container name to debugger-jf98g.
  If you don't see a command prompt, try pressing enter.
  / # pwd
  /
  / # exit
  Session ended, resume using 'kubectl attach minimal-pod -c
  debugger-jf98g -i -t' command when the pod is running
You can create a copy of a container in a new Pod with --copy-to for more
complex debugging scenarios.
```

Copying a File from the Container

The tar binary needs to exist in container

```
$ kubectl exec -it nginx -n code-red -- /bin/sh
# tar cvf error-log.tar
/var/log/nginx/error.log # exit
$ kubectl cp code-red/nginx:error-log.tar error-log.tar
$ ls
error-log.tar
 File to be copied
 Namespace
 Pod
```

Accessing Container Logs

Renders logs produced by application or process running in a container

```
$ kubectl logs hazelcast
...
May 25, 2020 3:36:26 PM com.hazelcast.core.LifecycleService
INFO: [10.1.0.46]:5701 [dev] [4.0.1] [10.1.0.46]:5701 is STARTED
```

- You can stream the logs with the command line option -f. This option is helpful if you want to see logs in real time.
- You can still get back to the logs of the previous container by adding the -p command line option in case the container crashes or is restarted.

Executing a Command in a Container

Interactive exploration of a container, or seeing the direct effect of a command

```
$ kubectl exec -it hazelcast -- /bin/sh
# ...
$ kubectl exec hazelcast -- env
...
DNS_DOMAIN=cluster
```

- The command line option -it opens an interactive shell. This is useful if you want to inspect the configuration of your application or debug the current state of your application.
- For direct execution of a command in the container simply append to --.

Creating a Temporary Pod

Meant for experimentation, deleted after use

```
$ kubectl run busybox
 --image=busybox
 --rm   Removes Pod after executing
 the provided command
 -it
 --restart=Never
 -- wget 10.1.0.41
Connecting to 10.1.0.41:80 (10.1.0.41:80)
...
pod "busybox" deleted
```

Declaring Environment Variables

Define zero to many key-value pairs

```
apiVersion: v1
kind: Pod
metadata:
 name: spring-boot-app
spec:
 containers:
 - image: bmuschko/spring-boot-app:1.5.3
 env: spring-boot-app
 name:   Typical naming conventions
 - name: SPRING_PROFILES_ACTIVE
 value: prod   for keys are not enforced
 - name: VERSION
 value: '1.5.3'
```

PODS

Declaring Command and Args

Assign or overwrite container entrypoint and arguments

```
apiVersion: v1
kind: Pod
metadata:
 name: spring-boot-app
spec:
 containers:
 - image: bmuschko/spring-boot-app:1.5.3
 name: spring-boot-app
 command: ["/bin/sh"]
 args: ["-c", "while true; do date; sleep 10; done"]
```

Deleting a Pod

Graceful deletion of workload can take up to 30 seconds

```
$ kubectl delete pod hazelcast
pod "hazelcast" deleted
$ kubectl delete -f pod.yaml
pod "hazelcast" deleted
$ kubectl delete -f pod.yaml --now
pod "hazelcast" deleted
```

PODS

What is a Namespace?

Organizes and isolates Kubernetes object that belong together

- ● An API construct to avoid naming collisions and represent a scope for object names.
- A good use case for namespaces is to isolate the objects by team or responsibility.
- ● The default namespace hosts object that haven’t been assigned to an
- explicit namespace.
- ● Namespaces starting with the prefix kube- are not considered end
- user-namespaces. They have been created by the Kubernetes system. You will not
- have to interact with them as an application developer.
- ● Reference a namespace with the command line option --namespace or -n.

PODS

Creating a Namespace with Imperative Command

"Organize objects in namespace called code-red"

```
$ kubectl create namespace code-red
namespace/code-red created
$ kubectl run nginx --image=nginx --restart=Never -n code-red
pod/nginx created
$ kubectl get pods -n code-red
NAME   READY   STATUS   RESTARTS AGE
nginx 1/1 Running 0   13s
```

PODS

Listing Namespaces

Includes default, Kubernetes-internal, and custom namespaces

```
$ kubectl get namespaces
NAME   STATUS   AGE
default   Active   157d
kube-node-leas   Active   157d
e kube-public   Active   157d
kube-system   Active   157d
```

PODS

Namespace YAML Manifest

"Organize objects in namespace called code-red"

```
apiVersion: v1
kind: Namespace
metadata:
 name: code-red
```

Deleting a Namespace

Cascade-deletes containing objects

```
$ kubectl delete namespace code-red
namespace "code-red" deleted
$ kubectl get pods -n code-red
No resources found in code-red namespace.
```

Create and Manage Pods

LAB 3 Pods

The Pod is the smallest schedulable unit in Kubernetes. Create Pods imperatively, generate YAML with --dry-run, edit live manifests, and use the exam-critical $do alias that saves 30+ seconds per question.

You'll build

Set exam-speed aliases, run Pods imperatively, scaffold YAML with $do, then apply and inspect.

Key commands: alias k=kubectl · k run · k describe · » `--restart=Never`

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · COMMANDS

Create and Manage Pods

Step 1 — Set exam-speed aliases (run these first on exam day)

```
alias k=kubectl
export do="--dry-run=client -o yaml"
```

Step 2 — Create a Pod imperatively

```
k run web --image=nginx:1.25 --port=80
k get pod web -o wide          # shows node and Pod IP
```

Step 3 — Generate a manifest without creating it

```
k run web2 --image=nginx:1.25 --port=80 $do > web2.yaml
k apply -f web2.yaml
k get pod web2 --show-labels
```

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

Create and Manage Pods (cont.)

Step 4 — Inspect a running Pod

```
k describe pod web
k get pod web -o jsonpath='{.status.podIP}'; echo
```

Step 5 — Override the container command

```
k run sleeper --image=busybox --restart=Never -- sh -c 'sleep 3600'
k delete pod web web2 sleeper --force --grace-period=0
```

Note --restart=Never creates a bare Pod (not a Deployment). Everything after -- becomes the container command. --force --grace-period=0 skips the 30s grace period.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · TEST IT

Create and Manage Pods

✅ Test it

k get pod web2 --show-labels shows the tier=frontend label, and the --dry-run=client -o yaml workflow produces a valid manifest you can apply.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

RUN TO COMPLETION

Batch Workloads in Kubernetes

- A Job runs Pods until a set number of successful completions, then stops — it does not restart on success.
- A CronJob creates Jobs on a schedule using standard cron syntax (*/1 * * * * = every minute).
- Job Pod templates must use restartPolicy: Never or OnFailure — never Always.
- Trigger a CronJob now with: kubectl create job --from=cronjob/<name> <job-name>.

© 2026 Tertiary Infotech Academy Pte Ltd

EXAM-TESTED EVERY SITTING

Key Fields to Memorise

Job

- completions — successes needed
- parallelism — Pods at once
- backoffLimit — failures allowed

Job (timing)

- activeDeadlineSeconds
- Hard wall-clock ceiling
- Overrides backoffLimit

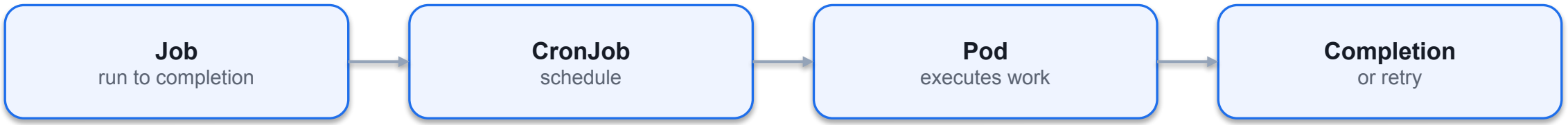
CronJob

- schedule + timeZone
- concurrencyPolicy: Allow/Forbid/Replace
- successful/failedJobsHistoryLimit

© 2026 Tertiary Infotech Academy Pte Ltd

Jobs and CronJobs

One-time operations and scheduled operations



What is a Job?

Kubernetes primitive for executing a one-time operation

- ● It runs a specific task until a specified number of completions has been reached.
- ● The actual work managed by a Job is still running inside of a Pod. Therefore, you
- can think of a Job as a higher-level coordination instance for Pods executing the
- workload.
- ● Upon completion of a Job and its Pods, Kubernetes does not automatically delete
- the objects - they will stay until they’re explicitly deleted. Keeping those objects
- helps with debugging the command run inside of the Pod and gives you a chance to
- inspect the logs.

SECTION

Job Object Hierarchy A Job manages Pods to run the workload

Creating a Job with Imperative Command

"Increment a counter and render its value on the terminal"

```
$ kubectl create
 job counter
 --image=nginx:1.24.0
 -- /bin/sh -c 'counter=0; while [ $counter -lt 3 ]; do
 counter=$((counter+1)); echo "$counter"; sleep 3; done;'
job.batch/counter created
```

Job YAML Manifest

job-definition.yaml pod-definition.yaml

```
apiVersion: batch/v1   apiVersion: v1
kind: Job   kind: Pod
metadata:   metadata:
 name: calculate-pod-job   name: calculate-pod
spec:   spec:
 template:   containers:
 spec:   - image: ubuntu
 containers:   name: calculate-pod
 - image: ubuntu   command: [‘expr’,’2’, ‘+’,’1’]
 name: calculate-pod   restartPolicy: Never
 command: [‘expr’,’2’, ‘+’,’1’]
 restartPolicy: Never
```

Inspecting a Job

Job is completed when configured # of executions has been reached

```
$ kubectl get jobs
NAME   COMPLETION   DURATION AGE
counte   S   13s 13s
r   0/1
$ kubectl get jobs
NAME   COMPLETION   DURATION AGE
counte   S   15s 19s
r   1/1
$ kubectl get pods
NAME   READY   STATUS   RESTARTS AGE
counter-z6kd   0/1   Complete   0   51s
j   d
```

Inspecting Pod Logs

Workload can be inspected during and after execution

```
$ kubectl logs counter-z6kdj
```

Job Operation Types

Execution runtime behavior of workload can be fine-tuned

- ● The default behavior of a Job is to run the workload in a single Pod and expect one
- successful completion (non-parallel Job).
- ● The attribute spec.completions controls the number of required
- successful completion.
- ● The attribute spec.parallelism allows for executing the workload by
- multiple pods in parallel.

Restart Behavior

Re-running workload if execution is unsuccessful

- ● The attribute spec.backoffLimit determines the number of retries a Job
- attempts to successfully complete the workload until the executed command finishes
- with an exit code 0. The default is 6, which means it will execute the workload 6
- times before the Job is considered unsuccessful.
- ● The attribute spec.template.spec.restartPolicy needs to be
- declared explicitly. The restart policy of a Job can only be OnFailure or
- Never.

Restarting a Container on Failure

Restart container if it exits with a non-zero exit code

- spec.template.spec.restartPolicy: OnFailure

Starting a New Pod on Failure

The policy does not restart the container upon a failure, it starts a new Pod.

- spec.template.spec.restartPolicy: Never

Jobs — Run-to-Completion Workloads

LAB 4

Batch workloads

A Job runs Pods until a required number of successful completions is reached, then stops. CKAD tests completions, parallelism, backoffLimit and activeDeadlineSeconds in almost every sitting — you must write a Job manifest from memory.

You'll build

Create a one-shot Job, a parallel Job with completions, and explore backoffLimit and activeDeadlineSeconds.

Key commands: k create · apiVersion: batch/v1 · k apply · » `restartPolicy:

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · COMMANDS

Jobs — Run-to-Completion Workloads

Step 1 — One-shot Job (imperative)

```
k create job hello --image=busybox -- echo "hello CKAD 2026"
k logs -l job-name=hello
```


© 2026 Tertiary Infotech Academy Pte Ltd

Jobs — Run-to-Completion Workloads (cont.)

Step 2 — Parallel Job with multiple completions

```
apiVersion: batch/v1
kind: Job
metadata:
  name: pi-parallel
spec:
  completions: 5
  parallelism: 2
  backoffLimit: 4
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: pi
        image: perl:5.34
        command: ["perl", "-Mbignum=bpi", "-wle", "print bpi(50)"]
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · COMMANDS

Jobs — Run-to-Completion Workloads (cont.)

Step 3 — Apply and watch completions climb

```
k apply -f parallel.yaml
k get job pi-parallel -w        # COMPLETIONS ticks 0/5 → 5/5
```

Note restartPolicy: Never is mandatory in Job Pod templates. activeDeadlineSeconds is a hard wall-clock ceiling — it overrides backoffLimit.

© 2026 Tertiary Infotech Academy Pte Ltd

What is a CronJob?

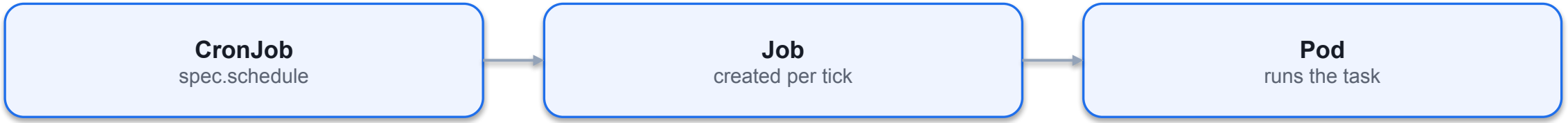
Kubernetes primitive for executing workload periodically based a schedule

- A CronJob is essentially a Job with similar characteristics.
- The schedule can be defined with a cron-expression you may already know from
- Unix cron jobs.

JOBS & CRONJOBS

CronJob Object Hierarchy

A CronJob manages Jobs which delegate the workload to Pods



Creating a CronJob with Imperative Command

"Render the current date on standard output every hour"

```
$ kubectl create
 cronjob current-date
 --schedule="* * * * *"
 --image=nginx:1.24.0
 -- /bin/sh -c 'echo "Current date: $(date)"'
cronjob.batch/current-date created
```

CronJob YAML Manifest

"Increment a counter and render its value on the terminal"

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: counter
spec:
  The crontab expression used
  schedule: "*/1 * * * *"
  jobTemplate:   to run CronJob periodically
  spec:
  template:
  spec:
  restartPolicy: Never   Run in a new Pod
  containers:
  - args:
```

Inspecting a CronJob

Keeps a history of failed and successful Pods

```
$ kubectl get cronjobs
NAME   SCHEDULE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
counter   */1 * * * *   False   0   26s   1h
$ kubectl get jobs --watch
NAME   COMPLETIONS   DURATION   AGE
counter-1557334380   1/1   3s   2m24s
counter-1557334440   1/1   3s   84s
counter-1557334500   1/1   3s   24s
```

Configuring Retained History

Being able to know what happened during workload execution retroactively

- ● Even after completing a task in a Pod controlled by a CronJob, it will not be deleted
- automatically. Keeping a historical record of Pods can be tremendously helpful for
- troubleshooting failed workloads or inspecting the logs.
- ● The attribute spec.successfulJobsHistoryLimit configures the number
- of successful executions. Defaults to 3 successful Jobs.
- ● The attribute spec.failedJobsHistoryLimitconfigures the number of failed
- executions. Defaults to 1 failed Job.

CronJobs — Scheduled Workloads

LAB 5 Batch workloads

A CronJob creates Jobs on a time-based schedule using standard cron syntax. CKAD tests concurrencyPolicy, timeZone, startingDeadlineSeconds, history limits, manual triggers and suspend/resume — these fields appear verbatim in exam questions.

You'll build

Create a CronJob imperatively and declaratively, suspend/resume it, then trigger it manually.

Key commands: k create · apiVersion: batch/v1 · k patch · » `concurrencyPolicy`:

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · COMMANDS

CronJobs — Scheduled Workloads

Step 1 — Create a CronJob imperatively

```
k create cronjob date-printer --image=busybox \
  --schedule="*/1 * * * *" -- sh -c "date; echo from cronjob"
k get cronjob date-printer
```


© 2026 Tertiary Infotech Academy Pte Ltd

CronJobs — Scheduled Workloads (cont.)

Step 2 — Declarative CronJob with all key fields

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: report
spec:
  schedule: "*/2 * * * *"
  timeZone: "Asia/Singapore"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 2
  startingDeadlineSeconds: 30
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers:
          - name: report
            image: busybox
            command: ["sh", "-c", "echo report at $(date)"]
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · COMMANDS

CronJobs — Scheduled Workloads (cont.)

Step 3 — Suspend, resume and trigger manually

```
k patch cronjob report -p '{"spec":{"suspend":true}}'    # pause
k patch cronjob report -p '{"spec":{"suspend":false}}'   # resume
k create job --from=cronjob/report report-manual           # run now
```

Note concurrencyPolicy: Allow (default) / Forbid / Replace. timeZone is a newer field — always set it. create job --from=cronjob/<name> runs the workload immediately.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · TEST IT

CronJobs — Scheduled Workloads

✅ Test it

k get cronjob report shows SUSPEND True after the patch, and k create job --from=cronjob/report report-manual produces an immediate one-off run.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

Why More Than One Container?

- Containers in a Pod share the network namespace and can share volumes (emptyDir is the usual glue).
- Sidecar — a helper alongside the main app (log shipper, proxy) sharing a volume.
- Init container — runs to completion before main containers (seed a volume, wait for a service).
- Always target a specific container with kubectl logs -c <name> and kubectl exec -c <name>.

© 2026 Tertiary Infotech Academy Pte Ltd

ONE POD, MANY ROLES

Container Patterns

Sidecar

- Runs beside the main app
- Shares an emptyDir volume
- Log shipping / proxy / sync

Init container

- Runs first, to completion
- Pod shows Init:N/M
- Seed volume · wait for DNS

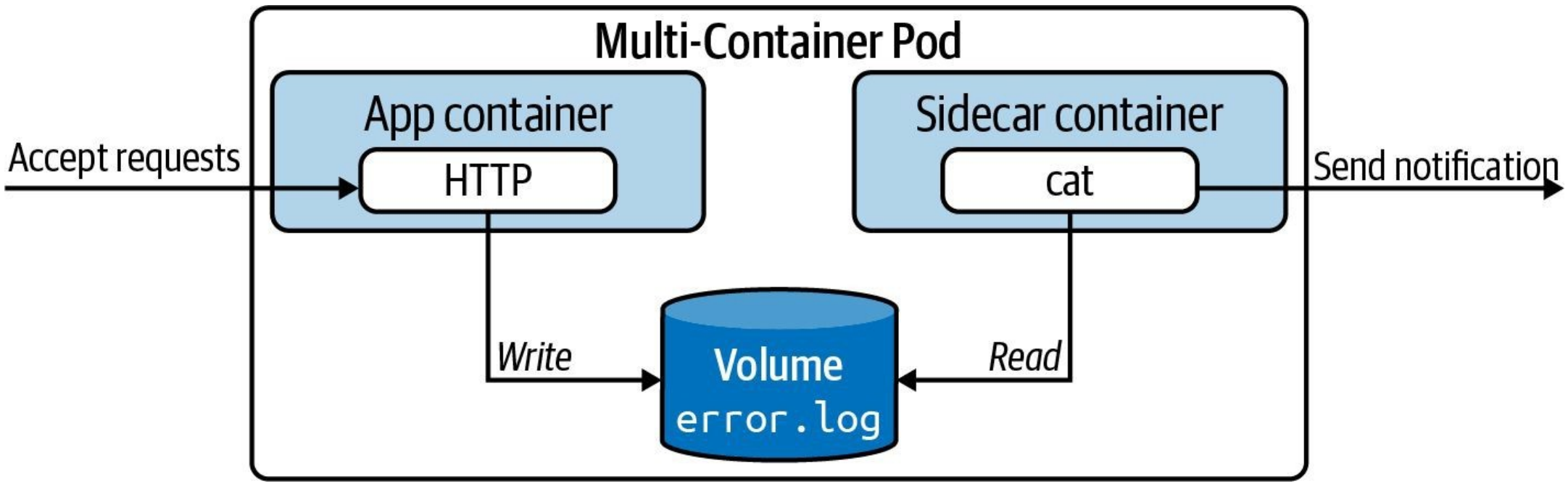
Native sidecar

- initContainer + restartPolicy: Always
- Kubernetes 1.29+
- Runs for the Pod's lifetime

© 2026 Tertiary Infotech Academy Pte Ltd

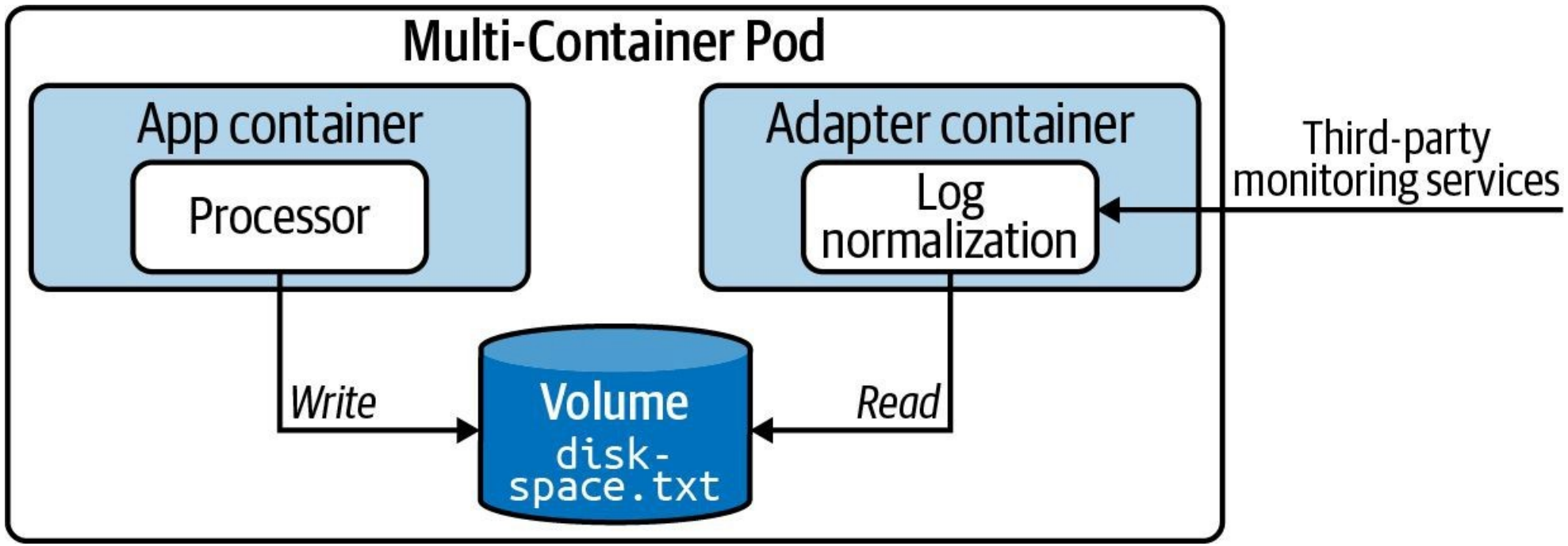
Sidecar Pattern

"Send a notification of error log contains entries"



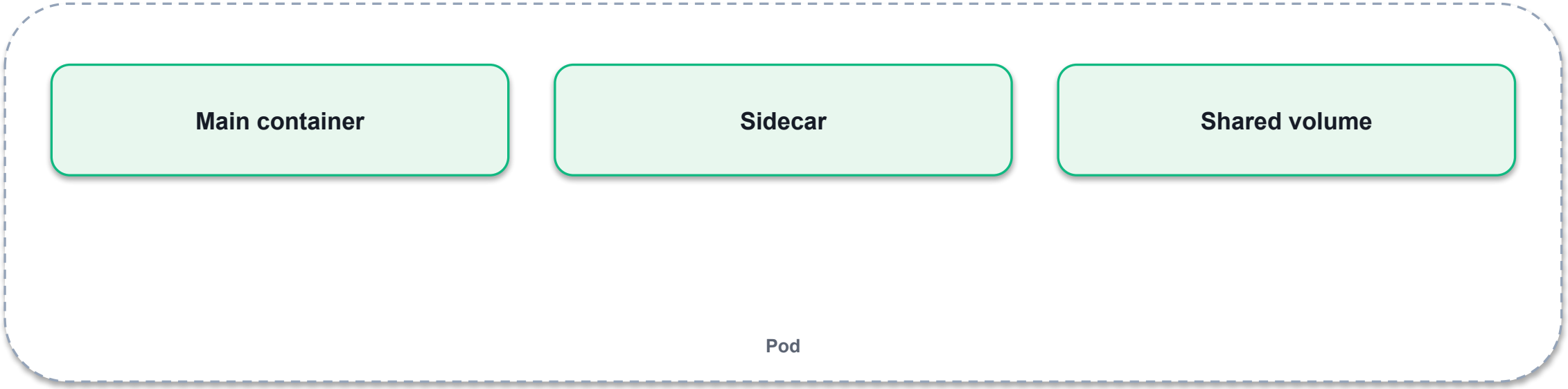
Adapter Pattern

"Transform log data to JSON to make it consumable by external system"



Multi-Container Pods

Defining and interacting with multiple containers, design patterns



MULTI-CONTAINER PODS

Defining Multiple Containers in a Pod

There are warranted use cases for wanting to run more than a single container

- ● The general guideline is to operate a microservice in a single container per Pod
- which promotes a decentralized, decoupled, and distributed architecture.
- ● Specific use cases call for running multiple containers per Pod. For example, you
- may want to provide helper functionality that runs alongside the application. In other
- cases, you may want to initialize your Pod by executing setup scripts, commands, or
- any other kind of preconfiguration procedure before the application container should
- start.

Design Patterns

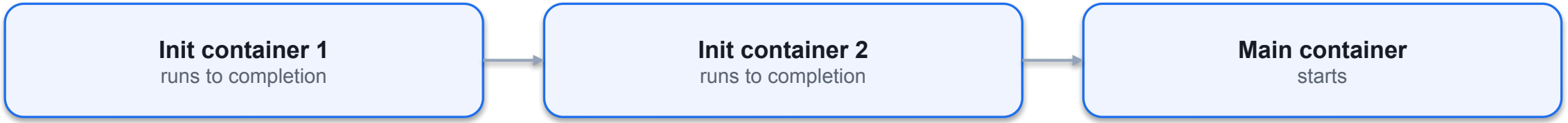
Emerged from Pod requirements

- Init Container Sidecar
- ● Provide initialization logic concerns to be ● The sidecars are not part of the main traffic
- run before the main application even starts. or API of the primary application and
- ● Example: Downloading a configuration files operate asynchronously.
- required by application. ● Example: Watcher capabilities.
- Adapter Ambassador
- ● Transforms the output produced by the ● Provides a proxy for communicating with
- application to make it consumable in the external services to hide and/or abstract
- format needed by another part of the the complexity.
- system. ● Example: Rate-limiting functionality for
- ● Example: Massaging log data. HTTP(S) calls to an external service.

MULTI-CONTAINER PODS

Init Container Life Cycle

Initialization logic before main application containers



Defining an Init Container

Declared adjacent to main application containers with spec.initContainers

```
apiVersion: v1
kind: Pod
metadata:
 name: multi-container
spec:
 initContainers:
 Uses the same attributes as
 - image: init:3.2.1
 name: app-initializer   main application containers
 containers:
 - image: nginx
 name: web-server
```

Creating a Pod with an Init Container

Init container execution renders in status column

```
$ kubectl create -f init.yaml
pod/business-app created
$ kubectl get pod business-app
NAME   READY   RESTART
AGE   STATUS   S
 0/1 Init:0/1 0   2s
business-ap
p
$ kubectl get pod business-app
NAME   READY   STATUS   RESTARTS AGE
business-ap   1/1   Runnin   0   8s
p   g
```

Defining More Than One Main Application Container

Add another container definition under spec.containers

```
apiVersion: v1
kind: Pod
metadata:
  name: multi-container
spec:
  containers:
  - image: nginx:1.13.0
  name: web-server
  - image: transformer:1.0.0   Defines a second container
  name: helper   running alongside the
  application container
```

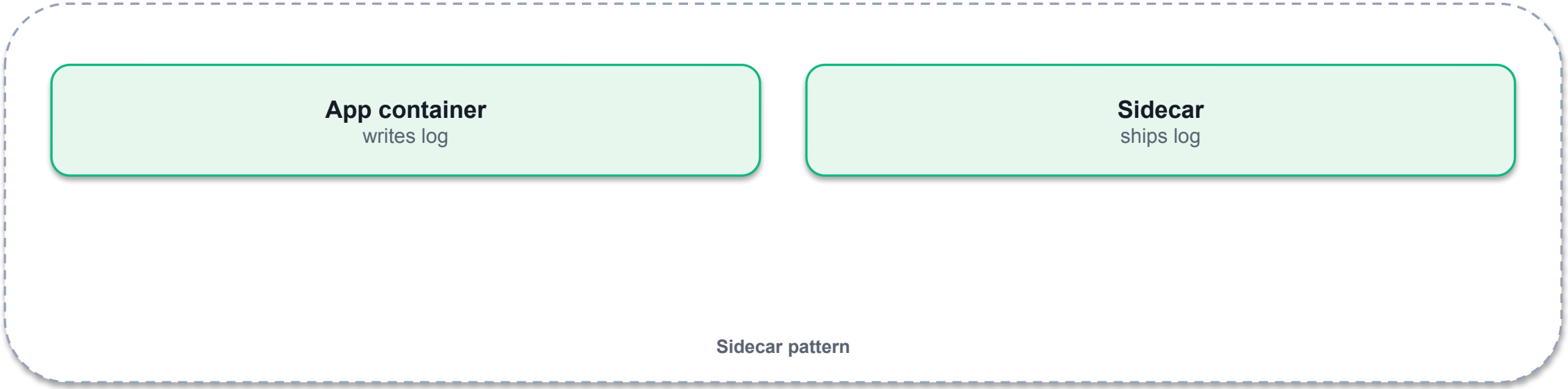
Creating a Pod with Multiple Application Containers

The ready column indicates the number of started containers

```
$ kubectl create -f multi-container.yaml
pod/multi-container created
$ kubectl get pod business-app
NAM   READY   STATUS  RESTARTS
E
 Running 0   2s
AGE
multi-container 2/2
```

Sidecar Pattern

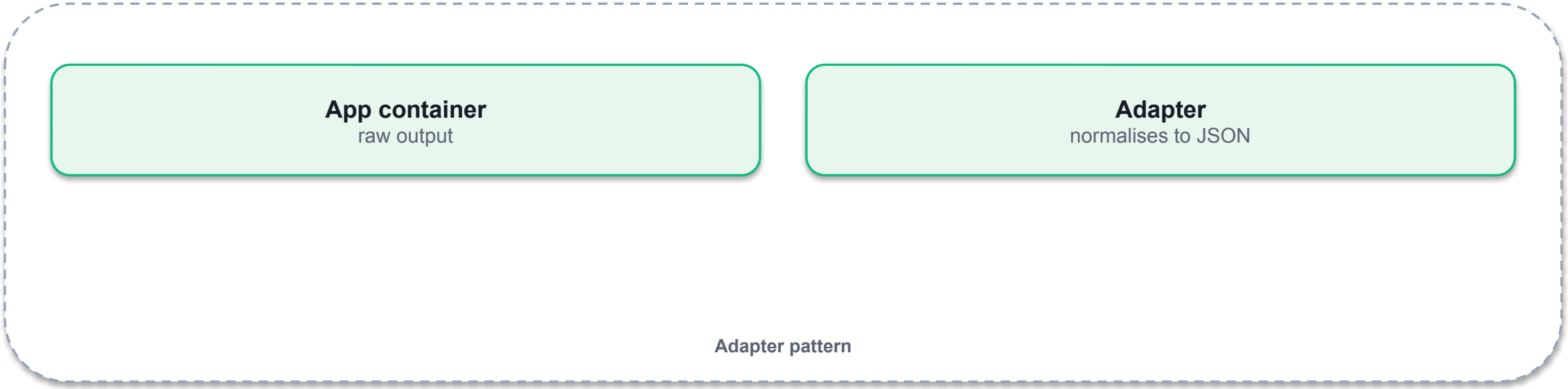
"Send a notification of error log contains entries"



MULTI-CONTAINER PODS

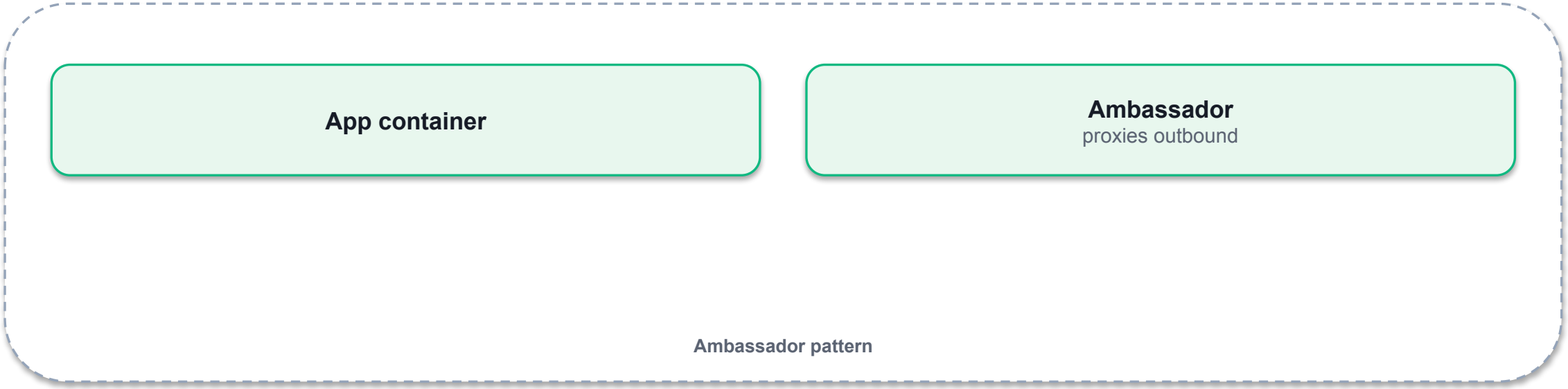
Adapter Pattern

"Transform log data to JSON to make it consumable by external system"



Ambassador Pattern

"OAuth authentication handled by network communication proxy"



Multi-Container Pods — Sidecar Pattern

LAB 6 Multi-container Pods

Containers in the same Pod share a network namespace and can share volumes. CKAD tests the sidecar pattern (a helper reads/writes a shared volume) and the native sidecar feature (Kubernetes 1.29+). You must exec into and read logs from each container.

You'll build

Run a sidecar that tails a shared emptyDir log, read each container's logs, then build a native sidecar.

Key commands: apiVersion: v1 · k logs · initContainers: · » With

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · COMMANDS

Multi-Container Pods — Sidecar Pattern (cont.)

Step 2 — Read logs from each container separately

```
k logs app-with-sidecar -c app | head -5
k logs app-with-sidecar -c log-shipper | head -5
k exec -it app-with-sidecar -c app -- sh -c 'wc -l /var/log/app.log'
```

Step 3 — Native sidecar container (Kubernetes 1.29+)

```
initContainers:
- name: log-collector
  image: busybox
  restartPolicy: Always
  command: ["sh","-c","while true; do echo alive; sleep 5; done"]
```

Note With multiple containers in a Pod, always specify -c <name> for logs and exec. emptyDir is the standard glue volume between sidecar and main containers.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · TEST IT

Multi-Container Pods — Sidecar Pattern

✅ Test it

k logs app-with-sidecar -c log-shipper shows the same lines app is writing, proving both containers share the emptyDir volume.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

Init Containers

LAB 7 Multi-container Pods

Init containers run in order, to completion, before any main container starts. Use them to seed shared volumes, wait for upstream services, or do one-time setup. CKAD asks you to write init containers and read Init:N/M Pod status correctly.

You'll build

Seed an nginx web root with an init container, then add an init container that waits for a Service via DNS.

Key commands: apiVersion: v1 · k exec · initContainers: · » The

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · COMMANDS

Init Containers

Step 1 — Init container that seeds the web root

```
apiVersion: v1
kind: Pod
metadata:
  name: web-init
spec:
  volumes:
  - name: html
    emptyDir: {}
  initContainers:
  - name: seed
    image: busybox
    command: ["sh","-c","echo '<h1>Seeded by init</h1>' > /work/index.html"]
    volumeMounts:
    - {name: html, mountPath: /work}
  containers:
  - name: web
    image: nginx:1.25
    volumeMounts:
    - {name: html, mountPath: /usr/share/nginx/html}
```


© 2026 Tertiary Infotech Academy Pte Ltd

Init Containers (cont.)

Step 2 — Verify the seeded content

```
k exec web-init -- cat /usr/share/nginx/html/index.html
```

Step 3 — Init container that waits for a Service

```
initContainers:
- name: wait-for-db
  image: busybox
  command: ["sh","-c","until nslookup db.default.svc.cluster.local; do sleep 2; done"]
```

Note The Pod stays in Init:0/1 until DNS resolves. Creating the db Service unblocks it. If an init container fails it is restarted per the Pod's restartPolicy.

© 2026 Tertiary Infotech Academy Pte Ltd

Init Containers

✅ Test it

k exec web-init -- cat /usr/share/nginx/html/index.html returns the seeded heading, proving the init container populated the volume before nginx started.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

POD-SCOPED STORAGE

Volume Types in Domain 1

emptyDir: {}

- Created with the Pod
- Shared by all containers
- Deleted with the Pod

emptyDir: Memory

- tmpfs — backed by RAM
- Fast scratch space
- Counts to memory limit

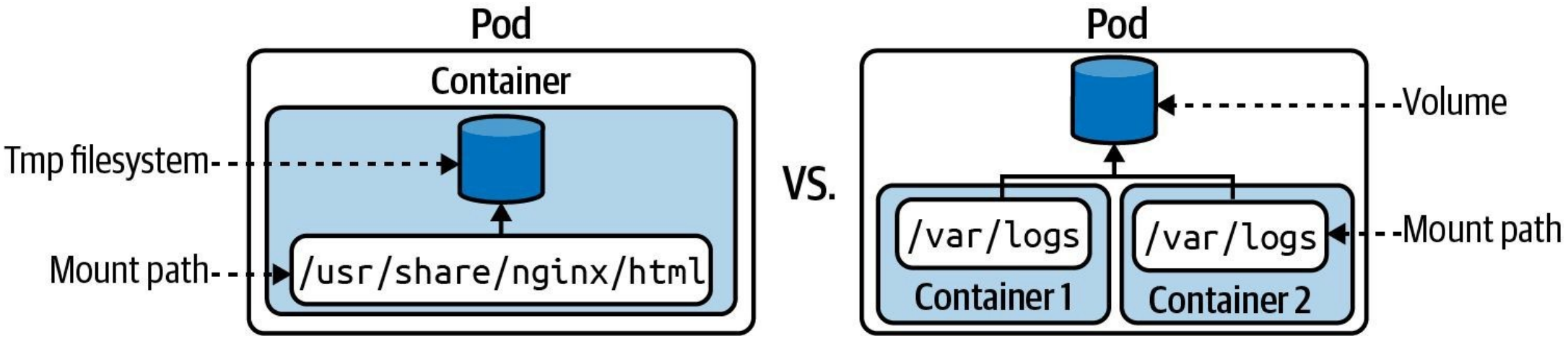
hostPath

- Mounts a node directory
- DaemonSets / node tools only
- Security risk in multi-tenant

© 2026 Tertiary Infotech Academy Pte Ltd

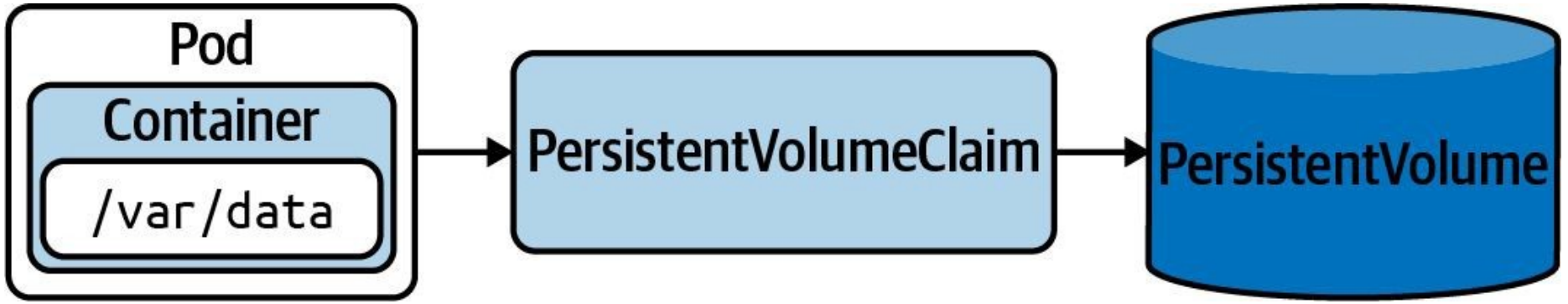
Ephemeral Volume

Useful for sharing data between containers, or for caching data



Persistent Volume

Persist data that outlives a Pod, node, or cluster restart



SECTION

Volumes Ephemeral and persistent storage for Pods

VOLUMES

What is a Volume?

A directory, possibly with some data in it accessible to the containers in a Pod

- Preferred way to store persistent data for docker containers
- Volumes are managed by Docker
- Isolated from the host file system

SECTION

Pods are ephemeral Can be stopped or destroyed

SECTION

How do you store the data?

VOLUMES

Volume Types

Determines the medium that backs the Volume and its runtime behavior

- Type Description
- emptyDir Empty directory in Pod with read/write access. Only persisted for the
- lifespan of a Pod.
- hostPath File or directory from the host node’s filesystem.
- configMap, secret Provides a way to inject configuration data.
- nfs An existing NFS (Network File System) share. Preserves data after Pod
- restart.
- persistentVolumeClaim Claims a Persistent Volume.

VOLUMES

Ephemeral Volume

- Ephemeral Volumes exist for the lifespan of a Pod. They are useful if you want to

- share data between multiple containers running in the Pod or caching of data.

VOLUMES

Defining an Ephemeral Volume

Provide type and assign mount path per container

```
apiVersion: v1
kind: Pod
metadata:
 name:
 my-container
spec:
 volumes:
 - name: logs-volume   Define Volume name and type
 emptyDir: {}
 containers:
 - image: nginx
 name:
 volumeMounts:
 my-container
```

Using a Volume

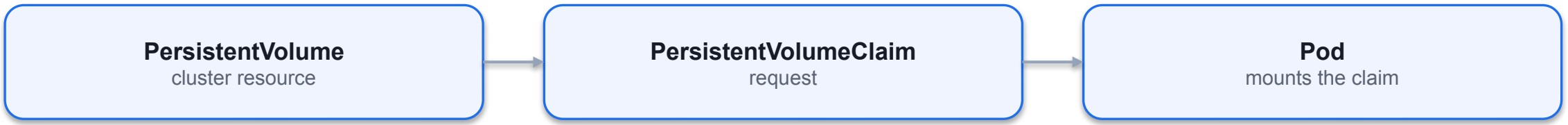
You can shell into container and navigate to mount path

```
$ kubectl create -f pod-with-vol.yaml
pod/my-container created
$ kubectl exec -it my-container -- /bin/sh
# cd /var/logs   Mount path became
#   accessible in container
pwd
/var/logs
# touch
app-logs.txt # ls
app-logs.txt
```

VOLUMES

Persistent Volume

Persist data that outlives a Pod, node, or cluster restart



VOLUMES

Static vs. Dynamic Provisioning

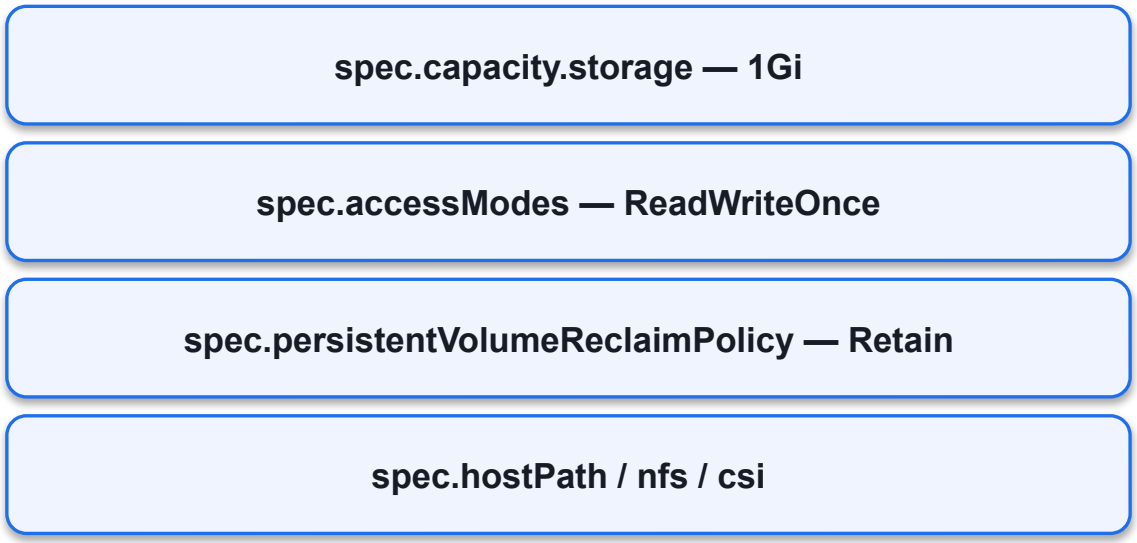
PersistentVolume object is created manually or automatically

- ● Static Provisioning: Storage device needs to be created first. The
- PersistentVolume object references the storage device and needs to be created
- manually.
- ● Dynamic Provisioning: The PersistentVolumeClaim object references a storage
- class. The PersistentVolume object is created automatically.
- ● Storage Class: Object that knows how to provision a storage device with specific
- performance requirements.

VOLUMES

Defining a PersistentVolume

Define storage capacity, access mode, and host path



VOLUMES

Access Mode

Defines read/write capabilities of Volume

- Type Description
- ReadWriteOnce Read-write access by a single node.
- ReadOnlyMany Read-only access by many nodes.
- ReadWriteMany Read-write access by many nodes.
- ReadWriteOncePod Read/write access mounted by a single Pod.

Creating a PersistentVolume

Summarized most important configuration and status

```
$ kubectl create -f db-pv.yaml
persistentvolume/db-pv
created
$ kubectl get pv db-pv
NAME   CAPACITY ACCESS MODES   RECLAIM POLICY   STATUS   ...
db-pv   1Gi   RWO   Retain   Available   ...
  The object hasn't been
  consumed by a claim yet
```

Defining a PersistentVolumeClaim

Will bind to a PersistentVolume based on resource request

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
 name: db-pvc
spec:
 accessModes:
 - ReadWriteOnce
 resources:
 requests:
 storage: 256Mi
 storageClassName: ""   Assign an empty storage class name to
 use a statically-created
 PersistentVolume
```

Creating a PersistentVolumeClaim

Binds to PersistentVolume if requirements can be fulfilled

```
$ kubectl create -f db-pvc.yaml
persistentvolumeclaim/db-pvc
created
NAME   STATUS VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
$ kubectl get pvc db-pvc
db-pvc Bound   pvc   512m   RWO   7s
Binding to the
  PersistentVolume object
  was successful
```

VOLUMES

Reclaim Policy

What should happen to PersistentVolume when Claim is deleted?

- Type Description
- Retain Default. When PVC is deleted, PV is “released” and can be reclaimed.
- Delete
- Deletion removes PV and associated storage.
- Recycle
- Deprecated. Use dynamic binding instead.

Mounting a PersistentVolumeClaim in a Pod

Use Volume type persistentVolumeClaim

```
apiVersion: v1
kind: Pod
metadata:
  name: app-consuming-pvc
spec:
  volumes:
  - name: app-storage
  persistentVolumeClaim:
  claimName: db-pvc   Reference the Volume by claim name
  containers:
  - image: alpine
  ...
  volumeMounts:
  - mountPath:
```

Assigning a StorageClass for Dynamic Provisioning

Creates PersistentVolume object automatically via storage class

```
 apiVersion: v1
 kind: PersistentVolumeClaim
 metadata:
apiVersion: storage.k8s.io/v1   name: pvc
kind: StorageClass   spec:
metadata:   accessModes:
 name: aws-ebs   - ReadWriteMany
provisioner: kubernetes.io/aws-ebs   resources:
 requests:
 storage: 256Mi
 storageClassName: aws-ebs
```

Volumes — emptyDir and hostPath

LAB 8 Volumes

Containers are ephemeral — data written to the container filesystem is lost on restart. Volumes survive restarts and can be shared between containers. CKAD tests emptyDir, emptyDir.medium: Memory and hostPath, plus mounting strategies.

You'll build

Share an emptyDir between two containers, mount a RAM-backed tmpfs volume, then mount a node directory with hostPath.

Key commands: volumes: · » Volumes

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · COMMANDS

Volumes — emptyDir and hostPath

Step 1 — emptyDir shared between two containers

```
volumes:
- name: shared
  emptyDir: {}
containers:
- name: writer
  image: busybox
  command: ["sh","-c","echo hello-shared > /data/msg.txt; sleep 3600"]
  volumeMounts: [{name: shared, mountPath: /data}]
- name: reader
  image: busybox
  command: ["sh","-c","sleep 5; cat /data/msg.txt; sleep 3600"]
  volumeMounts: [{name: shared, mountPath: /data}]
```


© 2026 Tertiary Infotech Academy Pte Ltd

Volumes — emptyDir and hostPath (cont.)

Step 2 — emptyDir backed by RAM (tmpfs)

```
volumes:
- name: fast
  emptyDir:
    medium: Memory
    sizeLimit: 64Mi
```

Step 3 — hostPath: access files on the node

```
volumes:
- name: etc
  hostPath:
    path: /etc
    type: Directory
```

Note Volumes are declared under spec.volumes and consumed via spec.containers[].volumeMounts. Avoid hostPath in multi-tenant clusters — it is a security risk; use it only for DaemonSets and node-level tools.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · TEST IT

Volumes — emptyDir and hostPath

✅ Test it

k logs scratch -c reader prints hello-shared, proving the writer and reader containers share the same emptyDir volume.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

02

DAY 2

Application Deployment

Deployments · Rollouts · Blue/Green & Canary · Helm · Kustomize · Daemon/StatefulSet (CKAD Domain 2 — 20%)

© 2026 Tertiary Infotech Academy Pte Ltd

Labels and Annotations

Labels

- Key-value pairs assigned to an object
- Must follow naming conventions
- Used to query and filter objects from the CLI or from other objects

Annotations

- Represents human-readable metadata for an object
- Not meant for querying purposes
- Reserved annotations can add runtime behaviour to objects

© 2026 Tertiary Infotech Academy Pte Ltd

Example Labels

- Three different Pods, each with multiple label key-value pairs:
  - Pod 1: tier=backend, env=prod, app=miracle
  - Pod 2: tier=frontend, env=dev, app=crawler
  - Pod 3: tier=frontend, env=staging, version=v2.1
- Labels are set at creation time or added later with kubectl label.
- Multiple labels on the same object create a unique identity for filtering.

© 2026 Tertiary Infotech Academy Pte Ltd

Assigning Labels with Imperative Approach

Assign multiple labels to a Pod

```
kubectl label pod nginx tier=backend env=prod app=miracle
pod/nginx labeled
```

List Pods with their labels

```
kubectl get pods --show-labels
NAME    ...LABELS
nginx   ...tier=backend,env=prod,app=miracle
```

Remove a label by appending '-' to its key

```
kubectl label pod nginx tier-
pod/nginx labeled
```


© 2026 Tertiary Infotech Academy Pte Ltd

Assigning Labels in a YAML Manifest

Pod manifest with three label key-value pairs

```
apiVersion: v1
kind: Pod
metadata:
  name: pod1
  labels:
    tier: backend
    env: prod
    app: miracle
spec:
  ...
```


© 2026 Tertiary Infotech Academy Pte Ltd

Recommended Labels

Well-known label keys with the app.kubernetes.io prefix

```
metadata:
  labels:
    app.kubernetes.io/name: wordpress
    app.kubernetes.io/instance: wordpress-abcxzy
    app.kubernetes.io/version: "4.9.4"
    app.kubernetes.io/managed-by: helm
    app.kubernetes.io/component: server
    app.kubernetes.io/part-of: wordpress
```


© 2026 Tertiary Infotech Academy Pte Ltd

Selecting Labels

- A label selector uses a set of criteria to query for Kubernetes objects.
- Equality-based requirements — match objects with a specific key or key=value:
  - tier=frontend (equal) • tier!=frontend (not equal)
- Set-based requirements — match objects where a key's value is in a set:
  - 'tier in (frontend, backend)' • 'version notin (v1)'
- Multiple selectors are ANDed together.

© 2026 Tertiary Infotech Academy Pte Ltd

Label Selection from the CLI

Equality-based: Pods with tier=frontend AND env=dev

```
kubectl get pods -l tier=frontend,env=dev --show-labels
NAME    ...LABELS
pod2    ...app=crawler,env=dev,tier=frontend
```

Key-only: Pods that have a 'version' label (any value)

```
kubectl get pods -l version --show-labels
NAME    ...LABELS
pod3    ...app=crawler,env=staging,version=v2.1
```

Set-based with Boolean OR

```
kubectl get pods -l 'tier in (frontend,backend),env=dev' --show-labels
```


© 2026 Tertiary Infotech Academy Pte Ltd

Label Selection in YAML

A NetworkPolicy selects Pods via podSelector.matchLabels

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-network-policy
spec:
  podSelector:
    matchLabels:
      tier: frontend
  ...
```


© 2026 Tertiary Infotech Academy Pte Ltd

Example Annotations

- A Pod can carry any number of annotation key-value pairs:
  - commit: 866a8dc
  - author: 'Benjamin Muschko'
  - branch: 'bm/bugfix'
- Unlike labels, annotation values are unrestricted in length.
- Annotations are visible via kubectl describe and kubectl get -o yaml.

© 2026 Tertiary Infotech Academy Pte Ltd

Assigning Annotations with Imperative Approach

Assign annotations to a Pod

```
kubectl annotate pod nginx commit='866a8dc' branch='bm/bugfix'
pod/nginx annotated
```

Inspect annotations via describe

```
kubectl describe pods nginx
Annotations:  branch: bm/bugfix
              commit: 866a8dc
```

Remove an annotation by appending '-' to its key

```
kubectl annotate pod nginx commit-
pod/nginx annotated
```


© 2026 Tertiary Infotech Academy Pte Ltd

Assigning Annotations in a YAML Manifest

Pod manifest with three annotation key-value pairs

```
apiVersion: v1
kind: Pod
metadata:
  name: pod1
  annotations:
    commit: 866a8dc
    author: 'Benjamin Muschko'
    branch: 'bm/bugfix'
spec:
  ...
```


© 2026 Tertiary Infotech Academy Pte Ltd

Well-Known Annotations

Reserved annotations may influence runtime behaviour (e.g. Pod Security Standards)

```
apiVersion: v1
kind: Namespace
metadata:
  name: secured
  annotations:
    kubernetes.io/description: "Manages objects for business app."
    pod-security.kubernetes.io/warn: "baseline"
```


© 2026 Tertiary Infotech Academy Pte Ltd

Exam Essentials

- Labels are key-value pairs assigned to Kubernetes objects. Values must follow naming and length restrictions (max 63 chars, alphanumeric + separators).
- Primitives such as Deployments, Services, and NetworkPolicies use label selection for filtering, querying, and routing.
- Use annotations to provide human-readable metadata that cannot be queried. Some reserved annotations influence runtime behaviour (e.g. Pod Security Standards at namespace level).

© 2026 Tertiary Infotech Academy Pte Ltd

LABELS & ANNOTATIONS

Labels and Annotations

Label assignment and selection, providing meta information to objects

Labels
identifying metadata

Selectors
query by label

Annotations
non-identifying

Reserved keys
app.kubernetes.io/*

LABELS & ANNOTATIONS

What is a Label?

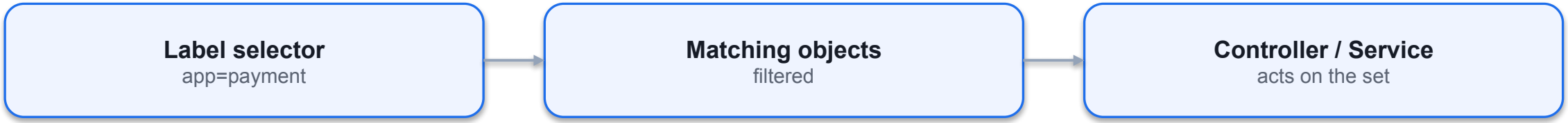
Essential to querying, filtering and sorting Kubernetes objects

- ● Key-value pairs assigned to Kubernetes objects with the imperative label
- command or using the metadata.labels[] attribute.
- ● Not meant for elaborate, multi-word descriptions of its functionality.
- ● Kubernetes limits the length of a label to a maximum of 63 characters and a range
- of allowed alphanumeric and separator characters.

LABELS & ANNOTATIONS

Selecting Labels

Label selector uses a set of criteria to query for objects



Assigning Labels with Imperative Approach

The label command lets you add or remove one or many labels

```
$ kubectl label pod nginx tier=backend env=prod app=miracle
 Declares three label
pod/nginx labeled   key-value pairs
$ kubectl get pods --show-labels
NAME ... LABELS
pod1 ... tier=backend,env=prod,app=miracle
 Removes the label
$ kubectl label pod nginx tier-
pod/nginx labeled   with key tier
$ kubectl get pods --show-labels
NAME ... LABELS
pod1 ... env=prod,app=miracle
```

LABELS & ANNOTATIONS

Recommended Labels

Naming convention starts with app.kubernetes.io

```
apiVersion: apps/v1
kind: Deployment
metadata:
 labels:
 app.kubernetes.io/name: wordpress
 app.kubernetes.io/instance: wordpress-abcxzy
 app.kubernetes.io/version: "4.9.4"
 app.kubernetes.io/managed-by: helm
 app.kubernetes.io/component: server
 app.kubernetes.io/part-of: wordpress
...
```

LABELS & ANNOTATIONS

Example Labels

Three different Pods with each three key-value pairs

Pod A
app=web
tier=frontend

Pod B
app=api
tier=backend

Pod C
app=db
tier=data

LABELS & ANNOTATIONS

Label Selection from the CLI

You can specify equality-based and set-based requirements

```
$ kubectl get pods -l tier=frontend,env=dev --show-labels
 Boolean
NAME ... LABELS   AND
pod2 ... app=crawler,env=dev,tier=frontend   expression
$ kubectl get pods -l version --show-labels
 Key-only
NAME ... LABELS   expression
pod3 ... app=crawler,env=staging,version=v2.1
$ kubectl get pods -l 'tier in (frontend,backend),env=dev' --show-labels
NAME ... LABELS
pod2 ... app=crawler,env=dev,tier=frontend
 Set-based expression
 follows Boolean OR
```

Label Selection in YAML

Certain objects provide label selection via their API

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: frontend-network-policy
spec:
 podSelector:
 Select Pods by labels that this
 matchLabels:
 tier: frontend   network policy should apply to
...
```

LABELS & ANNOTATIONS

What is an Annotation?

Descriptive metadata without the ability to be queryable

- ● Key-value pairs for providing descriptive, human-readable metadata assigned to
- Kubernetes objects with the imperative annotation command or using the
- metadata.annotations[] attribute.
- ● Cannot be used for querying or selecting objects.
- ● Make sure to put the value of an annotation into single- or double-quotes if it
- contains special characters or spaces.

SECTION

Example Annotations A Pod with three key-value pairs

Assigning Annotations with Imperative Approach

The annotation command lets you add or remove one or many annotations

```
$ kubectl annotate pod nginx commit='866a8dc' branch='bm/bugfix'
 Declares three
pod/nginx annotated   annotation
 key-value pairs
$ kubectl describe pods my-pod
Annotations: branch: bm/bugfix
 commit: 866a8dc
 Removes the
$ kubectl annotate pod nginx commit-   annotation with key
pod/nginx annotated
 commit
$ kubectl describe pods my-pod
Annotations:   branch:
bm/bugfix
```

LABELS & ANNOTATIONS

Well-Known Annotations

Reserved annotations that may influence runtime treatment

```
apiVersion: v1
kind: Namespace
metadata:
 name: secured
 annotations:
 kubernetes.io/description: "Manages objects for business app."
 pod-security.kubernetes.io/warn: "baseline"
 Enforces Pod
 Security Standards
 at the namespace
 level
```

DESIRED STATE

Why Deployments?

- A bare Pod is not rescheduled if it dies — you need a controller to keep the desired count.
- A Deployment manages a ReplicaSet, which manages the Pods — a three-tier ownership chain.
- kubectl scale changes the replica count; the ReplicaSet controller reconciles immediately.
- Old ReplicaSets are kept for rollback (revisionHistoryLimit, default 10).

© 2026 Tertiary Infotech Academy Pte Ltd

DEPLOYMENT → REPLICASET → POD

The Ownership Chain

Deployment

- You declare desired state
- replicas + Pod template
- Owns one active ReplicaSet

ReplicaSet

- Keeps N Pods running
- Hash-suffixed name
- Self-heals deleted Pods

Pod

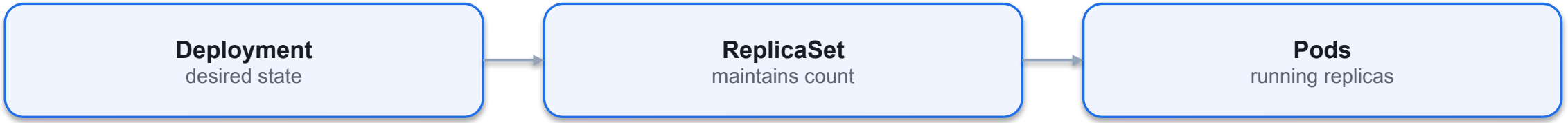
- The running workload
- Recreated if it dies
- Never edited directly

© 2026 Tertiary Infotech Academy Pte Ltd

DEPLOYMENTS

Deployments

Replicating and scaling workload, deployment strategies



DEPLOYMENTS

What is a Deployment?

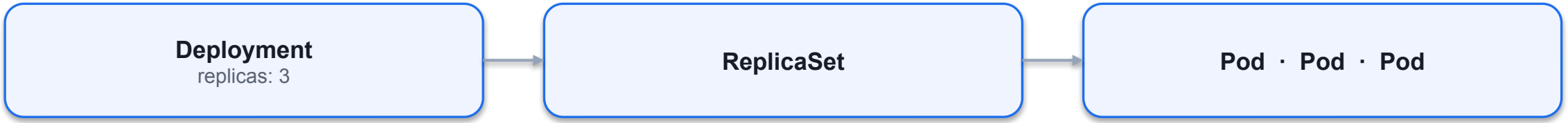
Scaling and replication features for a set of Pods

- ● Controls a predefined number of Pods with the same configuration, so-called
- replicas.
- ● The number of replicas can be scaled up or down to fulfill load requirements.
- ● Updates to the replica configuration can be updated easily and is rolled out
- automatically.

DEPLOYMENTS

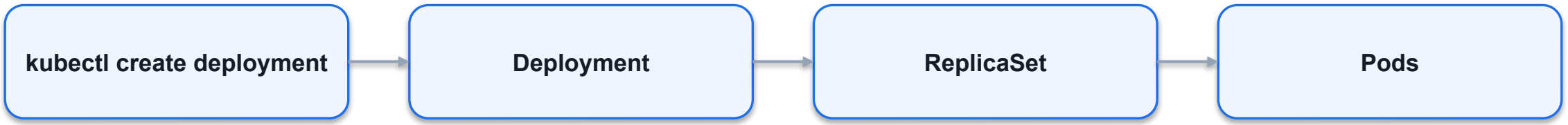
Example Deployment

A Deployment that controls three replicas



Creating a Deployment with Imperative Approach

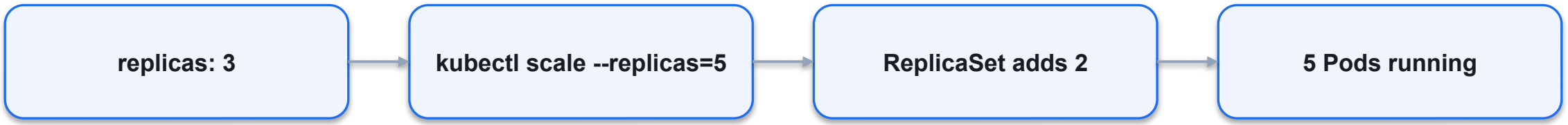
Creates objects for the Deployment, ReplicaSet and Pod(s)



DEPLOYMENTS

Manually Scaling a Deployment

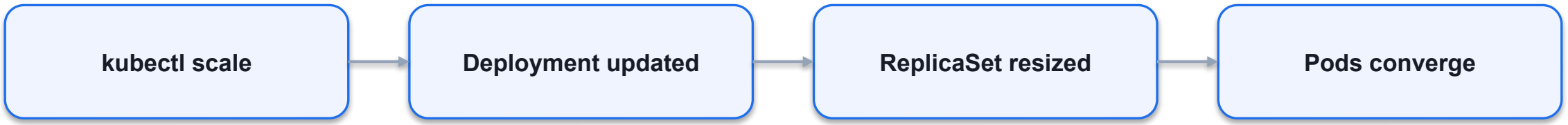
"We measured performance under load. Scale up to exactly x number of Pods."



DEPLOYMENTS

Scaling a Deployment with Imperative Approach

Scaling objects for the Deployment, ReplicaSet and Pod(s)



DEPLOYMENTS

Listing Deployment with Imperative Approach

Listing objects for the Deployment, ReplicaSet and Pod(s)

kubectl get deploy

kubectl get rs

kubectl get pods

kubectl get all

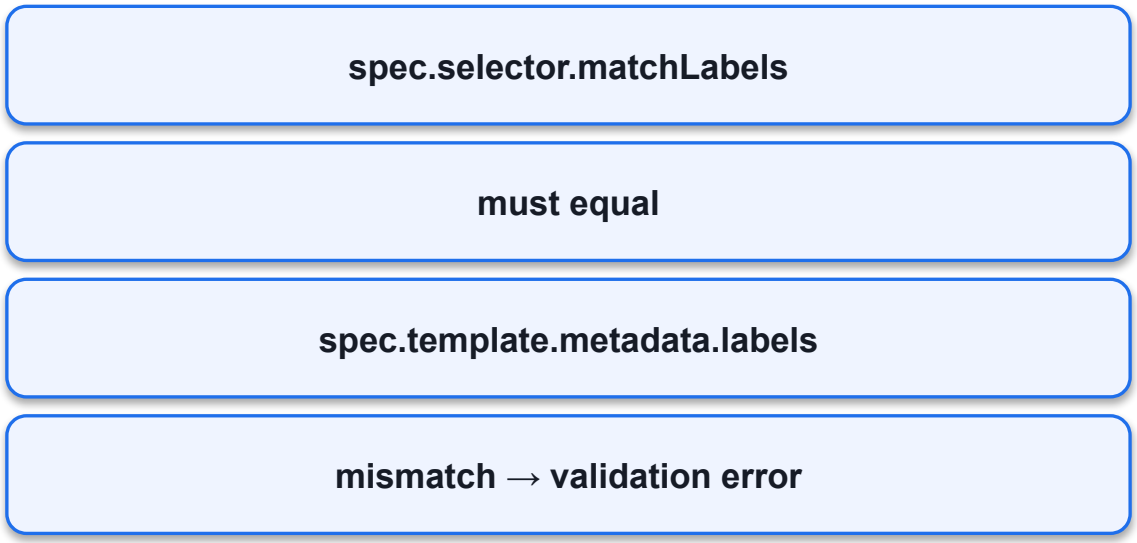
kubectl describe deploy

kubectl rollout status

DEPLOYMENTS

Deployment YAML Manifest

Creates an error if label selector and template labels do not match



Deployment Template Attributes

Replica configuration is defined under spec.template

```
  Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
  app: my-deploy
  name: my-deploy
spec:
  Pod
  replicas: 3
  selector:
  matchLabels:   apiVersion:v1
  tier: backend   Template spec attributes   kind: Pod
  template:   metadata:
```

Deployments and ReplicaSets

LAB 9 Deployments & ReplicaSets

A Deployment manages a ReplicaSet, which manages Pods. CKAD tests the full ownership chain, scaling, environment-variable injection and reading ReplicaSet history — required to debug failed rollouts.

You'll build

Create a Deployment imperatively, scale it, generate YAML, inject env vars, then read its ReplicaSet history.

Key commands: k create · k scale · k set · » `kubectl

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 9 · COMMANDS

Deployments and ReplicaSets

Step 1 — Create a Deployment imperatively

```
k create deployment web --image=nginx:1.25 --replicas=3
k get deploy,rs,pod -l app=web
```

Step 2 — Scale up and down

```
k scale deployment web --replicas=5
k scale deployment web --replicas=2
k get pods -l app=web
```

Step 3 — Generate Deployment YAML and apply

```
k create deployment api --image=httpd:2.4 --replicas=2 $do > api.yaml
k apply -f api.yaml
```


© 2026 Tertiary Infotech Academy Pte Ltd

Deployments and ReplicaSets (cont.)

Step 4 — Inject an env var and observe ReplicaSet history

```
k set env deployment/api APP_COLOR=blue
k get rs -l app=api    # old RS 0/0/0, new RS 2/2/2
```

Note kubectl set env / set image mutate the Pod template and trigger a new ReplicaSet (a rolling update). Old ReplicaSets are kept for rollback — revisionHistoryLimit (default 10) controls how many.

© 2026 Tertiary Infotech Academy Pte Ltd

Deployments and ReplicaSets

✅ Test it

k get deploy,rs,pod -l app=web shows one Deployment owning one ReplicaSet owning the Pods, and k get rs -l app=api shows two ReplicaSets after the env change.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

ZERO DOWNTIME

How a Rolling Update Works

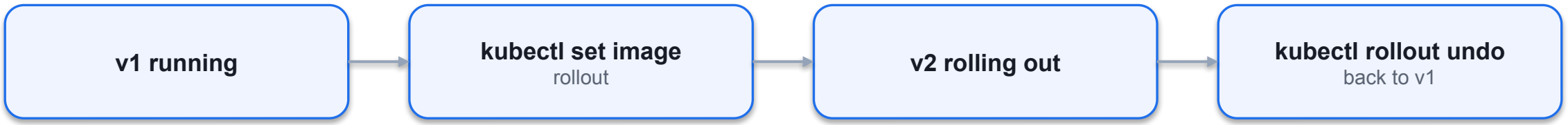
- kubectl set image mutates the Pod template — this creates a new ReplicaSet and starts a rollout.
- maxSurge = extra Pods allowed above desired; maxUnavailable = Pods allowed down — together they tune speed vs availability.
- kubectl rollout status watches progress; rollout history lists every revision.
- rollout pause batches several changes into one ReplicaSet; rollout undo reverts instantly.

© 2026 Tertiary Infotech Academy Pte Ltd

DEPLOYMENTS

Roll Out and Roll Back Feature

"Look ma, shiny new features. Let’s deploy them to production!"



DEPLOYMENTS

Rolling Out a New Revision

The rollout history keeps track of changes

```
$ kubectl rollout history deployment my-deploy
deployment.apps/my-deploy
REVISION CHANGE-CAUSE
1   <none>
  Changes the
$ kubectl set image deployment my-deploy nginx=nginx:1.19.2   assigned image
deployment.apps/my-deploy image updated
  for Pod template
$ kubectl rollout history deployment my-deploy
deployment.apps/my-deploy
REVISION CHANGE-CAUSE
1   <none>
2   <none>
```

DEPLOYMENTS

Rolling Back to Previous Revision

You can revert to any revision available in the history

```
$ kubectl rollout undo deployment my-deploy --to-revision=1
deployment.apps/my-deploy rolled back
$ kubectl rollout history deployment my-deploy
deployment.apps/my-deploy
REVISION   CHANGE-CAUSE
2   <none>
3   <none>
  Kubernetes deduplicates a revision
  if it points to the same changes
```

Setting a Change Cause for a Revision

Tracked by annotation with key kubernetes.io/change-cause

```
$ kubectl rollout history deployment my-deploy
deployment.apps/my-deploy
REVISION CHANGE-CAUSE
2   <none>
3   <none>
$ kubectl annotate deployment
  What changed
my-deploy
kubernetes.io/change-cause="image updated to 1.16.1"
deployment.apps/my-deploy annotated   with this revision?
$ kubectl rollout history deployment my-deploy
deployment.apps/my-deploy
REVISION CHANGE-CAUSE
2   <none>
```

DEPLOYMENTS

Common Deployment Strategies

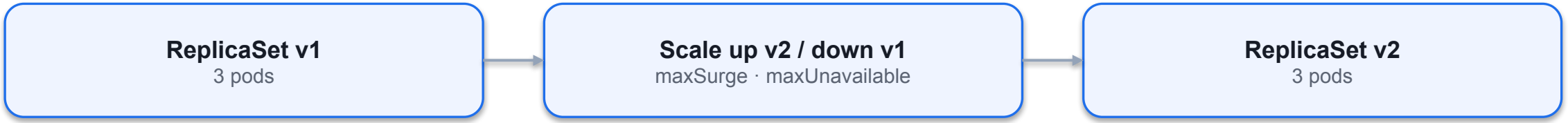
Choosing the right deployment procedure depends on the needs

- ● Rolling Update: Release a new version on a rolling update fashion, one after the
- other.
- ● Recreate: Terminate the old version and release the new one.
- ● Blue/green: Release a new version alongside the old version then switch traffic.
- ● Canary: Release a new version to a subset of users, then proceed to a full rollout.

DEPLOYMENTS

Rolling Update Deployment Strategy

Secondary ReplicaSet is created with new version of application



SECTION

Rolling Update Declaratively

SECTION

Rollout Status

SECTION

What happens if there is an error?

Rolling Updates and Rollback

LAB 10 Rolling Updates & Rollback

A Deployment performs zero-downtime upgrades by gradually replacing Pods one ReplicaSet at a time. CKAD tests maxSurge, maxUnavailable, rollout pause/resume, history inspection and rollback — often as a timed multi-step question.

You'll build

Tune the rolling-update strategy, trigger an image update, inspect history, pause/resume, then roll back.

Key commands: k create · k patch · k set · k rollout · » Pausing

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 10 · COMMANDS

Rolling Updates and Rollback

Step 1 — Create the initial Deployment

```
k create deployment web --image=nginx:1.24 --replicas=4
k rollout status deployment/web
```

Step 2 — Tune the rolling-update strategy

```
k patch deployment web -p \
  '{"spec":{"strategy":{"rollingUpdate":{"maxSurge":1,"maxUnavailable":1}}}}'
```

Step 3 — Trigger an image update and watch the rollout

```
k set image deployment/web nginx=nginx:1.25
k rollout status deployment/web
k get rs -l app=web        # old RS drains to 0, new RS ramps to 4
```


© 2026 Tertiary Infotech Academy Pte Ltd

Beyond the Default Rollout

- Blue/Green runs two complete versions; a Service selector switch flips 100% of traffic atomically.
- Canary sends a small slice of traffic to the new version by running it at a low replica ratio.
- Both are built from plain Deployments and Services using labels — no extra tooling for CKAD.
- Rollback is instant: re-patch the selector (blue/green) or scale the canary back down.

© 2026 Tertiary Infotech Academy Pte Ltd

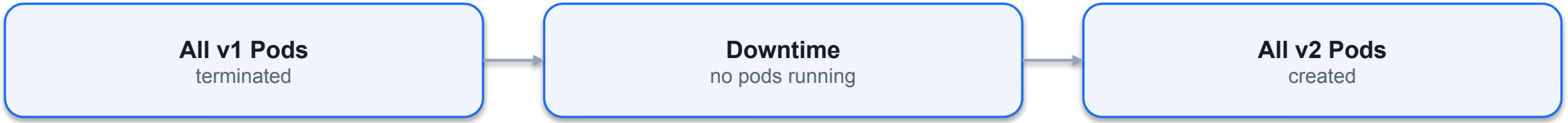
SECTION

Deployment Strategy

DEPLOYMENT STRATEGIES

Recreate Deployment Strategy

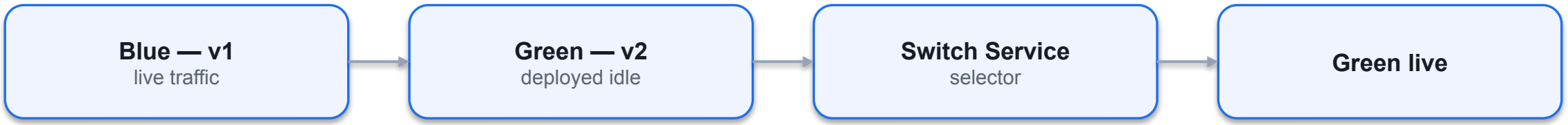
Terminates all the running instances then recreate them



DEPLOYMENT STRATEGIES

Blue/Green Deployment Strategy

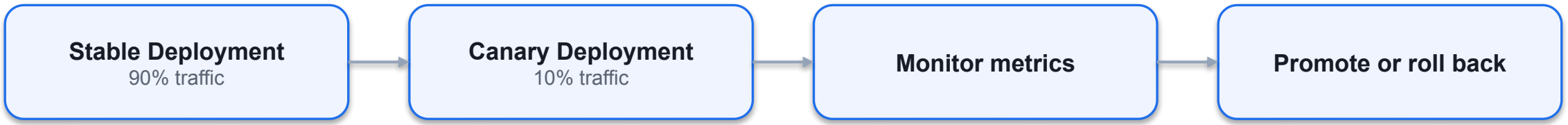
Deployment object with new version is created alongside



DEPLOYMENT STRATEGIES

Canary Deployment Strategy

Two Deployment objects with different traffic distribution



DAY 2 · BLUE/GREEN

Blue/Green Deployment

LAB 11

Blue/Green Deployment

Blue/Green keeps two complete copies of the app alive at once. A Service selector switch flips 100% of traffic from blue to green in one atomic operation — with instant rollback if anything breaks.

You'll build

Deploy blue (live) and green (idle), smoke-test green, flip the Service selector, then roll back with one patch.

Key commands: k apply · GREEN_POD=$(k get · k patch · » Cutover

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 11 · COMMANDS

Blue/Green Deployment

Step 1 — Deploy blue (current production) behind a Service

```
# Deployment web-blue (labels app=web, version=blue) + Service web
# selector: {app: web, version: blue}
k apply -f blue.yaml
k get pods -l app=web --show-labels
```

Step 2 — Deploy green (next version, no live traffic yet)

```
# Deployment web-green (labels app=web, version=green), nginx:1.25
k apply -f green.yaml
k get pods -l app=web --show-labels
```

Step 3 — Smoke-test green directly before cutover

```
GREEN_POD=$(k get pod -l version=green -o jsonpath='{.items[0].metadata.name}')
k exec $GREEN_POD -- curl -sI localhost:80 | head -2
```


© 2026 Tertiary Infotech Academy Pte Ltd

Blue/Green Deployment (cont.)

Step 4 — Flip the Service to green (atomic cutover)

```
k patch service web -p '{"spec":{"selector":{"app":"web","version":"green"}}}'
k describe svc web | grep Selector
```

Step 5 — Roll back instantly if needed

```
k patch service web -p '{"spec":{"selector":{"app":"web","version":"blue"}}}'
```

Note Cutover and rollback are both a single kubectl patch service selector update — atomic, milliseconds, no Pod churn. Blue stays running as an instant fallback.

© 2026 Tertiary Infotech Academy Pte Ltd

Blue/Green Deployment

✅ Test it

After the patch, k describe svc web | grep Selector shows version=green, and patching it back to version=blue restores the old version instantly.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

DAY 2 · CANARY

Canary Deployment

LAB 12

Canary Deployment

A canary release sends a small fraction of live traffic to a new version while the bulk stays on stable. In Kubernetes the split is approximated by replica ratios behind one broad Service selector — no special tooling required.

You'll build

Run a 9-replica stable and 1-replica canary behind one Service, verify the ~10% split, then promote.

Key commands: k apply · k run · k scale · » Traffic

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 12 · COMMANDS

Canary Deployment

Step 1 — Stable Deployment (90% of traffic)

```
# Deployment web-stable: replicas 9, labels app=web,track=stable
# Service web: selector {app: web}, port 5678
k apply -f stable.yaml
```

Step 2 — Canary Deployment (10% of traffic)

```
# Deployment web-canary: replicas 1, labels app=web,track=canary
k apply -f canary.yaml
k get pods -l app=web --show-labels
```

Step 3 — Verify the traffic split

```
k run probe --image=busybox --restart=Never -it --rm -- sh -c \
  'for i in $(seq 1 30); do wget -qO- web:5678; done | sort | uniq -c'
# ~27 stable, ~3 canary
```


© 2026 Tertiary Infotech Academy Pte Ltd

Canary Deployment (cont.)

Step 4 — Promote: scale up canary, retire stable

```
k scale deployment web-canary --replicas=9
k scale deployment web-stable --replicas=0
```

Note Traffic split is approximated by replica ratio (9:1 ≈ 90/10). Promotion is just kubectl scale on both Deployments — no Service change. For precise percentage splits use a service mesh (Istio/Linkerd) — beyond CKAD scope.

© 2026 Tertiary Infotech Academy Pte Ltd

Types of Autoscalers

HPA — Horizontal

- Horizontal Pod Autoscaler (HPA)
- Scales the number of Pod replicas based on CPU/memory thresholds
- Standard Kubernetes feature
- Uses the metrics-server component

VPA — Vertical

- Vertical Pod Autoscaler (VPA)
- Scales CPU/memory allocation for existing Pods from historic metrics
- Provided by cloud providers as an add-on or installed manually
- Both autoscalers require metrics-server and resource requests/limits on Pods

© 2026 Tertiary Infotech Academy Pte Ltd

Autoscaling a Deployment by CPU

- The HPA monitors average CPU utilisation across all running replicas.
- Example: threshold = 80% CPU, current = 15% → no scale-out needed.
- Example: threshold = 80% CPU, current = 95% → HPA adds replicas.
- Scaling decisions respect minReplicas and maxReplicas bounds.
- kubectl autoscale is the imperative shortcut to create an HPA.

© 2026 Tertiary Infotech Academy Pte Ltd

Inspecting the Horizontal Pod Autoscaler

List HPAs — shows current utilisation vs threshold

```
kubectl get hpa
NAME        REFERENCE               TARGETS    MINPODS  MAXPODS  REPLICAS  AGE
app-cache   Deployment/app-cache    15%/80%    3        5        3         2m14s
```

Describe for detailed events and scaling history

```
kubectl describe hpa app-cache
```


© 2026 Tertiary Infotech Academy Pte Ltd

EXERCISE

Creating a Horizontal Pod Autoscaler for a Deployment

© 2026 Tertiary Infotech Academy Pte Ltd

Types of Autoscalers

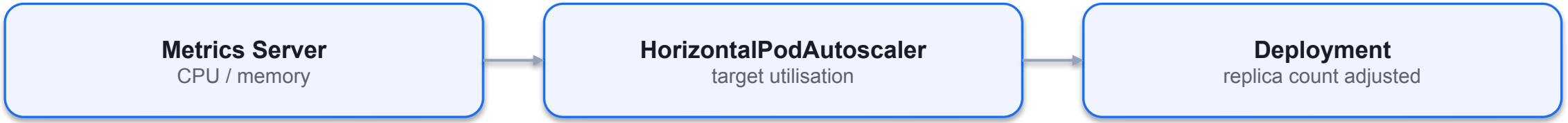
Scaling decisions are made based on metrics

- ● Horizontal Pod Autoscaler (HPA): A standard feature of Kubernetes that scales
- the number of Pod replicas based on CPU and memory thresholds.
- ● Vertical Pod Autoscaler (VPA): Scales the CPU and memory allocation for existing
- Pods based on historic metrics. Supported by a cloud provider as an add-on or
- needs to be installed manually.
- ● Both autoscalers use the metrics server. You need to install the component. A Pod
- should also set the resource requests and limits.

DEPLOYMENT STRATEGIES

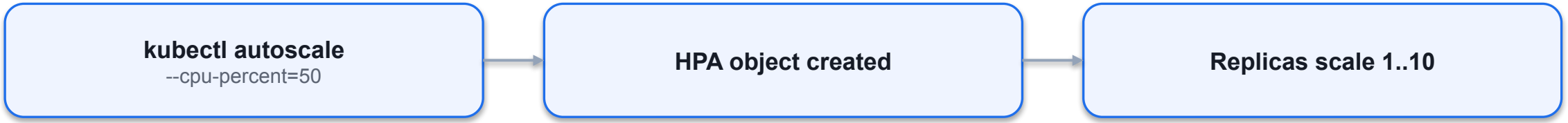
Horizontal Pod Autoscaler (HPA)

Kubernetes primitive that reaches into metrics server



Autoscaling a Deployment by CPU

"Auto-scale based on average CPU utilization across all replicas."



Horizontal Pod Autoscaler YAML Manifest

Creates an error if label selector and template labels do not match

```
apiVersion: autoscaling/v2
kind:
HorizontalPodAutoscaler
metadata:
 name: app-cache
spec:
 scaleTargetRef:
 apiVersion:   The scaling target,
 apps/v1 kind:
 Deployment name:
 a Deployment
 minReplicas:
 app-cache 3
 maxReplicas: 5
```

Inspecting the Horizontal Pod Autoscaler

Use scale command or change spec.replicas attribute in live object

```
$ kubectl get hpa
NAME   REFERENCE   TARGET   MINPOD   MAXPOD   REPLICAS   AGE
app-cach   Deployment/app-cach   S   S 3   S 5   3
e 2m14s   e   15%/80
  %
  Renders current resource utilization
  and compares them with thresholds
```

PACKAGE & RELEASE

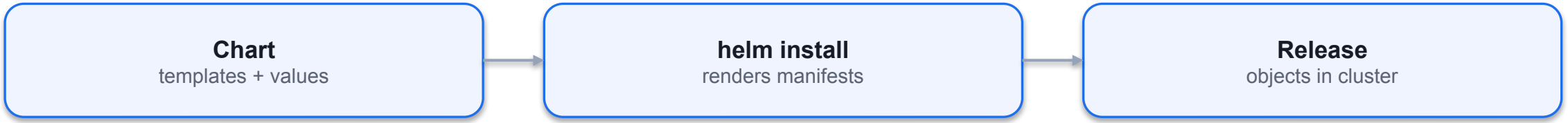
Charts, Releases & Revisions

- A chart is a bundle of templated YAML; a release is one deployed instance of a chart.
- Workflow: repo add → install → upgrade → rollback → uninstall.
- --set overrides values at install/upgrade; --reuse-values keeps prior overrides.
- helm history tracks every revision; helm get manifest shows what was actually applied.

© 2026 Tertiary Infotech Academy Pte Ltd

Helm

Deploying an application defined by a set of YAML manifests



What is Helm?

Package manager and templating engine for a set of manifests

- ● The artifact produced by the Helm executable is a so-called chart file bundling the
- manifests that comprise the API resources of an application.
- ● Chart files can be distributed to a chart repository e.g. central chart repository and
- consumed from there (e.g., for running Grafana or PostgreSQL).
- ● At runtime, it replaces placeholders in YAML template files with actual, end-user
- defined values.

Finding a Chart

Searches the Artifact Hub for a chart with a matching name

```
$ helm search hub jenkins
URL   CHART   VERSION APP
https://artifacthub.io/packages/helm/bitnami/jenkins   11.0.6   2.361.2
...
```

Installing a Chart

You may have to add an external repository that hosts chart file

```
$ helm repo add bitnami https://charts.bitnami.com/bitnami
"bitnami" has been added to your repositories
$ helm install jenkins bitnami/jenkins
...
CHART NAME: jenkins
CHART VERSION: 11.0.6
APP VERSION: 2.361.2
** Please be patient while the chart is being deployed **
```

Listing Installed Charts

Information about chart details

```
$ helm list
NAME NAMESPACE   REVISION ...   STATUS   CHART   APP VERSION
jenkins default   1   deployed   jenkins-11.0.6   2.361.2
$ kubectl get pods
NAME   READY   STATUS   RESTARTS   AGE
pod/bitnami-jenkins-764b44cc99-twf6f   1/1   Running   0   7m34s
...
```

Standard Chart Structure

Predefined directory structure with mandatory Chart.yaml and values.yaml

```
$ tree
.
├── Chart.yaml
├── templates
│   ├── web-app-pod-template.yaml
│   └── web-app-service-template.yaml
└── values.yaml
```

Chart File

Describes the meta information of the chart (e.g., name and version)

```
Chart.yaml
  apiVersion: 1.0.0
  name: web-app
  version: 2.5.4
```

Values File

Key-value pairs used at runtime to replace placeholders in the YAML manifests

```
values.yaml
  db_host: mysql-service
  db_user: root
  db_password: password
  service_port: 3000
Additional built-in objects are available for use.
```

Template File

YAML manifest that can define placeholders using double curly braces

web-app-pod-template.yaml

```
apiVersion: v1
kind: Pod
metadata:
 name: web-app
spec:
 containers:
 - image: bmuschko/web-app:1.0.1
 name: web-app
 env:
 - name: DB_HOST
 value: {{ .Values.db_host }}
 - name: DB_USER
 value: {{ .Values.db_user }}
```

Previewing Final Templates

Replaces placeholders with actual values from values.yaml

```
$ helm template .
---
# Source: Web Application/templates/web-app-pod-template.yaml
apiVersion: v1
kind: Pod
metadata:
 name: web-app
spec:
 containers:
 - image: bmuschko/web-app:1.0.1
 name: web-app
 env:
 - name: DB_HOST
 value: mysql-service
```

Installing a Chart from Local Files

Helpful for trying out the deployment before publishing the chart file

```
$ helm install web-app .
NAME: web-app
LAST DEPLOYED: Wed Apr 19 11:58:34
NAMESPACE:
default STATUS:
deployed
REVISION: 1
TEST
$   SUITE:
 kubectl   None
 get pods,services
NAME   READY   RESTARTS   AGE
 STATUS
pod/web-app   1/1 Running   0   3m24s
```

Overriding Values

End user can tweak runtime behavior to personal requirements

```
$ helm install --namespace custom --create-namespace --set service_port=9090 web-app .
NAME: web-app
LAST DEPLOYED: Wed Apr 19 11:58:34
NAMESPACE:
custom STATUS:
deployed
REVISION: 1
TEST SUITE: None
  Providing a custom   Override values in
  namespace (defaults to   values.yaml
  default namespace)
```

Bundling the Template Files into a Chart Archive File

Archive file is usually published to a chart repository for consumption

```
$ helm package .
Successfully packaged chart and saved it to:
/Users/bmuschko/helm/web-app-2.5.4.tgz
$ ls
web-app-2.5.4.tgz
```

DAY 2 · HELM

Helm — Install, Upgrade, Rollback

LAB 13 Helm

Helm is the Kubernetes package manager. A chart is a bundle of templated YAML; a release is a deployed instance. CKAD tests install, upgrade, rollback, list and value overrides — all under exam time pressure.

You'll build

Add a chart repo, install a release with overrides, inspect it, upgrade, roll back, then uninstall.

Key commands: helm repo · helm install · helm get · helm upgrade · » `--reuse-values`

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 13 · COMMANDS

Helm — Install, Upgrade, Rollback

Step 1 — Add a chart repository

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo bitnami/nginx | head -5
```

Step 2 — Install a release with value overrides

```
helm install web bitnami/nginx \
  --set service.type=ClusterIP --set replicaCount=2
helm list
```

Step 3 — Inspect the generated manifests and values

```
helm get manifest web | head -40
helm get values web
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 13 · COMMANDS

Helm — Install, Upgrade, Rollback (cont.)

Step 4 — Upgrade, then roll back

```
helm upgrade web bitnami/nginx --set replicaCount=4 --reuse-values
helm history web
helm rollback web 1
helm uninstall web
```

Note --reuse-values preserves prior overrides while changing only the new one. helm history <release> tracks every revision; rollback creates a new revision rather than deleting history.

© 2026 Tertiary Infotech Academy Pte Ltd

REUSE WITHOUT TEMPLATES

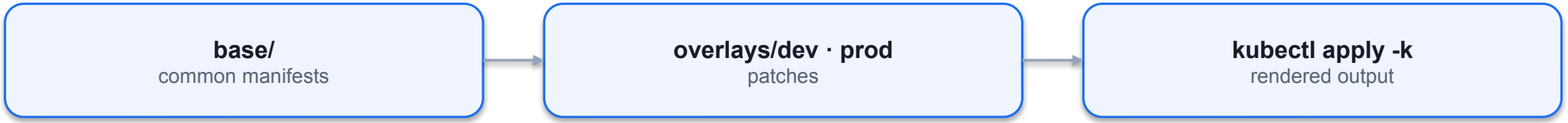
Base + Overlays

- Kustomize reuses one base set of manifests and layers environment-specific changes on top.
- namePrefix and commonLabels are applied to every resource in the overlay.
- The images block overrides image tags without editing the base files; patches do strategic merges.
- Built into kubectl: kubectl apply -k and kubectl kustomize — no extra binary required.

© 2026 Tertiary Infotech Academy Pte Ltd

Kustomize

Template-free customization of application configuration



What is Kustomize?

Customizing application manifests without templates

- Writing manifests for application stacks is straightforward, however, deploying them to different Kubernetes runtime environments with varying configuration gets tricky.
- Instead of copying and modifying manifests per environment, you can use kustomize to merge configuration change. Kustomize does not require using a template engine or changing the original manifests.
- Kustomize can be used as a subcommand/flag of kubectl or as a separate executable.

Kustomize Use Cases

Convenient manifest management

- ● Generating manifests from other sources. For example, creating a ConfigMap and
- populating its key-value pairs from a properties file.
- ● Adding common configuration across multiple manifests. For example, adding a
- namespace and a set of labels for a Deployment and a Service.
- ● Composing and customizing a collection of manifests. For example, setting resource
- boundaries for multiple Deployments.

Basic Kustomize Setup

Regular YAML manifests + a kustomization.yaml file

```
.
└── base
    ├── deployment.yaml   apiVersion: apps/v1
    ├── kustomization.yaml   kind: Deployment
    └── service.yaml   ...
  apiVersion: v1   apiVersion: kustomize.config.k8s.io/v1beta1
  kind: Service   kind: Kustomization
  ...   ...
```

Kustomization File Structure

Generate or transform other Kubernetes Resource Model (KRM) objects

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- {pathOrUrl}
 What existing resources
- ...   are to be customized
generators:
- {pathOrUrl}   What new resources
- ...   should be created
transformers:
- {pathOrUrl}   What to do to the
- ...
 aforementioned resources
validators:
```

Example Kustomization File

Merges labels into a set of resources

```
kustomization.yaml
  apiVersion: kustomize.config.k8s.io/v1beta1
  kind: Kustomization
  labels:   Defines labels for all
   layer: backend   processed resources
  resources:
   - deployment.yaml   Declares resources
   - service.yaml   to be modified
```

Generating a Kustomize File

Imperative command to create kustomization.yaml file

```
$ cd base
$ ls .
deployment.yaml service.yaml
$ kustomize create --resources=deployment.yaml,service.yaml
  --labels=layer:backend
$ ls .
deployment.yaml kustomization.yaml service.yaml
```

Building and Previewing a Set of Resources

Applies the rules in kustomization.yaml and renders the result to the output

```
$ kustomize build base   Alternatively, you can
apiVersion: v1   also use kubectl
kind: Service   kustomize base
metadata:
  labels:
  layer: backend
...
---
  Applied labels
apiVersion: apps/v1
kind: Deployment   from rules
metadata:
  labels:
  layer: backend
```

Managing the Resources

Use the typical commands provided by kubectl

```
$ kubectl apply -k base
service/backend-service configured
deployment.apps/backend-deployment created
$ kubectl get deployments,services
NAME   READY   UP-TO-DATE   AVAILABLE AGE
deployment.apps/backend-deployment   1/1   1   1   7s
NAME   TYPE   CLUSTER-IP   EXTERNAL-IP  ...
service/backend-service   ClusterIP   10.107.228.58 <none>   ...
$ kubectl delete -k base
service "backend-service" deleted
deployment.apps "backend-deployment" deleted
```

Environment Kustomize Setup

Subdirectories contain environment-specific configuration

- .
- ├── base
- │ ├── deployment.yaml
- │ ├── kustomization.yaml
- │ └── service.yaml
- ├── dev
- │ ├── config.properties
- │ └── kustomization.yaml Create a dedicated
- └── prod directory + configuration
- ├── config.properties per environment
- └── kustomization.yaml

Example Production Environment

Base configuration is merged with the configuration of an environment

```
prod/kustomization.yaml
   apiVersion: kustomize.config.k8s.io/v1beta1
   kind: Kustomization
   resources:
   - ../base/ namespace:
   prod namePrefix:
   prod-
   configMapGenerator:
   - name: backend-configmap
   env: config.properties
   replicas:
   - name: backend-deployment
   count: 10
```

Merging Environment Configuration

Base configuration + environment configuration

```
$ kustomize build prod
apiVersion: v1
data:
  DB_USERNAME: prod-user
kind: ConfigMap
metadata:
  name: prod-backend-configmap-7gbb88gbhc
  namespace: prod
---
apiVersion: v1
kind: Service
metadata:
  labels:
  layer: backend
```

Kustomize Benefits

A good option for operators managing their own manifests

- ● Native to kubectl: Kustomize does not necessarily require you to install additional
- tooling.
- ● DRY approach: Define a base configuration just once with the ability to reuse it.
- Environment-specific configuration will be added as an overlay.
- ● Easy to learn: Does not require learning another template engine. You’ll use
- standard YAML manifests to define resources.
- ● Scalable variants: New configuration e.g. environments can be easily added.

Kustomize Overlays

LAB 14 Kustomize

Kustomize reuses a single base set of manifests and applies environment-specific overlays without templating. It is built into kubectl (-k). CKAD tests namePrefix, commonLabels, images and strategic-merge patches.

You'll build

Build a base, add dev and prod overlays (prefix, labels, image tag, replica patch), preview, then apply.

Key commands: resources: · kubectl kustomize · » `namePrefix`

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 14 · COMMANDS

Kustomize Overlays

Step 1 — Base kustomization (shared by all environments)

```
# base/kustomization.yaml
resources:
- deployment.yaml
- service.yaml
```

Step 2 — Dev overlay (name prefix + common label)

```
# overlays/dev/kustomization.yaml
resources:
- ../../base
namePrefix: dev-
commonLabels:
  env: dev
```


© 2026 Tertiary Infotech Academy Pte Ltd

Kustomize Overlays (cont.)

Step 3 — Prod overlay (image bump + replica patch)

```
# overlays/prod/kustomization.yaml
resources:
- ../../base
namePrefix: prod-
commonLabels: {env: prod}
images:
- name: nginx
  newTag: "1.26"
patches:
- path: replica-patch.yaml
```

Step 4 — Preview, then apply

```
kubectl kustomize overlays/prod | grep -E "name:|replicas:|image:"
kubectl apply -k overlays/dev
kubectl apply -k overlays/prod
```


© 2026 Tertiary Infotech Academy Pte Ltd

Kustomize Overlays (cont.)

Note namePrefix and commonLabels apply to every resource in the overlay. The images block overrides tags without editing base files. apply -k / kubectl kustomize are built into kubectl — no extra binary.

© 2026 Tertiary Infotech Academy Pte Ltd

Specialised Controllers

- A DaemonSet runs exactly one Pod per matching node — log collectors, CNI agents, node exporters.
- A StatefulSet gives Pods stable names (db-0, db-1) and ordered, graceful start/stop.
- A StatefulSet needs a headless Service (clusterIP: None) for per-Pod DNS.
- Scale-down always removes the highest ordinal first; scale-up adds the next in order.

© 2026 Tertiary Infotech Academy Pte Ltd

THREE WORKLOAD SHAPES

Pick the Right Controller

Deployment

- Stateless, interchangeable
- Any number of replicas
- Rolling updates

DaemonSet

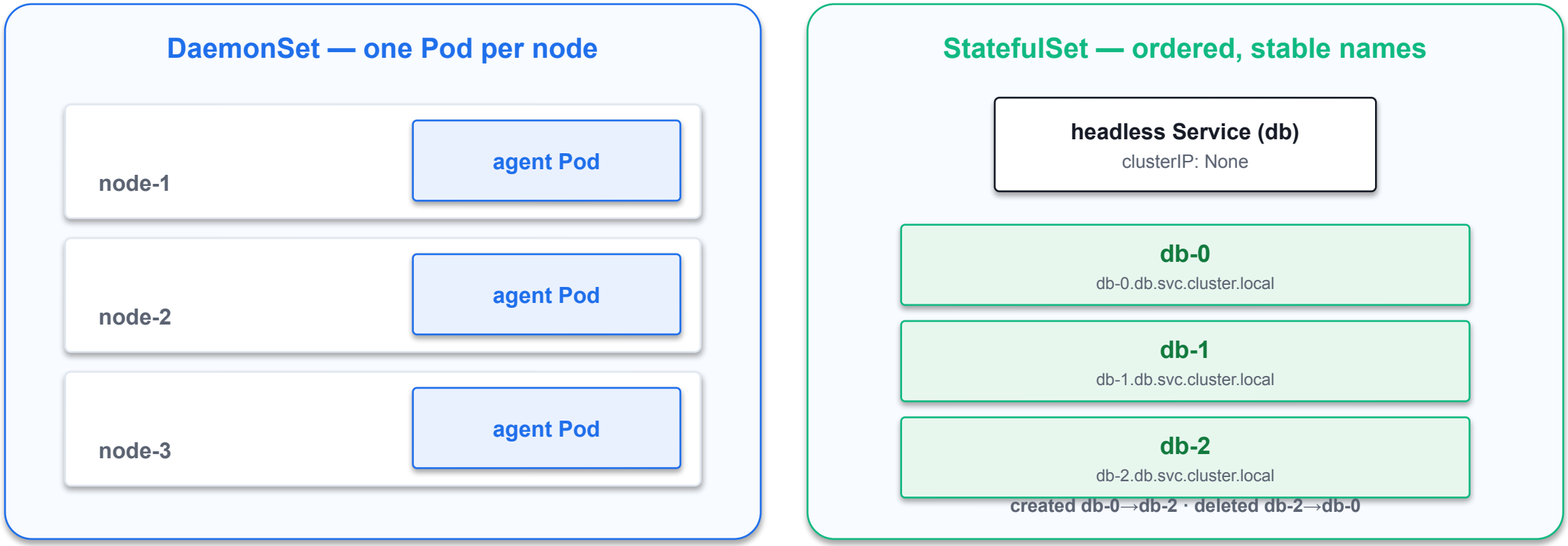
- One Pod per node
- Node-level agents
- Scales with the cluster

StatefulSet

- Stable identity + storage
- Ordered start/stop
- Databases, queues

© 2026 Tertiary Infotech Academy Pte Ltd

DaemonSet vs StatefulSet



DaemonSet = **one Pod per matching node**; StatefulSet = **stable identity + ordered start/stop**.

© 2026 Tertiary Infotech Academy Pte Ltd

SECTION

Exam Objectives and Overview

SCHEDULING

What is a DaemonSet?

- Resource that ensures exactly one (or more) copy of a Pod runs on every (or every
- matching) Node in the cluster
- Contrast with a Deployment, which runs N replicas spread across nodes as the
- scheduler sees fit

SCHEDULING

Use cases

- Log collectors (Fluentd, filebeat), metrics exporters (prometheus node exporter),

- security scanners
- Host-path volume managers, CNI plugins, CSI drivers, edge workloads

SCHEDULING

Yaml Manifest

selector: must match the Pod

```
template’s labels
template: same as for
Deployments—a Pod spec.
hostPath (common): mount
host filesystems into each
Pod.
```

SCHEDULING

Deploy & Verify

- kubectl apply -f daemonset.yaml

```
- kubectl get daemonset -n logging
- kubectl get pods -n logging -o wide
- kubectl rollout status daemonset/fluentd -n logging
If you add a new node, the DaemonSet controller automatically launches a new Pod
on it; when you remove a node, its Pod is cleaned up.
```

SCHEDULING

What is a StatefulSet?

- Set of pods with stable unique identities and optional stable storage

- ● Pods are created in ordinal order, from 0 up to N-1
- ● Controller waits for Pod 0 to be running and ready before creating pod 1 etc.
- ● Like Deployments, each Pod is scheduled by the kube-scheduler based on capacity,
- taints, selectors, affinity, etc.

DaemonSets and StatefulSets

LAB 15 DaemonSets & StatefulSets

A DaemonSet runs exactly one Pod per matching node (log collectors, CNI agents). A StatefulSet gives Pods stable identities and ordered start/stop (databases). CKAD expects you to know when to use each and write the YAML.

You'll build

Run a DaemonSet on every node, then a StatefulSet with a headless Service, stable DNS and ordered scaling.

Key commands: k apply · nslookup db-0.db.default.svc.cluster.local · » DaemonSet

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 15 · COMMANDS

DaemonSets and StatefulSets

Step 1 — DaemonSet on every node

```
# DaemonSet node-agent, image busybox
k apply -f ds.yaml
k get ds,pods -l app=node-agent -o wide   # DESIRED = node count
```

Step 2 — Headless Service for the StatefulSet

```
# Service db: clusterIP: None, selector app=db, port 5432
k apply -f headless.yaml
```

Step 3 — StatefulSet with stable identities

```
# StatefulSet db: serviceName db, replicas 3
k apply -f sts.yaml
k rollout status sts/db
k get pods -l app=db        # db-0 → db-1 → db-2 in order
```


© 2026 Tertiary Infotech Academy Pte Ltd

DaemonSets and StatefulSets

✅ Test it

k get pods -l app=db shows db-0, db-1, db-2 created in order, and nslookup db-0.db.default.svc.cluster.local resolves to a single Pod IP.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

SCHEDULING

What happens if we want to stop

a pod from getting deployed on

- certain nodes?

SCHEDULING

Taints & Toleration

- Taints marks a node as unschedulable

- Toleration marks a tainted node as schedulable

SCHEDULING

Taint Effects

- NoSchedule → Pods without a matching toleration will never be scheduled there.

```
●   PreferNoSchedule → Kubernetes will try to avoid placing Pods there, but it’s not a hard
requirement.
●   NoExecute → Existing Pods without the toleration are evicted, and new ones won’t be scheduled.
 kubectl taint nodes node-name key=value:taint-effect
kubectl taint nodes node1 app=purple:NoSchedule
```

SCHEDULING

What happens if we want a pod

to only go to a specific node?

nodeSelector
simple label match

Node affinity
expressive rules

Taints
node repels pods

Tolerations
pod accepts taint

SCHEDULING

Node Affinity

apiVersion: v1 ● Large or Medium ?

```
kind: Pod   ●   Not Small?
metadata:
  name: myapp-pod
spec:
  containers:
  - image: nginx
   name: web-server
  nodeSelector:
   size: Large
```

SCHEDULING

Node Affinity Types

- requiredDuringSchedulingIgnoredDuringExecution

- preferredDuringSchedulingIgnoredDuringExecution
- DuringScheduling DuringExecution
- Type1 Required Ignored
- Type2 Preferred Ignored

03

DAY 3

Observability + Config & Security

Probes · Logging · Metrics · Debug · ConfigMaps · Secrets · SecurityContext · RBAC (CKAD Domains 3 & 4 — 40%)

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 3

Observability & Maintenance

Probes · logging · metrics · debugging · API versions (15%)

© 2026 Tertiary Infotech Academy Pte Ltd

CONTAINER HEALTH

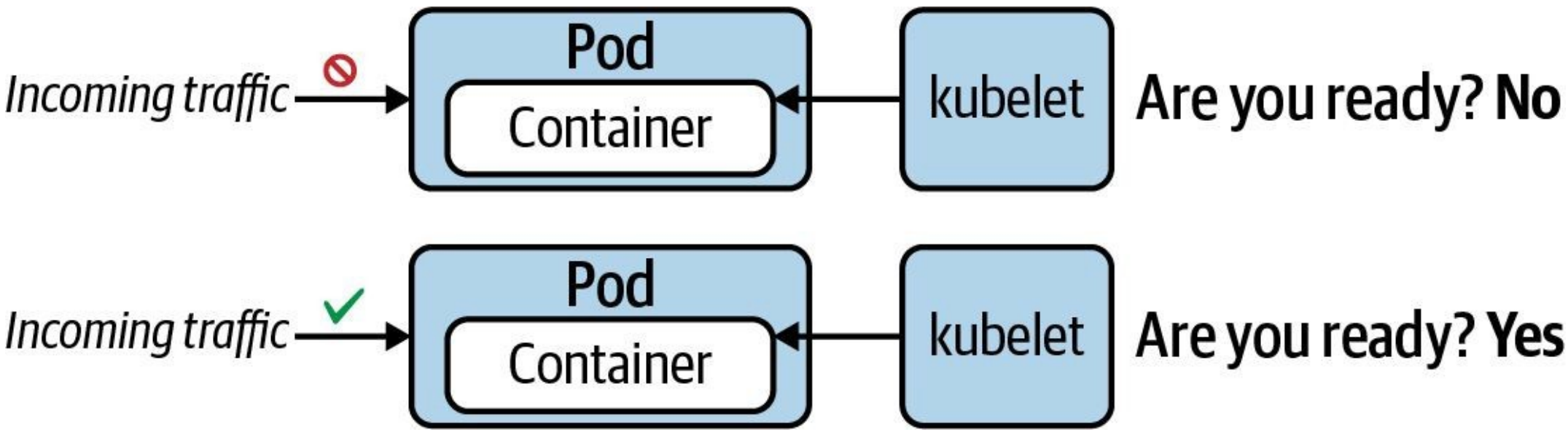
Three Probes, Three Jobs

- livenessProbe — restarts the container when it has hung or deadlocked.
- readinessProbe — removes the Pod from Service endpoints until it can serve traffic (no restart).
- startupProbe — gives slow apps time to boot; it blocks the other two until it succeeds.
- Each probe uses one handler: httpGet, tcpSocket or exec — pick by what the container exposes.

© 2026 Tertiary Infotech Academy Pte Ltd

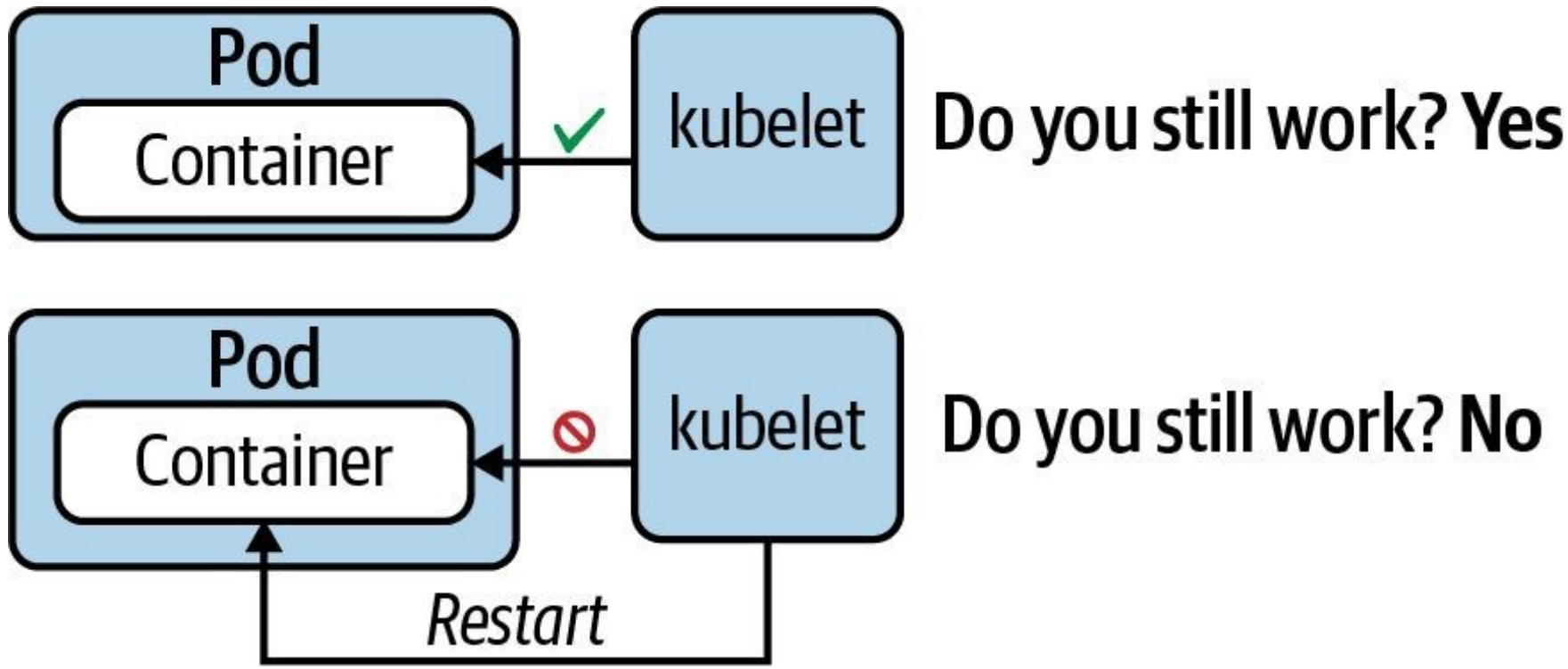
Readiness Probe

"Is application ready to handle requests?"



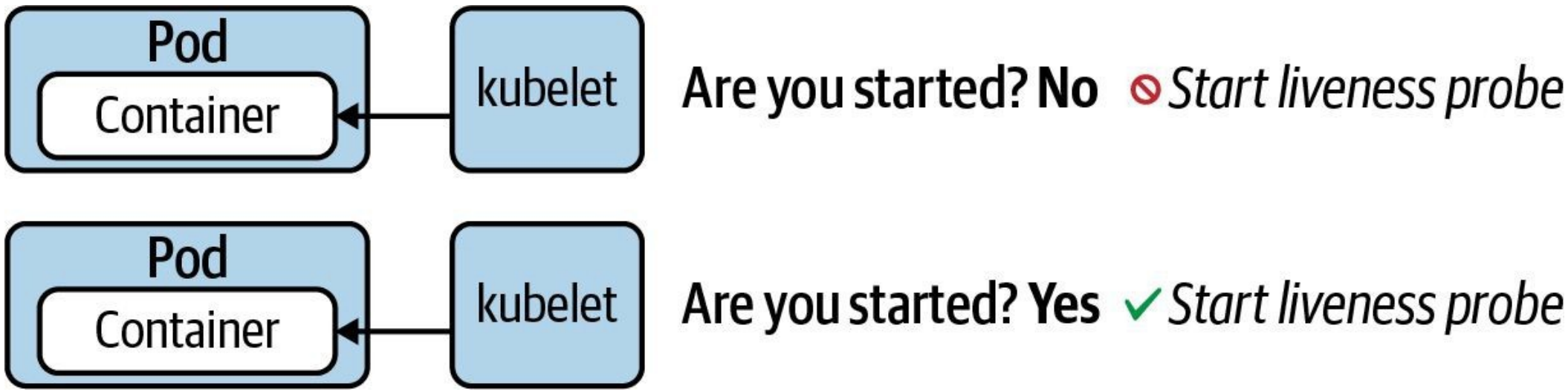
Liveness Probe

"Does the application still function without errors?"



Startup Probe

"Legacy application may need longer to start. Hold off on liveness probing."



SECTION

Recap

PROBES & OBSERVABILITY

What is a Probe?

Detection and potential correction of application runtime issues

- ● You can configure a container with health probes to execute a periodic mini-process
- that checks for certain conditions.
- ● Verification methods include running a custom command, sending a HTTP GET
- request, or opening a TCP socket connection. Pick the method most conducive to
- your application.
- ● The verification method defined by a probe is executed by the kubelet.

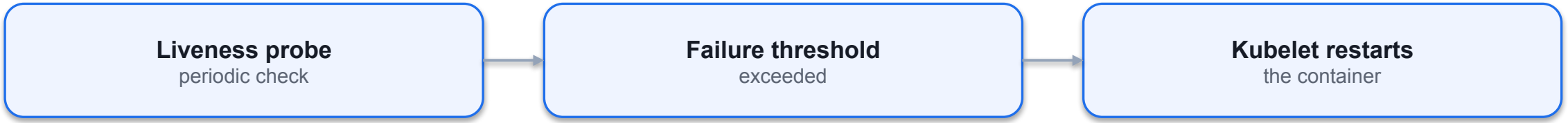
SECTION

Readiness Probe "Is application ready to handle requests?"

PROBES & OBSERVABILITY

Liveness Probe

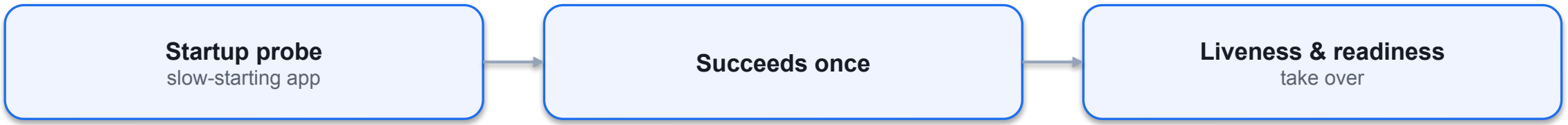
"Does the application still function without errors?"



PROBES & OBSERVABILITY

Startup Probe

"Legacy application may need longer to start. Hold off on liveness probing."



Health Verification Methods

Any verification method can be used by any of the probe types

- Method Option Description
- Custom Command exec.command Executes a command inside of the container e.g. a cat command
- and checks its exit code. Kubernetes considers an zero exit code
- to be successful. A non-zero exit code indicates an error.
- HTTP GET Request httpGet
- Sends an HTTP GET request to an endpoint exposed by the
- application. A HTTP response code in the range of 200 and 399
- indicates success. Any other response code is regarded as an
- error.
- TCP socket connection tcpSocket
- Tries to open a TCP socket connection to a port. If the connection
- could be established, the probing attempt was successful. The
- inability to connect is accounted for as an error.

Health Check Attributes

Any verification method can be used by any of the probe types

- Default
- Attribute Description
- Value
- initialDelaySeconds 0 Delay in seconds until first check is executed.
- periodSeconds 10 Interval for executing a check (e.g., every 20 seconds).
- timeoutSeconds 1 Maximum number of seconds until the check operation times out.
- successThreshold 1 Number of successful check attempts until probe is considered
- successful after a failure.
- failureThreshold 3 Number of failures for check attempts before probe is marked failed
- and takes action.
- terminationGracePeriod 30 Grace period before forcing a container to stop upon failure.
- Seconds

Defining a Readiness Probe

HTTP probes are very helpful for web applications

```
apiVersion: v1
kind: Pod
metadata:
 name: web-app
spec:
 containers:
 - name: web-app
 readinessProbe:
 image: eshop:4.6.3
 httpGet:
 Successful if HTTP status
 path: /
 code is between 200 and 399
 port: 8080
```

Defining a Liveness Probe

Querying an event log with a custom command

```
apiVersion: v1
kind: Pod
metadata:
 name: web-app
spec:
 containers:
 - name: web-app
 livenessProbe:
 image: eshop:4.6.3
 exec:
 Does the file get updated
 command:   periodically by the application
 - test `find /tmp/heartbeat.txt -mmin -1`   process?
 initialDelaySeconds: 10
```

Defining a Startup Probe

Opening a TCP socket connection exposed by the application

```
apiVersion: v1
kind: Pod
metadata:
 name: startup-pod
spec:
 containers:
 - image: httpd:2.4.46
 startupProbe:
 name: http-server
 tcpSocket:   Tries to open a TCP socket
 port: 80   connection to a port
 initialDelaySeconds: 3
 periodSeconds: 15
 livenessProbe:
```

Using a Named Port

Referencing a port by assigned name in multiple probes

```
ports:
- name: http-port
 containerPort: 8080
readinessProbe:
 httpGet:
 path: /healthz
 port: http-port
livenessProbe:
 httpGet:
 path: /healthz
 port: http-port
```

Liveness, Readiness & Startup Probes

LAB 16 Probes

Kubernetes uses three probes to manage health: livenessProbe restarts a failed container, readinessProbe removes it from Service endpoints, startupProbe gives slow apps time to boot. CKAD tests all three handlers (httpGet, tcpSocket, exec) and timing.

You'll build

Add HTTP readiness+liveness probes, trigger a liveness restart, add a TCP probe, then a startupProbe with exec.

Key commands: readinessProbe: · k exec · startupProbe: · » Three

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 16 · COMMANDS

Liveness, Readiness & Startup Probes

Step 1 — HTTP liveness + readiness probe

```
readinessProbe:
  httpGet: {path: /, port: 80}
  initialDelaySeconds: 2
  periodSeconds: 5
livenessProbe:
  httpGet: {path: /, port: 80}
  initialDelaySeconds: 10
  periodSeconds: 10
  failureThreshold: 3
```

Step 2 — Trigger a liveness failure (watch RESTARTS climb)

```
k exec web-probes -- rm /usr/share/nginx/html/index.html
sleep 35
k get pod web-probes
```


© 2026 Tertiary Infotech Academy Pte Ltd

Liveness, Readiness & Startup Probes (cont.)

Step 3 — TCP socket probe

```
readinessProbe:
  tcpSocket: {port: 6379}
  periodSeconds: 5
```

Step 4 — startupProbe for slow-starting apps

```
startupProbe:
  exec: {command: ["cat", "/tmp/ready"]}
  failureThreshold: 30
  periodSeconds: 5
```

Note Three handlers — httpGet, tcpSocket, exec. readinessProbe failure removes the Pod from Service endpoints without restarting it; livenessProbe failure restarts the container; startupProbe blocks the other two until the app is initialised.

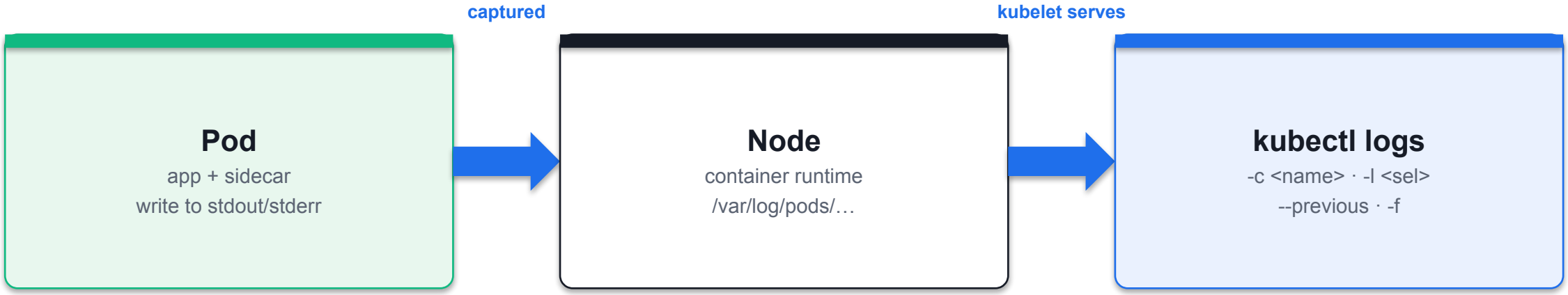
© 2026 Tertiary Infotech Academy Pte Ltd

Reading Container Logs

- Logs are whatever the container writes to stdout/stderr; the kubelet captures and serves them.
- --tail=N for recent lines; -f to stream live (like tail -f).
- --previous reads the prior terminated container — the key to diagnosing CrashLoopBackOff.
- -c <container> selects one container; -l <selector> aggregates logs across matching Pods.

© 2026 Tertiary Infotech Academy Pte Ltd

Where Logs Come From



Logs are whatever the container prints to **stdout/stderr**. Use **--previous** for a crashed container.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 17 · COMMANDS

Container Logging

Step 1 — Read recent lines and follow a live stream

```
k logs noisy --tail=10
k logs -f noisy &        # -f streams like tail -f
kill %1
```

Step 2 — Retrieve logs from a crashed container

```
k logs crasher --previous | head -5
```

Step 3 — Multi-container Pod logs

```
k logs multi -c app | head -3
k logs multi -c sidecar | head -3
k logs multi --all-containers=true --prefix=true | head -8
```


© 2026 Tertiary Infotech Academy Pte Ltd

Container Logging (cont.)

Step 4 — Aggregate logs across a Deployment

```
k logs deployment/fleet --tail=3
k logs -l app=fleet --prefix=true --tail=2
```

Note --tail=N for recent lines, -f to stream, --previous for crash loops, -c <name> for a specific container, -l <selector> to aggregate matching Pods. --prefix=true labels each line with [pod/container].

© 2026 Tertiary Infotech Academy Pte Ltd

Container Logging

✅ Test it

k logs crasher --previous shows the message printed before the crash, and k logs -l app=fleet --prefix=true interleaves lines from all three Pods.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

MEASURE IT

Resource Usage at a Glance

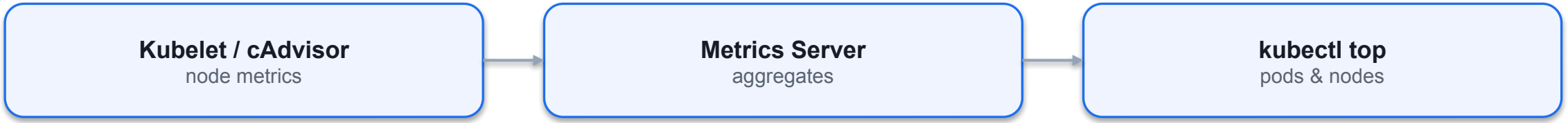
- kubectl top shows live CPU and memory for nodes and Pods — it requires metrics-server.
- metrics-server scrapes each kubelet's cAdvisor and serves the aggregated Metrics API.
- Sort with --sort-by=cpu / --sort-by=memory; span all namespaces with -A.
- On Killercoda's self-signed certs, patch the Deployment with --kubelet-insecure-tls.

© 2026 Tertiary Infotech Academy Pte Ltd

PROBES & OBSERVABILITY

Metrics Server

Retrieving, inspecting, and using resource metrics



kubectl top and Metrics Server

LAB 18

Metrics

kubectl top shows real-time CPU and memory for nodes and Pods. It requires the metrics-server add-on. CKAD tests installing metrics-server, reading node/Pod metrics, and sorting by resource usage.

You'll build

Install metrics-server, read node metrics, generate CPU load, then sort Pod metrics by CPU and memory.

Key commands: kubectl apply · k top · k run · » `kubectl

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 18 · COMMANDS

kubectl top and Metrics Server

Step 1 — Install metrics-server (and trust kubelet certs on Killercoda)

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml
kubectl patch -n kube-system deployment metrics-server --type=json -p='
[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'
until kubectl top node 2>/dev/null; do sleep 5; done
```

Step 2 — Top nodes

```
k top node        # CPU cores/% and memory bytes/% per node
```

Step 3 — Generate load, then top Pods sorted

```
k run cpu-burner --image=busybox -- sh -c 'while true; do :; done'
sleep 30
k top pod --sort-by=cpu
k top pod --sort-by=memory
k top pod -A --sort-by=cpu | head -10
```


© 2026 Tertiary Infotech Academy Pte Ltd

kubectl top and Metrics Server (cont.)

Note kubectl top needs metrics-server running. --kubelet-insecure-tls is required on Killercoda's self-signed certs. cpu-burner should top the CPU sort; -A spans all namespaces.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 18 · TEST IT

kubectl top and Metrics Server

✅ Test it

After ~30s, k top node reports CPU/memory and k top pod --sort-by=cpu lists cpu-burner at the top of the usage ranking.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

Debugging Pods and Events

LAB 19 Debugging

CKAD regularly includes broken workloads you must diagnose and fix under time pressure. Identify and resolve ImagePullBackOff, CrashLoopBackOff and OOMKilled, and use kubectl debug for live triage.

You'll build

Diagnose three classic failures with describe/logs/events, then inject an ephemeral debug container.

Key commands: k describe · k get · » ImagePullBackOff

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

Debugging Pods and Events (cont.)

Step 4 — Cluster-wide events + ephemeral debug container

```
k get events -A --sort-by=.lastTimestamp | tail -20
k get events --field-selector type=Warning
k debug crash -it --image=busybox --target=crash -- sh
```

Note ImagePullBackOff → wrong image/tag. CrashLoopBackOff → use logs --previous. OOMKilled → raise the memory limit. kubectl debug injects an ephemeral container sharing the target's namespaces — vital for distroless images with no shell.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 19 · TEST IT

Debugging Pods and Events

✅ Test it

k describe reveals the correct failure reason for each Pod (ImagePullBackOff, CrashLoopBackOff, OOMKilled), and k debug opens a shell inside the running Pod.

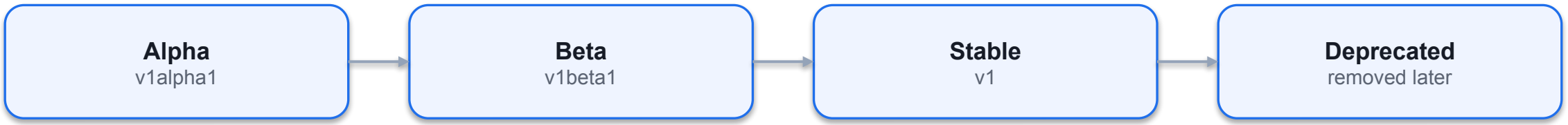
Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

MAINTENANCE

API Deprecations

Kubernetes release cycle, API deprecations and removals



MAINTENANCE

Deprecation Policy

Some APIs will be deprecated and eventually removed

- ● The Kubernetes project releases 3 versions per year (see release cadence).
- ● With each release, an API can become deprecated which means that is scheduled
- for removal or replacement. You can find more information about the details in the
- Kubernetes Deprecation Policy.
- ● The use of a deprecated API renders a warning message when creating the object
- that uses it.
- ● The Deprecated API Migration Guide shows a list of deprecated APIs and the
- scheduled releases that will remove support for the API.

MAINTENANCE

Listing Available API version

kubectl can discover API version useable in manifests

```
$ kubectl api-versions
admissionregistration.k8s.io/v
1 apiextensions.k8s.io/v1
apiregistration.k8s.io/v1
apps/v1
authentication.k8s.io/v1
authorization.k8s.io/v1
autoscaling/v1
autoscaling/v2
...
```

MAINTENANCE

Deprecated HPA API

API was deprecated with Kubernetes 1.23+, to be removed with 1.26

```
apiVersion: autoscaling/v2beta2
kind: HorizontalPodAutoscaler
metadata:
  name: php-apache
spec:
  scaleTargetRef:
  apiVersion: apps/v1
  kind: Deployment
  name: php-apache
  minReplicas: 1
  maxReplicas: 10
```

MAINTENANCE

Deprecated HPA API Warning Message

kubectl renders a warning message, but creates object

```
$ kubectl create -f hpa.yaml
Warning: autoscaling/v2beta2 HorizontalPodAutoscaler is deprecated
 in v1.23+, unavailable in v1.26+; use autoscaling/v2
HorizontalPodAutoscaler
```

MAINTENANCE

Removed ClusterRole API

Uses removed API with Kubernetes version 1.22

```
apiVersion: rbac.authorization.k8s.io/v1beta1
kind: ClusterRole
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

MAINTENANCE

Removed ClusterRole API Error Message

kubectl renders an error message, as API does not exist anymore

```
$ kubectl create -f clusterrole.yaml
error: resource mapping not found for name: "pod-reader" namespace:
 "" from "clusterrole.yaml": no matches for kind "ClusterRole" in
 version "rbac.authorization.k8s.io/v1beta1"
```

MAINTENANCE

Using the ClusterRole API Replacement

Usage of replacement API

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list"]
```

API Deprecations & kubectl explain

LAB 20 API Deprecations

The Kubernetes API evolves — old versions are deprecated and removed. CKAD tests kubectl explain, api-resources and api-versions. You must find the correct apiVersion for any resource on exam day using only kubectl.

You'll build

List resources and versions, explore schemas with explain, detect a removed API, and look up the current version.

Key commands: k api-resources · k explain · Pod/Service/ConfigMap/Secret/SA/Quota/LimitRange -> · » `kubectl

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 20 · COMMANDS

API Deprecations & kubectl explain

Step 1 — List resources, groups and served versions

```
k api-resources --api-group=apps
k api-resources --api-group=batch
k api-versions | sort
```

Step 2 — Explore a schema with explain

```
k explain deployment.spec.strategy.rollingUpdate
k explain pod.spec.containers.resources --recursive | head -30
```

Step 3 — Detect a removed API and find the replacement

```
# extensions/v1beta1 Ingress was removed in 1.22
k explain ingress --api-version=networking.k8s.io/v1 | head -10
k api-resources | grep -i ingress
```


© 2026 Tertiary Infotech Academy Pte Ltd

API Deprecations & kubectl explain (cont.)

Step 4 — Stable apiVersions to memorise (v1.35)

```
Pod/Service/ConfigMap/Secret/SA/Quota/LimitRange  -> v1
Deployment/StatefulSet/DaemonSet/ReplicaSet        -> apps/v1
Job/CronJob                                        -> batch/v1
Ingress/NetworkPolicy                              -> networking.k8s.io/v1
Role/RoleBinding/ClusterRole(+Binding)             -> rbac.authorization.k8s.io/v1
HorizontalPodAutoscaler                            -> autoscaling/v2
```

Note kubectl api-resources shows the correct apiVersion in its APIVERSION column — the exam-legal way to look it up. kubectl explain <res>.<field> documents fields without leaving the terminal.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 20 · TEST IT

API Deprecations & kubectl explain

✅ Test it

k api-resources | grep -i ingress shows networking.k8s.io/v1, and k explain deployment.spec.strategy.rollingUpdate prints the maxSurge/maxUnavailable fields.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

DOMAIN 4

Configuration & Security

ConfigMaps · Secrets · securityContext · ServiceAccounts · RBAC · quotas (25%)

© 2026 Tertiary Infotech Academy Pte Ltd

EXTERNAL CONFIG

Decouple Config from Image

- A ConfigMap holds non-secret key/value data; create it with --from-literal, --from-file or --from-env-file.
- Inject one key with configMapKeyRef, all keys with envFrom, or mount it as a file volume.
- Volume-mounted ConfigMaps update live (~60s); env-var injections are fixed at Pod startup.
- Keep config out of the image so the same image runs in every environment.

© 2026 Tertiary Infotech Academy Pte Ltd

CONFIGMAPS & SECRETS

ConfigMaps and Secrets

Defining and consuming configuration data

ConfigMap
plain configuration

Secret
base64, restricted

Env variables
key by key

Volume mount
files in the Pod

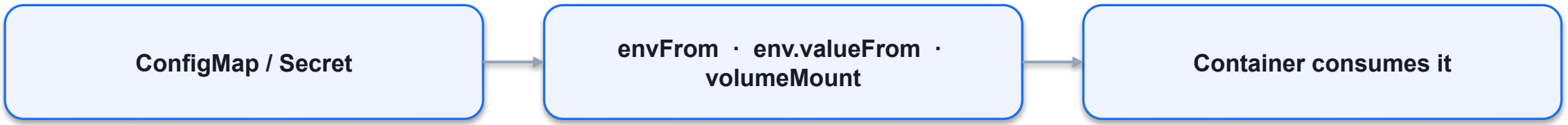
Defining Configuration Data

Control runtime behavior with centralized information

- ● Stored as key-value pairs in dedicated Kubernetes primitives with a lifecycle
- decoupled from consuming Pod.
- ● ConfigMap: Plain-text values suited for configuring applications e.g. flags, or URLs.
- ● Secret: Base64-encoded values suited for storing sensitive data like passwords,
- API keys, or SSL certificates. Values are not encrypted!
- ● ConfigMap and Secret objects are stored in etcd in unencrypted form by default.
- Encryption of data in etcd is configurable.

Consuming Configuration Data

Applicable to ConfigMap and Secret



Creating a ConfigMap with Imperative Approach

Key-value pairs can be parsed from different sources

```
$ kubectl create configmap db-config --from-literal=db=staging
configmap/db-config created
$ kubectl create configmap db-config --from-env-file=config.env
configmap/db-config created
$ kubectl create configmap db-config --from-file=config.txt
configmap/db-config created
$ kubectl create configmap db-config --from-file=app-config
configmap/db-config created
```

Source Options for Data Parsed by a ConfigMap

Configuration options and example values

- Option Description Example Value
- --from-literal Literal values, which are key-value pairs as plain --from-literal=locale=en_US
- text.
- --from-env-file A file that contains key-value pairs and expects --from-env-file=config.env
- them to be environment variables.
- --from-file Huge page resource type. --from-file=app-config.json
- --from-file A directory with one or many files. --from-file=config-dir

ConfigMap - Imperative Approach

Configuration options and example values

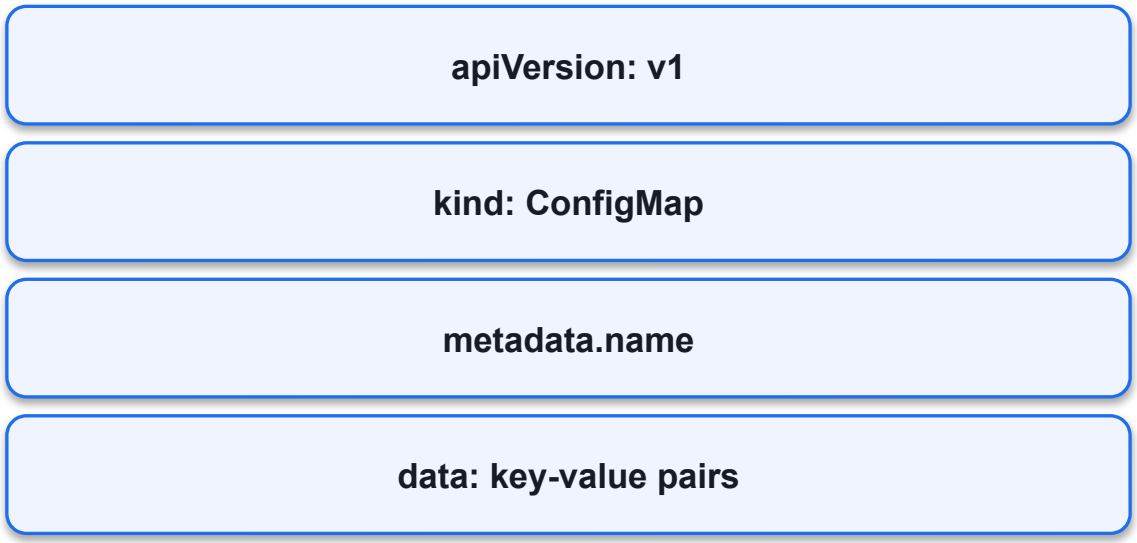
--from-literal=key=value

--from-file=path

--from-env-file=file.env

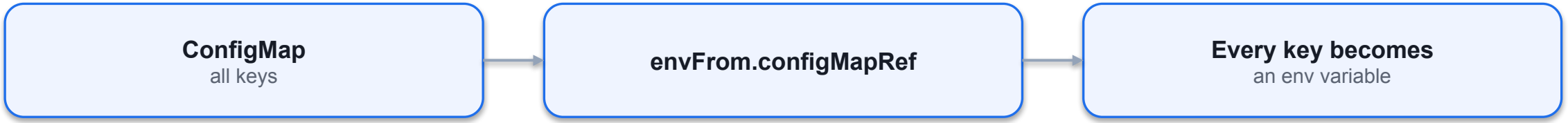
ConfigMap YAML Manifest

Key-value pair definition under data attribute



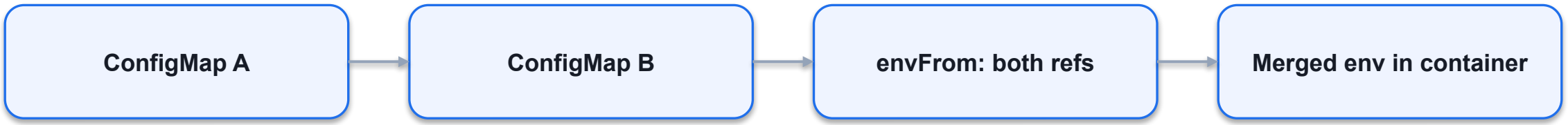
ENV Variables - Define all key-value pairs in configmap

Key-value pair definition under data attribute



ENV Variables - Define via Multiple ConfigMaps

Key-value pair definition under data attribute



Consuming Changed ConfigMap Data

Containers will not refresh consumed data upon a change

- ● The key-value pairs of a ConfigMap can be changed for a live object.
- ● Changed ConfigMap key-value pairs consumed by containers will not be reloaded
- automatically for an already-running Pod.
- ● The application code needs to implement logic to either reload the configuration
- data periodically or on-demand.

LAB 21 · TEST IT

ConfigMaps — Env & Volume Injection

✅ Test it

The single-key Pod logs COLOR=blue, the envFrom Pod shows all four variables, and the mounted ConfigMap appears as files under /etc/app.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

SENSITIVE DATA

Secrets vs ConfigMaps

- Secrets work like ConfigMaps but values are base64-encoded and access is more tightly controlled.
- Types: generic, tls (kubernetes.io/tls) and docker-registry (for imagePullSecrets).
- Inject with envFrom: secretRef (all keys) or secretKeyRef (one key); mount with defaultMode: 0400.
- Base64 is not encryption — restrict access with RBAC and enable encryption at rest.

© 2026 Tertiary Infotech Academy Pte Ltd

Consuming a Secret as Environment Variables

Accessed values are base64-decoded

```
apiVersion: v1 kind: Pod
metadata:
  name: backend
spec:
  containers:
  - image: bmuschko/web-
  app:1.0.1
    name: backend
```

```
envFrom:
- secretRef:
    name: mysecret
```

```
$ kubectl exec -it backend -- env
pwd=s3cre!
```

Consuming a Secret as a Volume

A good choice for processing a machine-readable configuration file

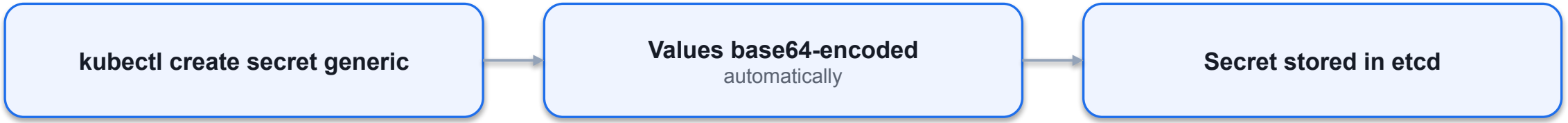
```
apiVersion: v1
kind: Pod
metadata:
  name:
  backend
spec:
  containers:
    - name:
      backe
      nd
      image:
bmuschko/
web-
app:1.0.1
      volume
```

```
$ kubectl exec -it backend -- /bin/sh
# ls -l /var/app ssh-privatekey


# cat /var/app/ssh-privatekey
-----BEGIN RSA PRIVATE KEY----- Proc-Type:
4,ENCRYPTED
DEK-Info:
AES-128-CBC,8734C9153079F2E8497C8075289E BBF1
...
-----END RSA PRIVATE KEY-----
```

Creating a Secret with Imperative Approach

Parsed values are base64-encoded automatically



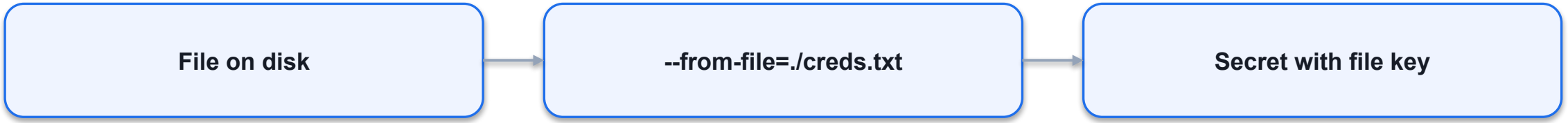
Source Options for Data Parsed by a Generic Secret

Configuration options and example values

- Option Description Example Value
- --from-literal Literal values, which are key-value pairs as --from-literal=password=secret
- plain text.
- --from-env-file A file that contains key-value pairs and --from-env-file=config.env
- expects them to be environment variables.
- --from-file Huge page resource type. --from-file=id_rsa=~/.ssh/id_rsa
- --from-file A directory with one or many files. --from-file=config-dir

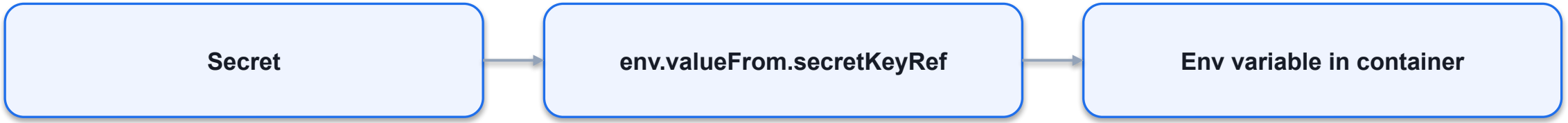
Create Secret from File

Configuration options and example values



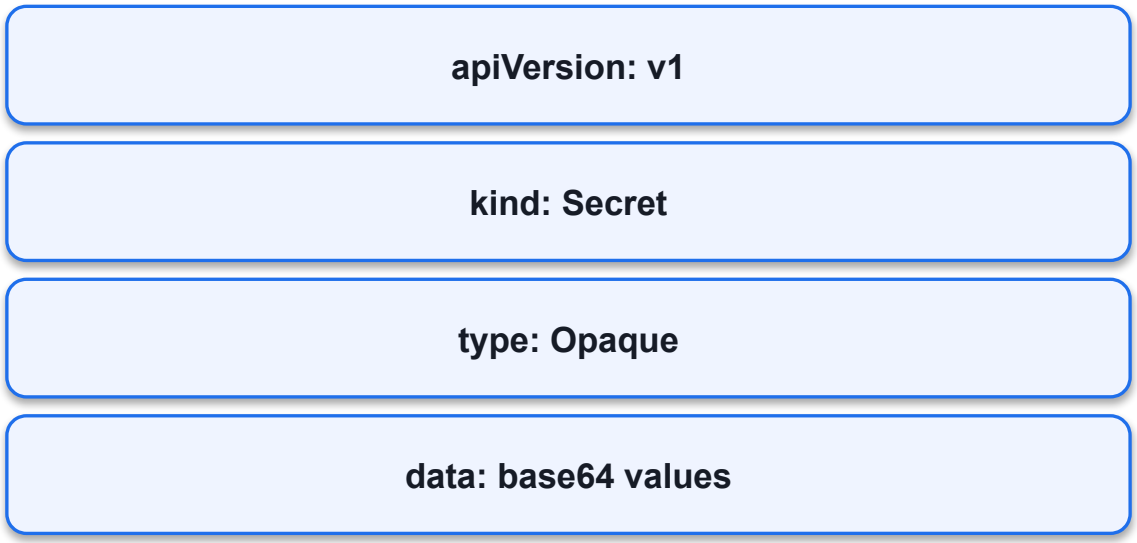
ENV Variables from Secret

Configuration options and example values



Declarative Approach

Configuration options and example values



SECTION

Describing Secrets

SECTION

Viewing Secrets

Secrets Are Not Really Secret

Techniques that help with making Secrets more secure

- ● Secret objects are stored in etcd in unencrypted form by default. Encryption of data
- in etcd is configurable.
- ● Define RBAC permissions to only allow developers to create, view, and modify
- Secret objects in dedicated namespaces. An even stricter policy could only allow
- administrators to manage Secrets.
- ● Use Kubernetes-external, third-party Secrets services for managing sensitive data,
- e.g. AWS Secrets Manager or HashiCorp Vault. You can consume the data with the
- help of the External Secrets Operator.

Secrets

LAB 22

Secrets

Secrets are like ConfigMaps but base64-encoded with stricter access. CKAD tests generic, tls and docker-registry types, env-var injection, file mounts with defaultMode, and decoding values. Secrets are not encrypted — protect them with RBAC.

You'll build

Create a generic Secret, decode it, inject as env vars, mount with 0400 mode, then create tls and docker-registry Secrets.

Key commands: k create · envFrom: · volumes: · » Types:

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 22 · COMMANDS

Secrets

Step 1 — Create a generic Secret and decode a value

```
k create secret generic db-cred \
  --from-literal=DB_USER=admin --from-literal=DB_PASS=s3cr3t
k get secret db-cred -o jsonpath='{.data.DB_PASS}' | base64 -d; echo
```

Step 2 — Inject Secret keys as env vars

```
envFrom:
- secretRef: {name: db-cred}
```

Step 3 — Mount a Secret as files with restrictive mode

```
volumes:
- name: s
  secret:
    secretName: db-cred
    defaultMode: 0400
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 22 · COMMANDS

Secrets (cont.)

Step 4 — TLS and docker-registry Secrets

```
k create secret tls demo-tls --cert=tls.crt --key=tls.key
k create secret docker-registry myreg \
  --docker-server=registry.example.com --docker-username=u \
  --docker-password=p --docker-email=u@example.com
```

Note Types: generic, tls (kubernetes.io/tls), docker-registry. envFrom: secretRef injects all keys; secretKeyRef injects one. Reference a registry Secret via spec.imagePullSecrets. Base64 ≠ encryption — restrict with RBAC.

© 2026 Tertiary Infotech Academy Pte Ltd

Secrets

✅ Test it

base64 -d on .data.DB_PASS returns s3cr3t, the mounted Secret files are mode 0400, and the tls Secret reports type kubernetes.io/tls.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

LEAST PRIVILEGE

Run Containers Safely

- securityContext sets the user, group and privileges a container runs with.
- Pod-level applies to all containers; container-level overrides one container.
- runAsNonRoot: true blocks any image whose default user is root.
- Hardened pattern: readOnlyRootFilesystem + capabilities.drop:[ALL] + allowPrivilegeEscalation:false.

© 2026 Tertiary Infotech Academy Pte Ltd

SECTION

Security Context Privilege and access control for a Pod

What is a Security Context?

Defines security settings for containers of a Pod

- ● Modeled as a set of attributes within the Pod specification, e.g. for configuring Linux
- capabilities, file access control, privileges to parent process or host system.
- ● Can be applied to all containers in Pod with spec.securityContext or
- for individual containers with spec.containers[].securityContext.

Security Context API

All attributes can be discovered in documentation for the Kubernetes version

- API Type Description
- PodSecurityContext Defines Pod-level security attributes.
- SecurityContext Defines container-level security attributes.

Defining a Security Context

Pod- and/or container-level definition

```
apiVersion: v1
kind: Pod
metadata:
 name: secured-pod
spec:
 securityContext:   Defined on the Pod-level (applies
 fsGroup: 3000   to all containers)
 volumes:
 - name: data-vol
 emptyDir: {}
 containers:
 - image: nginx:1.18.0
 name: secured-container
 volumeMounts:
```

Inspecting the Runtime Behavior

Rendering the file system Linux group ID

```
$ kubectl exec secured-pod -it -- /bin/sh
# cd /data/app
# touch hello.txt
# ls -l
total 0
-rw-r--r-- 1 root 3000 0 Apr 30 18:27 hello.txt
# exit
  File system group ID is
  assigned automatically
```

Overriding a Value on the Container-Level

Container-level definition wins

```
apiVersion: v1
kind: Pod
metadata:
 name: non-root-success
spec:
 securityContext:
 runAsNonRoot: false
 containers:
 - image: bitnami/nginx:1.23.1
 name: nginx
 securityContext:
 Takes precedence
 runAsNonRoot: true
```

SecurityContext

LAB 23 SecurityContext

securityContext controls the identity and privileges of containers. CKAD asks you to enforce non-root execution, a read-only root filesystem and dropped Linux capabilities — at Pod level (all containers) or container level (one override).

You'll build

Run as a set user/group, enforce runAsNonRoot, lock the root filesystem, then drop all capabilities.

Key commands: securityContext: · » Pod-level

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 23 · COMMANDS

SecurityContext

Step 1 — Run as a specific user and group (Pod level)

```
securityContext:
  runAsUser: 1000
  runAsGroup: 3000
  fsGroup: 2000     # applies to volume mounts
```

Step 2 — Enforce runAsNonRoot (blocks root images)

```
securityContext:
  runAsNonRoot: true
# nginx runs as root -> Pod refuses to start
```

Step 3 — Read-only root filesystem (container level)

```
securityContext:
  readOnlyRootFilesystem: true
# mount an emptyDir for any writable path (e.g. /tmp)
```


© 2026 Tertiary Infotech Academy Pte Ltd

SecurityContext (cont.)

Step 4 — Drop Linux capabilities (hardened container)

```
securityContext:
  capabilities:
    drop: ["ALL"]
    add: ["NET_BIND_SERVICE"]
  allowPrivilegeEscalation: false
```

Note Pod-level securityContext applies to all containers; container-level overrides one. The CKAD hardened pattern = runAsNonRoot: true + readOnlyRootFilesystem: true + capabilities.drop: [ALL] + allowPrivilegeEscalation: false.

© 2026 Tertiary Infotech Academy Pte Ltd

SecurityContext

✅ Test it

The non-root Pod logs uid=1000 gid=3000, the runAsNonRoot nginx Pod refuses to start, and writes to the read-only root filesystem are rejected.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

Pod Identity

- Every Pod runs as a ServiceAccount — the default one has minimal permissions.
- spec.serviceAccountName attaches a dedicated ServiceAccount to a Pod.
- A token, CA cert and namespace are projected into /var/run/secrets/kubernetes.io/serviceaccount/.
- automountServiceAccountToken: false enforces least privilege; kubectl create token mints short-lived tokens.

© 2026 Tertiary Infotech Academy Pte Ltd

What is a Service Account?

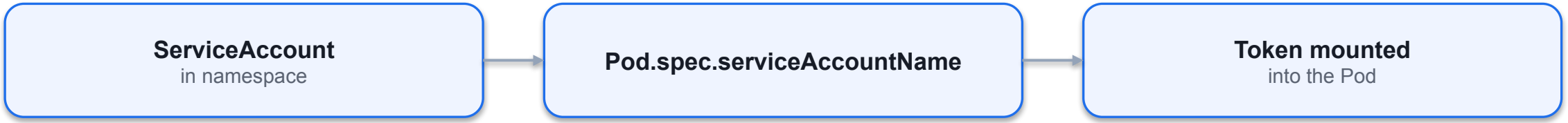
Non-human request to Kubernetes API from a process

- ● Processes running outside of Kubernetes or processes running inside of a container
- may need to interact with the Kubernetes API. A Service Account allows for
- authenticating with the API server through an authentication token.
- ● Authorization is controlled through RBAC and assigned to the Service Account.
- ● If not assigned explicitly, a Pod uses the default Service Account. The default
- Service Account has the same permissions as an unauthenticated user.
- ● Example: “I have a CI/CD process that retrieves Pod information by making calls to
- the Kubernetes API.”

SECURITY & RBAC

Service Account as Subject

Pod assign the Service Account by name



Service Account YAML Manifest

In a Pod manifest, refer to the Service Account by name

```
apiVersion: v1   apiVersion: v1
kind: ServiceAccount   kind: Pod
metadata:   metadata:
 name: sa-api   name: list-pods
 namespace: k97   namespace: k97
 spec:
 serviceAccountName: sa-api
 ...
```

Accessing Service Account Authentication Token

Automounted at specific Volume mount path

```
$ kubectl describe pod list-pods -n k97
...
Containers
 : pods:
 ...
 Mounts:
 /var/run/secrets/kubernetes.io/serviceaccount from
 kube-api-access-xnkwd (ro)
...
 /var/run/secrets/kubernetes.io/serviceaccount/toke
$ kubectl
 n   exec -it list-pods -n k97 -- /bin/sh
# cat
eyJhbGciOiJSUzI1N..
```

SECURITY & RBAC

Using Service Account in Pod

Enables authentication token auto-mounting by default

```
apiVersion: v1
kind: Pod
metadata:
 name: list-pods
 namespace: k97   List all Pods in the
spec:   Assigned name of
 serviceAccountName: sa-api   namespace k97
 containers:   Service Account   via an API call
 - name: pods
 image: alpine/curl:3.14
 command: ['sh', '-c', 'while true; do curl -s -k -m 5 -H
 "Authorization: Bearer $(cat
 /var/run/secrets/kubernetes.io/
 serviceaccount/token)" https://kubernetes.default.svc.cluster.
```

SECURITY & RBAC

RoleBinding YAML Manifest

Default RBAC policies don’t span beyond kube-system

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: serviceaccount-pod-rolebinding
 namespace: k97   Maps the Service
roleRef:
 Account as a
 apiGroup: rbac.authorization.k8s.io
 subject
 kind: ClusterRole
 name: list-pods-clusterrole
subjects:
 kind: ServiceAccount
- apiGroup: rbac.authorization.k8s.io
```

ServiceAccounts

LAB 24 ServiceAccounts

Every Pod runs as a ServiceAccount; the default one has minimal RBAC. CKAD tests creating dedicated ServiceAccounts, attaching them to Pods, disabling the auto-mounted token, and requesting short-lived tokens with kubectl create token.

You'll build

Create a ServiceAccount, attach it to a Pod, inspect the projected token, disable auto-mount, then mint a token.

Key commands: k create · k exec · k patch · TOKEN=$(k create · » `spec.serviceAccountName`

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 24 · COMMANDS

ServiceAccounts

Step 1 — Create a ServiceAccount and attach it to a Pod

```
k create serviceaccount app-sa
# Pod spec:
spec:
  serviceAccountName: app-sa
k get pod app -o jsonpath='{.spec.serviceAccountName}'; echo
```

Step 2 — Inspect the auto-mounted token inside the Pod

```
k exec app -- ls /var/run/secrets/kubernetes.io/serviceaccount/
```

Step 3 — Disable token auto-mount (least privilege)

```
k patch sa app-sa -p '{"automountServiceAccountToken": false}'
```


© 2026 Tertiary Infotech Academy Pte Ltd

ServiceAccounts (cont.)

Step 4 — Request a short-lived ad-hoc token

```
TOKEN=$(k create token app-sa --duration=1h)
echo $TOKEN | cut -c1-60
```

Note spec.serviceAccountName attaches a custom SA. Token, CA and namespace are projected under /var/run/secrets/kubernetes.io/serviceaccount/. automountServiceAccountToken: false enforces least privilege; kubectl create token replaces the old Secret-backed tokens (removed in 1.24+).

© 2026 Tertiary Infotech Academy Pte Ltd

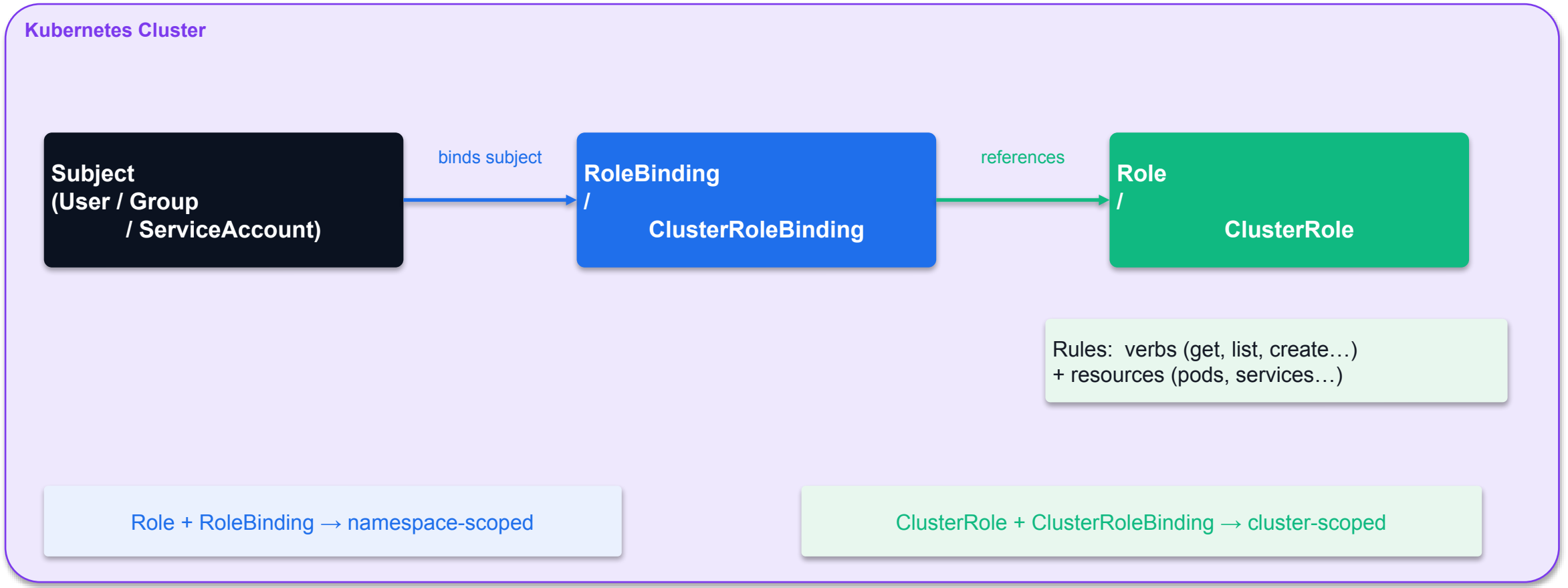
AUTHORIZATION

Role-Based Access Control

- A Role/ClusterRole lists allowed verbs (get, list, create, delete…) on resources.
- A RoleBinding/ClusterRoleBinding ties that Role to a subject (ServiceAccount, user or group).
- Role + RoleBinding = one namespace; ClusterRole + ClusterRoleBinding = cluster-wide.
- ClusterRole + RoleBinding limits a cluster role's verbs to a single namespace.
- Validate without logging in: kubectl auth can-i <verb> <resource> --as=<identity>.

© 2026 Tertiary Infotech Academy Pte Ltd

RBAC — Role-Based Access Control Model



© 2026 Tertiary Infotech Academy Pte Ltd

Role vs ClusterRole

Role (Namespace-scoped)

- Scoped to a single namespace
- kubectl get roles -n <ns>
- Common uses: app-specific read/write access
  - Example: allow listing pods in 'default' namespace only
- Paired with RoleBinding in the same namespace

ClusterRole (Cluster-scoped)

- Applies across all namespaces (or non-namespaced resources)
- kubectl get clusterroles
- Common uses: admin, monitoring, ingress controller
  - Example: allow listing nodes, PVs, namespaces cluster-wide
- Paired with ClusterRoleBinding or namespaced RoleBinding

© 2026 Tertiary Infotech Academy Pte Ltd

Defining a Role and RoleBinding

Role: grant read access to pods in the 'dev' namespace

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

RoleBinding: bind the Role to a ServiceAccount

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: dev
subjects:
- kind: ServiceAccount
  name: my-app-sa
  namespace: dev
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```


© 2026 Tertiary Infotech Academy Pte Ltd

RBAC · CLUSTER-WIDE

ClusterRole and ClusterRoleBinding

ClusterRole: read access to nodes (non-namespaced resource)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list"]
```

ClusterRoleBinding: bind to a user

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: read-nodes-global
subjects:
- kind: User
  name: jane
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: node-reader
  apiGroup: rbac.authorization.k8s.io
```


© 2026 Tertiary Infotech Academy Pte Ltd

RBAC — Essential kubectl Commands

Create a Role imperatively (quick in exam)

```
kubectl create role pod-reader \
  --verb=get,list,watch \
  --resource=pods \
  --namespace=dev
```

Bind the Role to a ServiceAccount

```
kubectl create rolebinding read-pods \
  --role=pod-reader \
  --serviceaccount=dev:my-app-sa \
  --namespace=dev
```

Check permissions for a ServiceAccount (auth can-i)

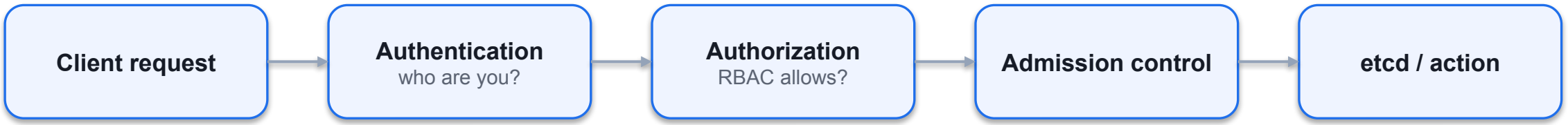
```
kubectl auth can-i list pods \
  --as=system:serviceaccount:dev:my-app-sa \
  --namespace=dev
# yes
```


© 2026 Tertiary Infotech Academy Pte Ltd

SECURITY & RBAC

Processing a Request to the API Server

Any client request to the API server needs to pass four phases



SECURITY & RBAC

API Request Processing Phases

Every request has to pass those phases in the following order

- ● Authentication: Validates the identity of the caller using a supported
- authentication strategy, e.g. client certificates or bearer tokens.
- ● Authorization: Determines if the identity provided in the first stage can access
- the verb and HTTP path request. This is usually implemented with RBAC.
- ● Admission Controller: Verifies if the request is well-formed and potentially needs
- to be modified before the request is processed. Ensures that the resource
- included in the request is valid (could also be implemented as part of admission
- control).

SECURITY & RBAC

Authentication via Credentials in Kubeconfig

The kubeconfig file at $HOME/.kube/config is evaluated by kubectl

```
apiVersion: v1
kind: Config
clusters:
- cluster:
 certificate-authority:
 /Users/bmuschko/.minikube/ca.crt
 extensions:
 - extension:
 last-update: Thu, 04 May 2023 08:48:09 MDT
 provider: minikube.sigs.k8s.io
 version: v1.30.1
 server:
 name: cluster_info
 API server endpoint for
```

SECURITY & RBAC

Managing Kubeconfig and Current Context

Interaction with kubeconfig configuration

```
$ kubectl config view   Renders the contents of
  the kubeconfig file
apiVersion:
v1 kind:
Config
current-context: bmuschko
...   Shows the
  currently-selected context
$ kubectl config current-context
bmuschko
  Switches to the context
$ kubectl config use-context johndoe   defined in the kubeconfig
Switched to context "johndoe".
$ kubectl config current-context
```

SECURITY & RBAC

What is RBAC?

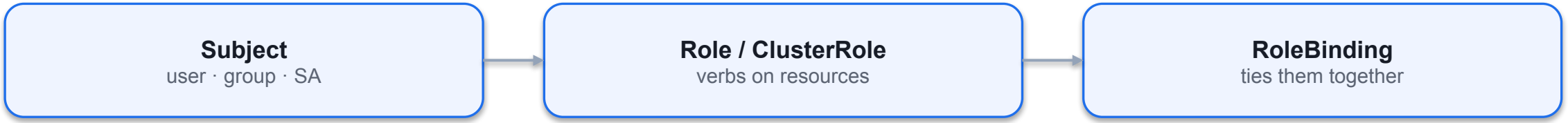
Restricting access control for clients interacting with API server

- ● Defines policies for users, groups, and ServiceAccounts by allowing or disallowing
- access to manage API resources.
- ● Enabling and configuring RBAC is mandatory for any organization with a strong
- emphasis on security.
- ● Example: “The human user John is only allowed to list and create Pods and
- Deployments, but nothing else.”

SECURITY & RBAC

RBAC High-Level Overview

Three key elements for understanding concept



SECURITY & RBAC

Involved RBAC Primitives

Restrict access to certain operations of API resources for subjects

Role
namespaced rules

ClusterRole
cluster-wide rules

RoleBinding
grants in a namespace

ClusterRoleBinding
grants cluster-wide

SECURITY & RBAC

Namespace and Cluster-Wide Permissions

Defining permission scopes

- ● The Role maps API resources to verbs for a single namespace.
- ● The RoleBinding maps a reference of a Role to one or many subjects for a
- single namespace.
- ● To apply permissions across all namespaces of a cluster or cluster-wide resources,
- use the corresponding API primitives ClusterRole and ClusterRoleBinding. The
- configuration options of the manifests are the same.

Creating a Role with Imperative Approach

Defines verbs and resources in comma-separated list

```
 $ kubectl create role read-only --verb=list,get,watch
  --resource=pods,deployments,services
 role.rbac.authorization.k8s.io/read-only created
  Operations: Only allow listing,
Resources: Primitives the   getting, watching
operations should apply to
```

SECURITY & RBAC

Role YAML Manifest

Can define a list of rules in an array

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
 name: read-only
rules:
- apiGroups: [""]
 resources: ["pods", "services"]
 verbs: ["list", "get", "watch"]   Resources with groups need
- apiGroups: ["apps"]   to be spelled out explicitly (in
 resources: ["deployments"]
 verbs: ["list", "get", "watch"]
 this case apps/Deployment
 )
```

Getting Role Details

Maps objects of a Kubernetes primitive to verbs

```
$ kubectl describe role read-only
Name:   read-only
Labels:   <none>
Annotations: <none>
PolicyRule:
 Resources   Non-Resource URLs   Resource Names   Verbs
 pods   []   []   [list get watch]
 services   []   []   [list get watch]
 deployments.apps   []   []   [list get watch]
```

Creating a RoleBinding with Imperative Approach

Maps subject to Role

```
$ kubectl create rolebinding read-only-binding --role=read-only
--user=johndoe
rolebinding.rbac.authorization.k8s.io/read-only-binding created
Subject: Binds a user to   Role: Reference the name of
the Role   the object
```

Getting RoleBinding Details

Only shows mapping between Role and subjects

```
$ kubectl describe rolebinding read-only-binding
Name:   read-only-binding
Labels:   <none>
Annotations:   <none>
Role:
 Kind: Role
 Name: read-only
Subjects:
 Kind Name Namespace
 User   johndoe
```

RBAC — Roles & RoleBindings

LAB 25

RBAC

RBAC controls who can do what on which resources. CKAD tests creating Roles, ClusterRoles, RoleBindings and ClusterRoleBindings imperatively, and validating with kubectl auth can-i --as. Know namespace- vs cluster-scope.

You'll build

Create a Role + RoleBinding for a ServiceAccount, validate with can-i, then a ClusterRole bound two ways.

Key commands: k create · k auth · » Role

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 25 · COMMANDS

RBAC — Roles & RoleBindings

Step 1 — Role: namespace-scoped permissions

```
k create role pod-reader \
  --verb=get,list,watch --resource=pods -n dev
```

Step 2 — RoleBinding: link Role to a ServiceAccount

```
k create rolebinding viewer-binding \
  --role=pod-reader --serviceaccount=dev:viewer -n dev
```

Step 3 — Validate with kubectl auth can-i

```
k auth can-i list pods   -n dev     --as=system:serviceaccount:dev:viewer  # yes
k auth can-i delete pods -n dev     --as=system:serviceaccount:dev:viewer  # no
k auth can-i list pods   -n default --as=system:serviceaccount:dev:viewer  # no
```


© 2026 Tertiary Infotech Academy Pte Ltd

RBAC — Roles & RoleBindings (cont.)

Step 4 — ClusterRole bound cluster-wide vs to one namespace

```
k create clusterrole node-reader --verb=get,list --resource=nodes
k create clusterrolebinding nodes-binding \
  --clusterrole=node-reader --serviceaccount=dev:nodes-sa   # all namespaces
k create rolebinding dev-node-reader \
  --clusterrole=node-reader --serviceaccount=dev:viewer -n dev  # only dev
```

Note Role + RoleBinding = namespace-scoped. ClusterRole + ClusterRoleBinding = cluster-wide. ClusterRole + RoleBinding = the cluster role's verbs limited to one namespace. kubectl auth can-i <verb> <res> --as=<identity> validates without logging in.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 25 · TEST IT

RBAC — Roles & RoleBindings

✅ Test it

k auth can-i list pods -n dev --as=system:serviceaccount:dev:viewer returns yes while delete pods and cross-namespace list return no.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

CRD YAML Manifest

Defines the schema for the custom SmokeTest object

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: smoketests.stable.bmuschko.com
spec:
  group: stable.bmuschko.com
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties:
              service: {type: string}
              path:    {type: string}
              timeout: {type: integer}
              retries: {type: integer}
  scope: Namespaced
  names:
    plural: smoketests
    singular: smoketest
    kind: SmokeTest
    shortNames: [st]
```


© 2026 Tertiary Infotech Academy Pte Ltd

Creating the CRD and Custom Object

Create the CRD first, then create instances

```
kubectl apply -f smoketest-crd.yaml
customresourcedefinition.apiextensions.k8s.io/smoketests.stable.bmuschko.com created

kubectl apply -f backend-smoke-test.yaml
smoketest.stable.bmuschko.com/backend-smoke-test created
```


© 2026 Tertiary Infotech Academy Pte Ltd

Discovering CRDs

List all installed CRDs

```
kubectl get crds
NAME                                  CREATED AT
smoketests.stable.bmuschko.com        2023-05-04T14:49:40Z
```

Discover custom resource types via api-resources

```
kubectl api-resources --api-group=stable.bmuschko.com
NAME         SHORTNAMES  APIVERSION                 NAMESPACED  KIND
smoketests   st          stable.bmuschko.com/v1     true        SmokeTest
```


© 2026 Tertiary Infotech Academy Pte Ltd

Controller Implementation

- You can CRUD custom resource objects, but no smoke-test logic is initiated without a controller.
- A controller acts as a reconciliation process: it polls for new SmokeTest objects via the API server and executes the smoke test, sending results to the external service.
- Controllers are typically written in Go or Python using a Kubernetes client library (e.g. client-go, kubernetes-client).

© 2026 Tertiary Infotech Academy Pte Ltd

EXERCISE

Defining and Interacting with a CRD

© 2026 Tertiary Infotech Academy Pte Ltd

Exam Essentials

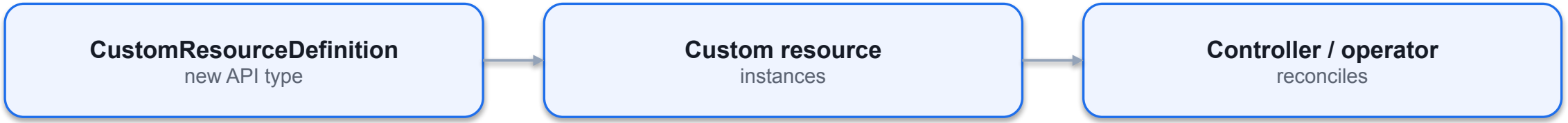
- You are not expected to implement a CRD schema yourself. Know how to discover and use them.
- Use kubectl get crds to list installed CRDs, and kubectl api-resources --api-group=<group> to see their types.
- Create objects from a CRD schema exactly as you would for built-in resources: kubectl apply -f <manifest>.yaml.
- Controller implementations are out-of-scope for the CKAD exam.

© 2026 Tertiary Infotech Academy Pte Ltd

CRDS

Custom Resource Definitions (CRDs)

Extending the Kubernetes API by custom primitives



CRDS

What is a Custom Resource Definition (CRD)?

Extension point for introducing custom API primitives

- ● Some use cases with custom requirements cannot be fulfilled with the built-in
- Kubernetes primitives and functionality.
- ● A CRD allows you to define, create, and persist objects for one or many custom
- primitives and expose them with the Kubernetes API.
- ● CRDs are combined with controllers to make them actually useful. Controllers
- interact with the API server and implement the custom logic. The combination of
- CRDs and controllers is called the Operator pattern.

SECTION

The Operator Pattern Consists of CRDs and Controllers

Example CRD

Smoke testing against a Service after deployment

- ● Requirement: An web-based application stack is deployed via a Deployment with
- one or more than one replica. The Service object routes network traffic to the Pods.
- ● Desired functionality: After the deployment happened, we want to run a quick
- smoke test against the Service’s DNS name to ensure that application is
- operational. The result of the smoke test will be sent to an external service for the
- purpose of rendering it on the UI dashboard.
- ● Goal: Implement the Operator pattern (CRD and controller) for the use case.

CRD YAML Manifest

Defines the schema for the custom object

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
 name: smoketests.stable.bmuschko.com   Combination of <plural>.<group>
spec:
 group: stable.bmuschko.com
 versions:
 - name: v1
 served: true
 Versions supported by CRD
 storage: true
 schema:
 openAPIV3Schema:
 type: object
```

CRDS

Custom Resource Object YAML Manifest

Instantiation of CRD kind

```
apiVersion:   Group and Version of custom kind
stable.bmuschko.com/v
1 kind: SmokeTest
metadata:   Kind defined by CRD
 name:
 backend-smoke-test   Attributes and values that make
spec:   custom kind configurable
 service:
 backend path:
 /health
 timeout: 600
 retries: 3
```

Creating the CRD and Custom Object

CRD object needs to be created before custom objects

```
$ kubectl apply -f loadtest-resource.yaml
customresourcedefinition.apiextensions.k8s.io/smoketests.stable.bmuschko.com
created
$ kubectl apply -f loadtest.yaml
smoketest.stable.bmuschko.com/backend-smoke-test created
```

Discovering CRDs

Once installed, you can list all available CRDs

```
$ kubectl api-resources --api-group=stable.bmuschko.com
NAME SHORTNAMES   APIVERSION   NAMESPACED   KIND
smoketests   st  stable.bmuschko.com/v1 true   SmokeTest
$ kubectl get crds
NAME   CREATED AT
smoketests.stable.bmuschko.com   2023-05-04T14:49:40Z
```

CRDS

Controller Implementation

The actual logic for smoke tests lives in the controller

- ● You can interact with the custom resource object using CRUD
- (create/read/update/delete) operations; however, no smoke test logic will actually be
- initiated.
- ● A controller acts as a reconciliation process by inspecting the state through the
- API server. At runtime, it polls for new SmokeTest objects and sends the result to
- the external service.
- ● Controllers can use one of the Kubernetes client libraries, written in Go or Python, to
- access custom resources.

Govern Resource Consumption

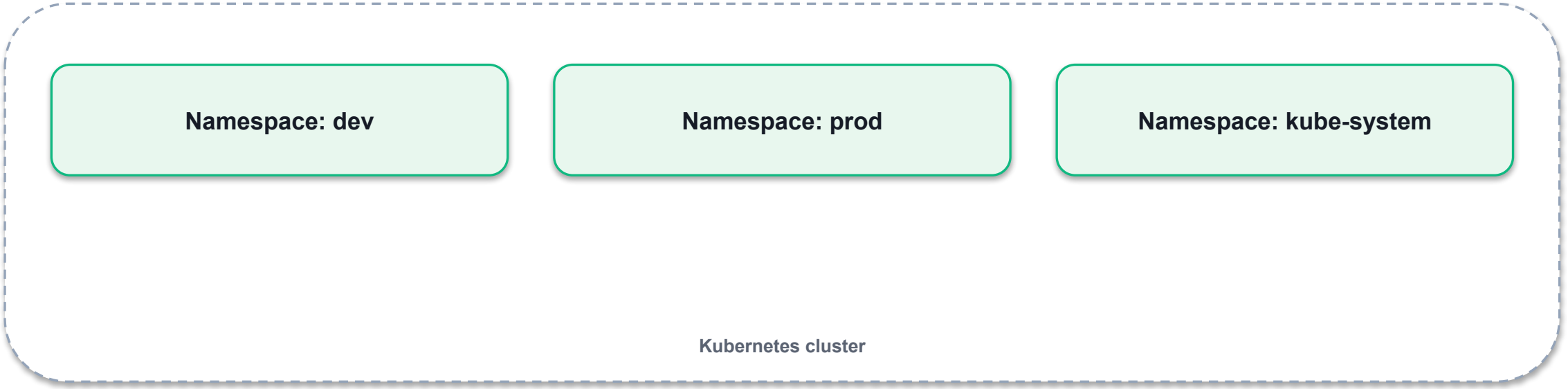
- ResourceQuota caps the aggregate resources a namespace can consume (pods, requests, limits).
- LimitRange enforces per-container defaults, defaultRequests and maximums.
- Without a LimitRange, Pods with no resource block are rejected in a quota-enforced namespace.
- kubectl describe quota shows Used vs Hard — the primary troubleshooting view.

© 2026 Tertiary Infotech Academy Pte Ltd

RESOURCE MANAGEMENT

Pods and Namespaces

Running and inspecting workload, organizing objects



RESOURCE MANAGEMENT

Managing Resources for Objects and Namespaces

It’s best practice to make a statement about resource consumption

- ● Containers can define a minimum amount of resources needed to run the
- application, as well as the maximum amount of resources the application is allowed
- to consume.
- ● Application developers should determine the right-sizing with load tests or at runtime
- by monitoring the resource consumption.
- ● Enforcement of resources can be constrained on a namespace-level with a
- ResourceQuota.
- ● A LimitRange is a policy that constrains or defaults the resource allocations for a
- single object (e.g., for a Pod or PersistentVolumeClaim).

RESOURCE MANAGEMENT

Resource Units in Kubernetes

CPU units and memory as fixed-point number or power-of-two equivalents

- ● Kubernetes measures CPU resources in millicores and memory resources in bytes.
- That’s why you might see resources defined as 600m or 100Mib.
- ● For a deep dive on those resource units, it’s worth cross-referencing the section
- “Resource units in Kubernetes” in the official documentation.

RESOURCE MANAGEMENT

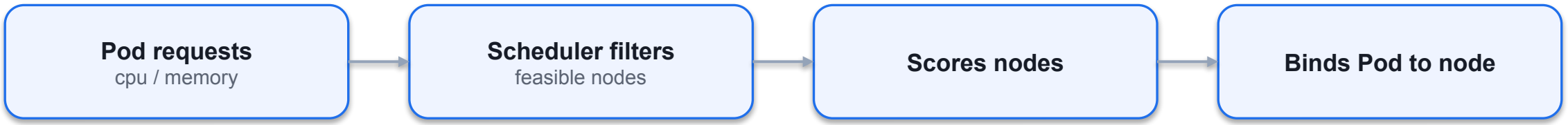
Container Resource Requests

Definition and runtime effects

- ● Containers can define a minimum amount of resources needed to run the
- application via spec.containers[].resources.requests.
- ● Resource types include CPU, memory, huge page, ephemeral storage.
- ● If Pod cannot be scheduled on any node due to insufficient resource availability, you
- may see a PodExceedsFreeCPU or PodExceedsFreeMemory status.

Example Scheduling Scenario

Node’s resource capacity needs to be able to fulfill it



Container Resource Requests Options

Configuration options and example values

- YAML Attribute Description Example Value
- spec.containers[].resources.request CPU resource type 500m (five hundred millicpu)
- s.cpu
- spec.containers[].resources.request Memory resource type 64Mi (2^26 bytes)
- s.memory
- spec.containers[].resources.request Huge page resource type hugepages-2Mi: 60Mi
- s.hugepages-<size>
- spec.containers[].resources.request Ephemeral storage resource 4Gi
- s.ephemeral-storage type

RESOURCE MANAGEMENT

Container Resource Requests YAML Manifest

Minimum amount of resources defined for a container

```
apiVersion: v1
kind: Pod
metadata:
 name: rate-limiter
spec:
 containers:
 - name: business-app
 image: bmuschko/nodejs-business-app:1.0.0
 ports:
 resources:
 - containerPort: 8080
 requests: memory:
 "256Mi" cpu:
 "1"
```

RESOURCE MANAGEMENT

Container Resource Limits

Definition and runtime effects

- ● Containers can define a maximum amount of resources allowed to consumed by the
- application via spec.containers[].resources.limits.
- ● Resource types include CPU, memory, huge page, ephemeral storage.
- ● Container runtime decides how to handle situation where application exceeds
- allocated capacity, e.g. termination of application process.

Container Resource Requests Options

Configuration options and example values

- YAML Attribute Description Example Value
- spec.containers[].resources.request CPU resource type 500m (five hundred millicpu)
- s.cpu
- spec.containers[].resources.request Memory resource type 64Mi (2^26 bytes)
- s.memory
- spec.containers[].resources.request Huge page resource type hugepages-2Mi: 60Mi
- s.hugepages-<size>
- spec.containers[].resources.request Ephemeral storage resource 4Gi
- s.ephemeral-storage type

Container Resource Limits YAML Manifest

Do not allow more than the allotted resource amounts

```
apiVersion: v1
kind: Pod
metadata:
 name: rate-limiter
spec:
 containers:
 - name: business-app
 image: bmuschko/nodejs-business-app:1.0.0
 ports:
 resources:
 - containerPort: 8080
 limits:
 memory: "512Mi"
 cpu: "2"
```

Pod Scheduling Details

After scheduling a Pod, you can check on runtime information

```
$ kubectl get pod rate-limiter -o yaml | grep nodeName:
  nodeName: worker-3
$ kubectl describe node worker-3
...
Non-terminated Pods:   (3 in total)
  Namespace Name CPU Requests   CPU Limits   ...
  --------- ---- ------------   ----------   ...
  default   rate-limiter   1250m (62%)   0 (0%)   ...
```

RESOURCE MANAGEMENT

Container Resource Requests/Limits YAML Manifest

It is considered best practice to define requests and limits for every container

```
spec:
 containers:
 - name: business-app
 ...
 resources:
 requests:
 memory: "256Mi"
 cpu: "1"
 limits:
 memory: "512Mi"
 cpu: "2"
```

RESOURCE MANAGEMENT

What is a ResourceQuota?

Defines resource constraints per namespace

- ● Constraints that limit aggregate resource consumption or limit the quantity of objects
- (e.g., number of Pods or Secrets) that can be created.
- ● requests.cpu/requests.memory: Across all pods in a non-terminal state, the
- sum of CPU/memory requests cannot exceed this value.
- ● limits.cpu/limits.memory: Across all pods in a non-terminal state, the sum of
- CPU/memory limits cannot exceed this value.

ResourceQuota YAML Manifest

Limits # of objects and aggregate container resource consumption

```
apiVersion: v1
kind:
ResourceQuota
metadata:
 name: awesome-quota
 namespace: team-awesome
 hard:
spec:
 pods: 2
 requests.cpu: "1"
 requests.memory:
 1024Mi limits.cpu: "4"
 limits.memory: 4096Mi
```

No Requests/Limits Definition

Scheduler rejects the creation of the object

```
$ kubectl create -f nginx-pod.yaml -n team-awesome
Error from server (Forbidden): error when creating "nginx-pod.yaml":
 pods "nginx" is forbidden: failed quota: awesome-quota: must
specify  limits.cpu,limits.memory,requests.cpu,requests.memory
```

RESOURCE MANAGEMENT

Exceeds Requests/Limits Definition

Scheduler rejects the creation of the object

```
$ kubectl create -f nginx-pod.yaml -n team-awesome
Error from server (Forbidden): error when creating "nginx-pod.yaml":
 pods "nginx" is forbidden: exceeded quota: awesome-quota,
requested:  pods=1,requests.cpu=500m,requests.memory=512Mi, used:
pods=2,  requests.cpu=1,requests.memory=1024Mi, limited: pods=2,
requests.cpu=1,requests.memory=1024Mi
```

Inspecting a ResourceQuota

Renders consumed resources and defined hard limits

```
$ kubectl describe resourcequota awesome-quota -n team-awesome
Name:   awesome-quot
Namespace   a
  Used Hard
:   team-awesome
  ---- ----
Resource
  2   4
--------
  2048m
limits.cpu
  4096m
limits.memory
  2   2
```

RESOURCE MANAGEMENT

What is a LimitRange?

Constraints or defaults the resource allocations for an object type

- ● Enforces minimum and maximum compute resources usage per Pod or container in
- a namespace.
- ● Enforces minimum and maximum storage request per PersistentVolumeClaim in a
- namespace.
- ● Enforces a ratio between request and limit for a resource in a namespace.
- ● Set default requests/limits for compute resources in a namespace and automatically
- injects them into containers at runtime.

RESOURCE MANAGEMENT

LimitRange YAML Manifest

Sets defaults, minimum and maximum CPU allocation for containers

```
apiVersion: v1
kind: LimitRange
metadata:
  name:
cpu-resource-constraint
spec:
  limits:
  - default:
  cpu: 500m   Defines default limits
  defaultRequest:
  cpu: 500m   Defines default requests
  max:
  cpu: "1"
  min:
```

Automatically Uses Container Defaults for Pod

Container doesn’t provide any resource requests/limits

```
$ kubectl create -f pod.yaml
pod/rate-limiter created
$ kubectl describe pod rate-limiter
...
Containers:
  business-app
  : Limits:
  cpu:
  ...
  500m
  Requests:
  cpu:
  500m
...
```

RESOURCE MANAGEMENT

Rejected Pod Creation for Exceeded Limits

Requests a higher maximum for CPU resources than defined by LimitRange

```
$ kubectl create -f pod.yaml
Error from server (Forbidden): error when creating "pod.yaml": pods
 "rate-limiter" is forbidden: maximum cpu usage per Container is 1,
 but limit is 2
```

ResourceQuota & LimitRange

LAB 26 Quotas & Limits

ResourceQuota caps the total resources used across a namespace; LimitRange enforces per-container defaults and maximums. CKAD tests both — write the YAML, apply them, and understand the rejection error messages.

You'll build

Apply a ResourceQuota and a LimitRange, watch defaults get injected, then violate the max and the quota.

Key commands: spec: · k run · for i · » ResourceQuota

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 26 · COMMANDS

ResourceQuota & LimitRange

Step 1 — ResourceQuota: namespace-wide ceilings

```
spec:
  hard:
    pods: "5"
    requests.cpu: "1"
    requests.memory: 1Gi
    limits.cpu: "2"
    limits.memory: 2Gi
```

Step 2 — LimitRange: per-container defaults and maximums

```
spec:
  limits:
  - type: Container
    default:        {cpu: 200m, memory: 256Mi}
    defaultRequest: {cpu: 100m, memory: 128Mi}
    max:            {cpu: 500m, memory: 512Mi}
```


© 2026 Tertiary Infotech Academy Pte Ltd

ResourceQuota & LimitRange (cont.)

Step 3 — Defaults injected; max enforced

```
k run a --image=nginx:1.25 -n team-a
k get pod a -n team-a -o jsonpath='{.spec.containers[0].resources}'; echo
# a container requesting cpu:1 is rejected: 'maximum cpu per Container is 500m'
```

Step 4 — Exceed the quota

```
for i in 1 2 3 4; do k run quota-$i --image=nginx:1.25 -n team-a; done
k run quota-5 --image=nginx:1.25 -n team-a   # 'exceeded quota: team-a-quota'
k describe quota team-a-quota -n team-a      # Used vs Hard
```

Note ResourceQuota limits aggregate usage; exceeding it rejects new Pods. LimitRange enforces per-container min/max/defaults. Without a LimitRange, Pods with no resource block cannot be created in a quota-enforced namespace.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 26 · TEST IT

ResourceQuota & LimitRange

✅ Test it

A Pod with no resource block gets requests/limits injected by the LimitRange, the 6th Pod is rejected with exceeded quota, and describe quota shows Used vs Hard.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

Managing Resources for Objects and Namespaces

- Containers should declare a minimum amount of resources needed (requests) and a maximum allowed (limits).
- Application developers should right-size resources with load tests or by monitoring production consumption.
- ResourceQuota enforces resource consumption at the namespace level.
- LimitRange constrains or defaults resource allocations for individual objects (Pod, PVC).

© 2026 Tertiary Infotech Academy Pte Ltd

Resource Units in Kubernetes

- CPU is measured in millicores (m). Example: 600m = 0.6 CPU cores.
- Memory is measured in bytes using fixed-point numbers or power-of-two suffixes:
  - Ki = kibibytes • Mi = mebibytes • Gi = gibibytes
  - K = kilobytes • M = megabytes • G = gigabytes
- Example: 256Mi = 268,435,456 bytes. See the official docs for full details.

© 2026 Tertiary Infotech Academy Pte Ltd

Container Resource Requests

- Containers declare a minimum amount of resources via spec.containers[].resources.requests.
- Resource types: CPU, memory, huge pages, ephemeral storage.
- The scheduler uses requests to decide which node can run the Pod.
- If no node has enough capacity → the Pod stays Pending with status PodExceedsFreeCPU or PodExceedsFreeMemory.

© 2026 Tertiary Infotech Academy Pte Ltd

Container Resource Requests YAML Manifest

Minimum resource amounts declared for a container

```
apiVersion: v1
kind: Pod
metadata:
  name: rate-limiter
spec:
  containers:
  - name: business-app
    image: bmuschko/nodejs-business-app:1.0.0
    ports:
    - containerPort: 8080
    resources:
      requests:
        memory: "256Mi"
        cpu: "1"
```


© 2026 Tertiary Infotech Academy Pte Ltd

Container Resource Limits

- Containers declare a maximum amount of resources via spec.containers[].resources.limits.
- Resource types: CPU, memory, huge pages, ephemeral storage.
- The container runtime enforces limits at runtime. Behaviour when exceeded:
  - CPU limit exceeded → process is throttled (not killed)
  - Memory limit exceeded → container may be OOMKilled (Out Of Memory)

© 2026 Tertiary Infotech Academy Pte Ltd

Container Resource Limits YAML Manifest

Maximum resource amounts declared for a container

```
apiVersion: v1
kind: Pod
metadata:
  name: rate-limiter
spec:
  containers:
  - name: business-app
    image: bmuschko/nodejs-business-app:1.0.0
    ports:
    - containerPort: 8080
    resources:
      limits:
        memory: "512Mi"
        cpu: "2"
```


© 2026 Tertiary Infotech Academy Pte Ltd

Container Resource Requests & Limits — Combined

It is best practice to define both requests and limits for every container

```
spec:
  containers:
  - name: business-app
    image: bmuschko/nodejs-business-app:1.0.0
    resources:
      requests:
        memory: "256Mi"
        cpu: "1"
      limits:
        memory: "512Mi"
        cpu: "2"
```


© 2026 Tertiary Infotech Academy Pte Ltd

EXERCISE

Defining Container Resource Requests and Limits

© 2026 Tertiary Infotech Academy Pte Ltd

Why Services?

- Pod IPs are ephemeral — a Service gives a stable virtual IP and DNS name in front of matching Pods.
- ClusterIP (default) is in-cluster only; NodePort opens a port on every node; LoadBalancer asks the cloud for an external IP.
- A Service finds its Pods by label selector; the matching set is tracked as EndpointSlices.
- Empty Endpoints almost always means a selector/label mismatch — the most common Service bug.

© 2026 Tertiary Infotech Academy Pte Ltd

THREE WAYS TO EXPOSE

Choosing a Service Type

ClusterIP

- In-cluster only
- Stable virtual IP + DNS
- Default type

NodePort

- Port on every node
- 30000–32767
- Reachable from outside

LoadBalancer

- Cloud external IP
- Builds on NodePort
- One Service, public entry

© 2026 Tertiary Infotech Academy Pte Ltd

Service Port Mapping — port, targetPort, nodePort

Three port fields: port (Service), targetPort (Pod container), nodePort (Node)

```
apiVersion: v1
kind: Service
metadata:
  name: web-svc
spec:
  type: NodePort
  selector:
    app: web
  ports:
  - port: 80          # port clients use to reach the Service (ClusterIP)
    targetPort: 8080   # port the container listens on
    nodePort: 30080    # port opened on every Node (30000-32767)
                       # omit nodePort for ClusterIP; not needed for LoadBalancer
```

Expose a deployment as a Service in one command

```
kubectl expose deployment web --port=80 --target-port=8080 --type=NodePort
kubectl get svc web
NAME   TYPE       CLUSTER-IP    PORT(S)        AGE
web    NodePort   10.96.1.50    80:30080/TCP   5s
```


© 2026 Tertiary Infotech Academy Pte Ltd

Service DNS — How Pods Resolve Services

<service-name>.<namespace>.svc.cluster.local

web-svc	.	production	.	svc	.	cluster.local	.
service name		namespace		resource type		cluster domain	

Short forms (within same namespace)

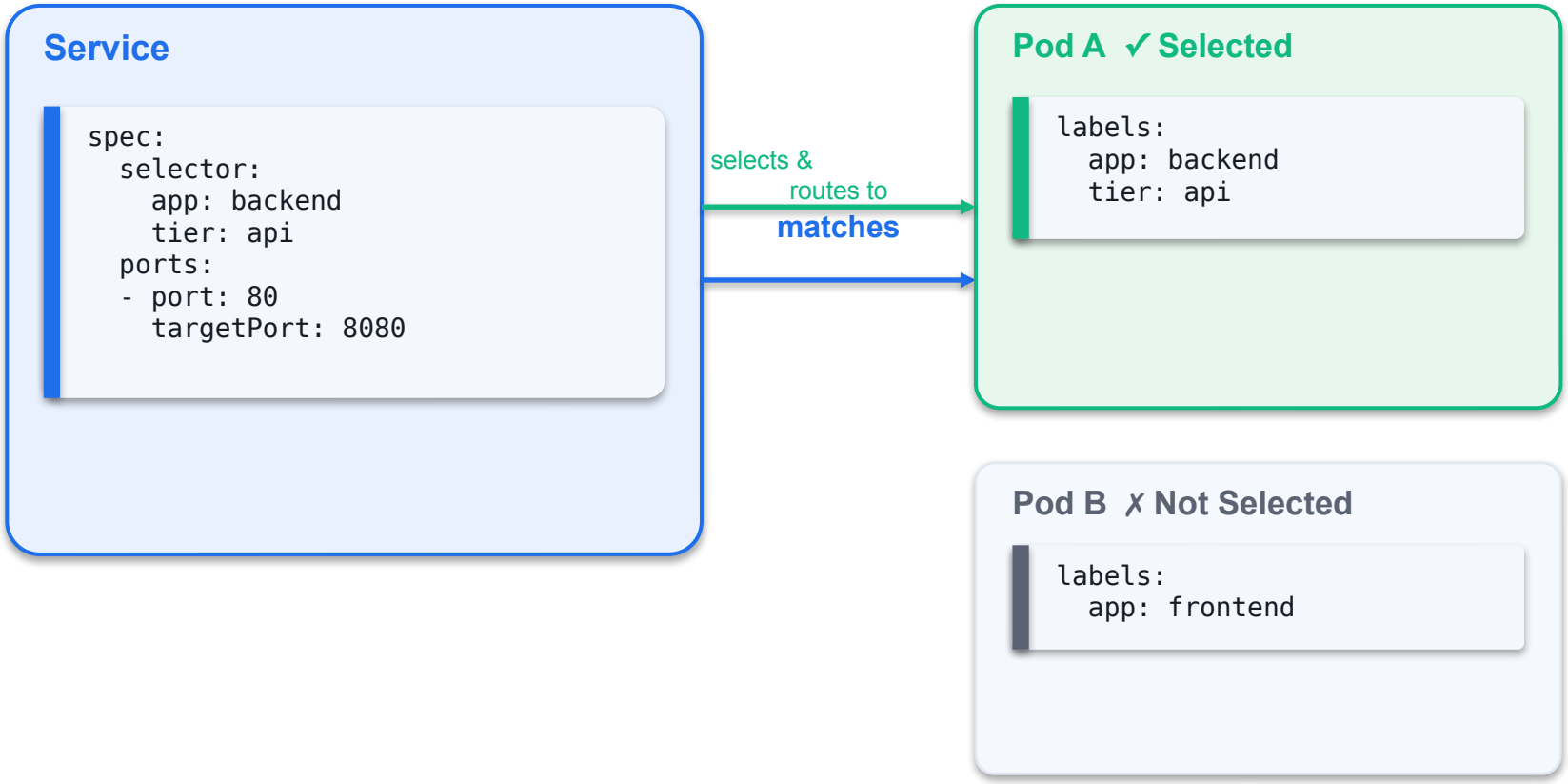
```
# From a pod in the SAME namespace — just use the service name:
curl http://web-svc/api

# From a pod in a DIFFERENT namespace — include the namespace:
curl http://web-svc.production/api

# Full FQDN always works:
curl http://web-svc.production.svc.cluster.local/api
```


© 2026 Tertiary Infotech Academy Pte Ltd

Service Selector Matches Pod Labels



kubectl get endpoints

Lists the IPs of all matching Pods.
Empty = selector mismatch —
first place to debug!

© 2026 Tertiary Infotech Academy Pte Ltd

Services — Quick Reference

Service Types

- ClusterIP (default): internal VIP, cluster-only
  - kubectl create service clusterip my-svc --tcp=80:8080
- NodePort: extends ClusterIP + opens port 30000-32767 on every node
  - kubectl create service nodeport my-svc --tcp=80:8080 --node-port=30080
- LoadBalancer: extends NodePort + provisions cloud LB
- ExternalName: DNS CNAME alias to external hostname

When to Use

- Use ClusterIP when only pods inside the cluster need access
- Use NodePort for simple external access without a cloud provider
- Use LoadBalancer in managed cloud environments (GKE, EKS, AKS)
- Headless service (clusterIP: None): no VIP — direct pod DNS records
  - kubectl create service clusterip my-svc --clusterip=None
- Check: kubectl get svc -A | kubectl describe svc <name>

© 2026 Tertiary Infotech Academy Pte Ltd

SECTION

Final Lab

SERVICES

What is a Service?

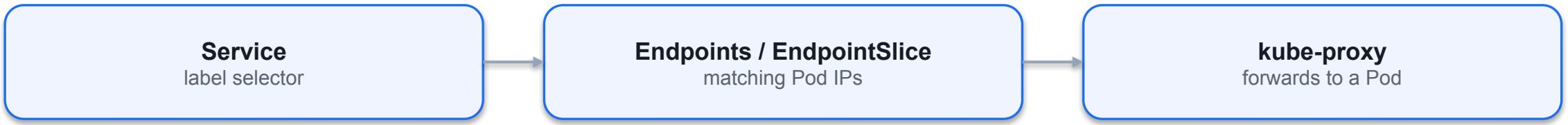
Stable endpoint for routing traffic to a set of Pods

- ● A Pod’s virtual IP address is not considered stable over time. A Pod restart leases a
- new IP address.
- ● Building a microservices architecture on Kubernetes requires network endpoints that
- are stable over time.
- ● A Service provides discoverable names and load balancing to a set of Pods.

SERVICES

Request Routing

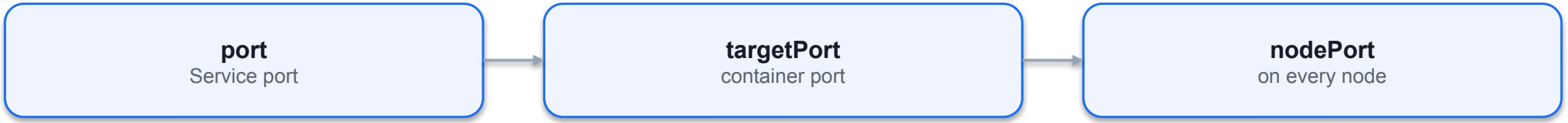
"How does a Service decide which Pod to forward the request to?"



SERVICES

Port Mapping

"How to map the Service port to the container port in a Pod?"



Creating a Service with Imperative Approach

Needs to provide the service type and ports

```
$ kubectl run echoserver
 --image=k8s.gcr.io/echoserver:1.10
 --port=8080
 Exposed container port
pod/echoserver
created
$ kubectl create
 service
 clusterip
 --tcp=80:8080   Incoming and outgoing port
 echoserver
service/echoserver created
$ kubectl run echoserver
 --image=k8s.gcr.io/echoserver:1.10
```

Service YAML Manifest

Needs to provide the service type and ports

```
apiVersion: v1
kind: Service
metadata:
 name: echoserver
spec:
 selector:
 app: echoserver   Label selection for Pods
 ports:
 - port: 80
 targetPort: 8080   Mapping of incoming to outgoing port
 protocol: TCP
```

SERVICES

Service Types

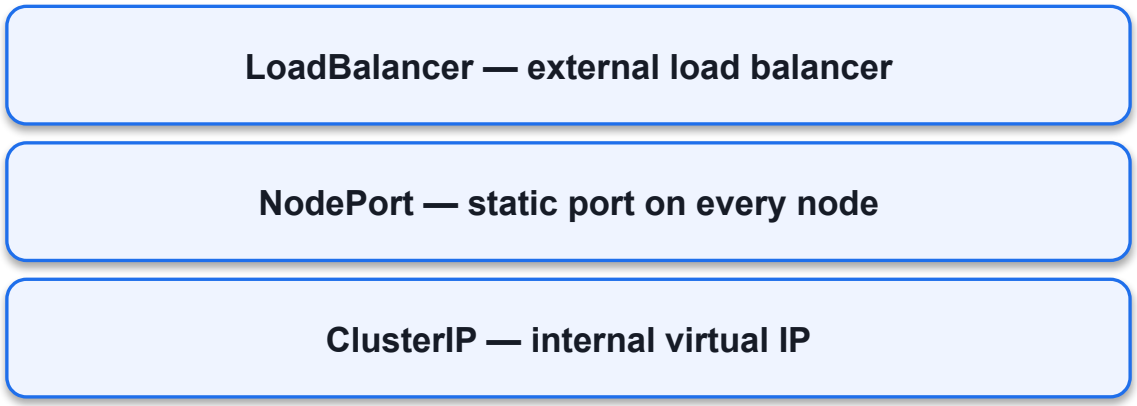
The following table shows a limited list of Service types

- Type Description
- ClusterIP Exposes the service on a cluster-internal IP.
- Only reachable from Pods within the cluster.
- Default if value for spec.type has not been
- assigned in manifest.
- Nodeport Exposes the service on each node's IP
- at a static port. Accessible from outside
- of the cluster.
- LoadBalancer Exposes the service externally using a cloud
- provider’s load balancer.

SERVICES

Service Type Inheritance

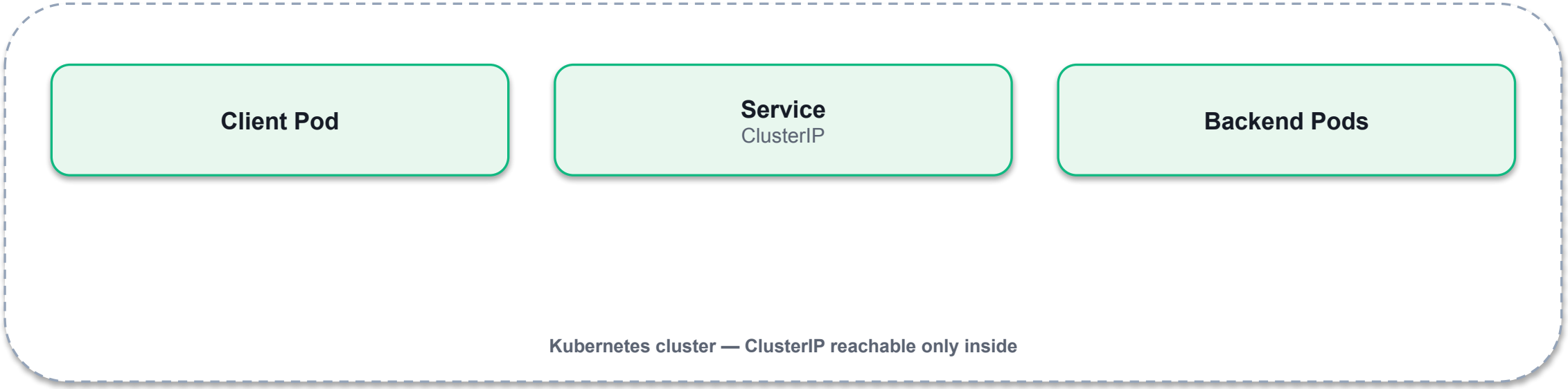
Services build on top of each other



SERVICES

ClusterIP Service Type

Only reachable from within the cluster, e.g. another Pod



SECTION

ClusterIP Service

SERVICES

ClusterIP Service YAML Manifest at Runtime

ClusterIP service type has been assigned if not already provided

spec.type: ClusterIP

spec.clusterIP assigned

spec.ports[].port / targetPort

selector matches Pod labels

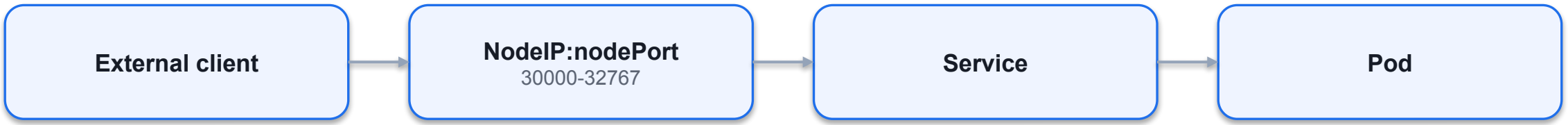
SECTION

ClusterIP - Binding Pods to Services

SERVICES

NodePort Service Type

Can be resolved from outside of the Kubernetes cluster



SERVICES

NodePort Service YAML Manifest at Runtime

Service type is NodePort, node port has been assigned

spec.type: NodePort

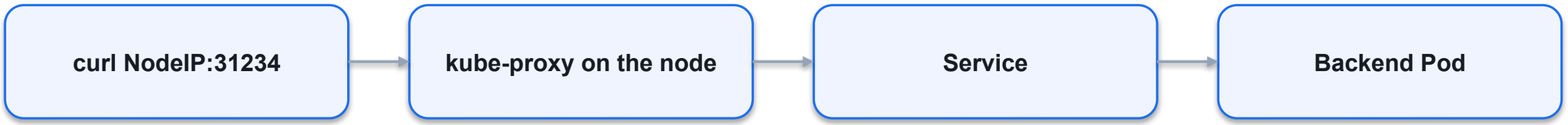
spec.ports[].nodePort: 31234

ClusterIP still assigned

SERVICES

Accessing a NodePort Service

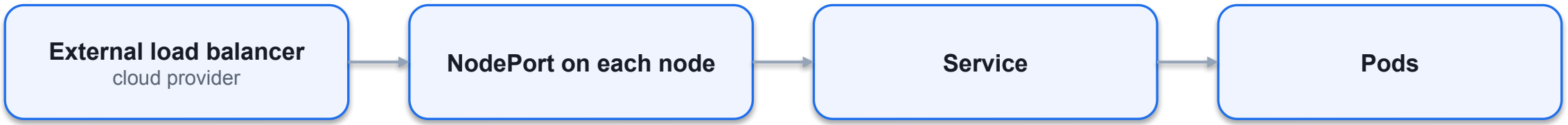
Use the combination of node IP and statically-assigned port



SERVICES

LoadBalancer Service Type

Routes traffic from existing, external load balancer to Service



SERVICES

Troubleshooting Services

Built-in mechanisms for identifying the root cause of a failure

kubectl get svc
type & ports

kubectl get endpoints
any Pod IPs?

kubectl describe svc
selector & events

kubectl exec + curl
from inside

SERVICES

Checking Service Connectivity

Identify Service Type and try to connect to it

```
$ kubectl get service echoserver
NAME   TYPE   CLUSTER-IP   EXTERNAL-IP   PORT(S) AGE
echoserve   ClusterI   10.96.254.0 <none>   8080/TCP 8s
r   P
$ kubectl run tmp --image=busybox --restart=Never -it --rm
-- wget echoserver:8080
Connecting to echoserver (10.96.254.0:8080)
wget: can't connect to remote host (10.96.254.0:8080): Connection
refuse
d
pod "tmp" deleted
pod tmp terminated
(Error)
Connection attempt failed
```

Verifying Endpoints

Value needs to expose targeted IP address and ports of Pods

```
$ kubectl get endpoints echoserver
NAME   ENDPOINTS   AGE
 Indicates issues with
echoserver   <none>   63s
 connection to Pods
$ kubectl get endpoints echoserver
NAME ENDPOINTS   AGE
echoserver   10.244.0.3:8080,10.244.0.4:808   63s
 Proper configuration of
 Service provides virtual IP
 address and ports to Pods
```

SERVICES

Checking Label Selection and Port Assignment

Value needs to expose targeted IP address and ports of Pods

```
$ kubectl describe service echoserver
Name:   echoserver
Namespace:   default
Labels:   svc=myservice
Selector:   app=echoserver   $ kubectl describe pod echoserver
Type:   ClusterIP   ...
IP Family Policy:   Labels:   app=echoserver
 SingleStack   Containers:
IP Families:   IPv4   echoserver:
IP:   Port:   8080/TCP
 10.111.148.223
IPs:   10.111.148.223
Port:   8080/TCP
TargetPort:   10.244.0.3:8080,10.244.0.4:8080
```

Checking Network Policies

Does any network policy block ingress traffic to Pod?

```
$ kubectl get networkpolicies
No resources found in default namespace.
$ kubectl get networkpolicies
NAME POD-SELECTOR AGE <none>
default-deny-ingress   9s
 Potentially blocks
 ingress traffic to all
 Pods in namespace
```

Ensuring Pod Functionality

The Pod or container may have a runtime issue

```
$ kubectl get pods
NAME READY   STATUS   RESTARTS AGE
echoserver   0/1   ErrImagePul   0   2s
 l
$ kubectl get pods
NAME READY   STATUS   RESTARTS AGE
 Runnin   0   5s   Failed to pull
echoserver   1/1
 container image
 g
 Looks successful on
 the surface-level
```

Services — ClusterIP, NodePort, LoadBalancer

LAB 27 Services

A Service gives a stable virtual IP and DNS name to a set of Pods. CKAD requires fluency with ClusterIP, NodePort and LoadBalancer types, kubectl expose, endpoint debugging and the selector-mismatch pattern.

You'll build

Expose a Deployment as ClusterIP, NodePort and LoadBalancer, inspect EndpointSlices, then debug a selector mismatch.

Key commands: k create · k expose · k patch · » ClusterIP

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 27 · COMMANDS

Services — ClusterIP, NodePort, LoadBalancer

Step 1 — Backend Deployment + ClusterIP Service

```
k create deployment web --image=nginx:1.25 --replicas=3
k expose deployment web --port=80 --target-port=80 --name=web-cip
k describe svc web-cip | grep -E "IP:|Endpoints:"
```

Step 2 — NodePort (a port on every node)

```
k expose deployment web --port=80 --target-port=80 --type=NodePort --name=web-np
PORT=$(k get svc web-np -o jsonpath='{.spec.ports[0].nodePort}')
curl -s http://localhost:$PORT | head -3   # 30000-32767
```

Step 3 — LoadBalancer + EndpointSlices

```
k expose deployment web --port=80 --type=LoadBalancer --name=web-lb
k get endpointslices -l kubernetes.io/service-name=web-cip
```


© 2026 Tertiary Infotech Academy Pte Ltd

Services — ClusterIP, NodePort, LoadBalancer (cont.)

Step 4 — Debug a selector mismatch (the #1 Service bug)

```
k patch svc web-cip -p '{"spec":{"selector":{"app":"does-not-exist"}}}'
k get endpoints web-cip        # empty = no Pod matches
k patch svc web-cip -p '{"spec":{"selector":{"app":"web"}}}'
```

Note ClusterIP = in-cluster only; NodePort = a port on every node; LoadBalancer = a cloud LB. Empty Endpoints almost always means the selector does not match Pod labels. EndpointSlices (v1.21+) succeed Endpoints.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 27 · TEST IT

Services — ClusterIP, NodePort, LoadBalancer

✅ Test it

k describe svc web-cip lists three endpoint IPs, the NodePort serves the page on localhost:$PORT, and a bad selector empties the Endpoints until it is fixed.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

TROUBLESHOOTING

Troubleshooting Services

Built-in mechanisms for identifying the root cause of a failure

© 2026 Tertiary Infotech Academy Pte Ltd

Checking Service Connectivity

Identify the Service type and attempt to connect to it

```
kubectl get service echoserver
NAME         TYPE       CLUSTER-IP      EXTERNAL-IP  PORT(S)    AGE
echoserver   ClusterIP  10.96.254.0     <none>       8080/TCP   8s
```

Run a temporary Pod and attempt to reach the Service

```
kubectl run tmp --image=busybox --restart=Never -it --rm \
  -- wget echoserver:8080
Connecting to echoserver (10.96.254.0:8080)
wget: can't connect to remote host (10.96.254.0:8080): Connection refused
pod "tmp" deleted
```


© 2026 Tertiary Infotech Academy Pte Ltd

Verifying Endpoints

No endpoints = label selector does not match any Pod

```
kubectl get endpoints echoserver
NAME         ENDPOINTS  AGE
echoserver   <none>     63s
```

Correct configuration exposes Pod IP addresses

```
kubectl get endpoints echoserver
NAME         ENDPOINTS                          AGE
echoserver   10.244.0.3:8080,10.244.0.4:8080   63s
```


© 2026 Tertiary Infotech Academy Pte Ltd

Checking Label Selection and Port Assignment

Inspect the Service — verify Selector and TargetPort

```
kubectl describe service echoserver
Name:      echoserver
Selector:  app=echoserver
Type:      ClusterIP
IP:        10.111.148.223
Port:      8080/TCP
TargetPort: 8080/TCP
Endpoints: 10.244.0.3:8080,10.244.0.4:8080
```

Compare with Pod labels — they must match the Selector

```
kubectl describe pod echoserver
Labels:  app=echoserver
Containers:
  echoserver:
    Port: 8080/TCP
```


© 2026 Tertiary Infotech Academy Pte Ltd

Checking Network Policies

Check for NetworkPolicies that may block ingress traffic

```
kubectl get networkpolicies
No resources found in default namespace.

# If policies exist, inspect them:
kubectl get networkpolicies
NAME                   POD-SELECTOR  AGE
default-deny-ingress   <none>        9s
```

A default-deny-ingress policy blocks ALL ingress — may need an allow rule

© 2026 Tertiary Infotech Academy Pte Ltd

Ensuring Pod Functionality

Check Pod status — even 'Running' may hide container problems

```
kubectl get pods
NAME        READY  STATUS        RESTARTS  AGE
echoserver  0/1    ErrImagePull  0         2s

# After fix:
kubectl get pods
NAME        READY  STATUS   RESTARTS  AGE
echoserver  1/1    Running  0         5s
```

Exec into the Pod or use logs to verify the application is actually serving

```
kubectl logs echoserver
kubectl exec -it echoserver -- wget -qO- localhost:8080
```


© 2026 Tertiary Infotech Academy Pte Ltd

EXERCISE

Troubleshooting a Service

© 2026 Tertiary Infotech Academy Pte Ltd

Exam Essentials

- Service misconfiguration is a common failure. Any misconfiguration prevents traffic from reaching the target Pods.
- Common causes: incorrect label selector (Service selector must match Pod labels), wrong port or targetPort assignment.
- Use kubectl get endpoints <service> to verify the Service has resolved Pod IP:port pairs.
- Check for NetworkPolicies that may block ingress traffic to the Pod.
- Verify the Pod itself is healthy: check status, logs, and exec a test request from inside the cluster.

© 2026 Tertiary Infotech Academy Pte Ltd

CLUSTER DNS

Names, Not IPs

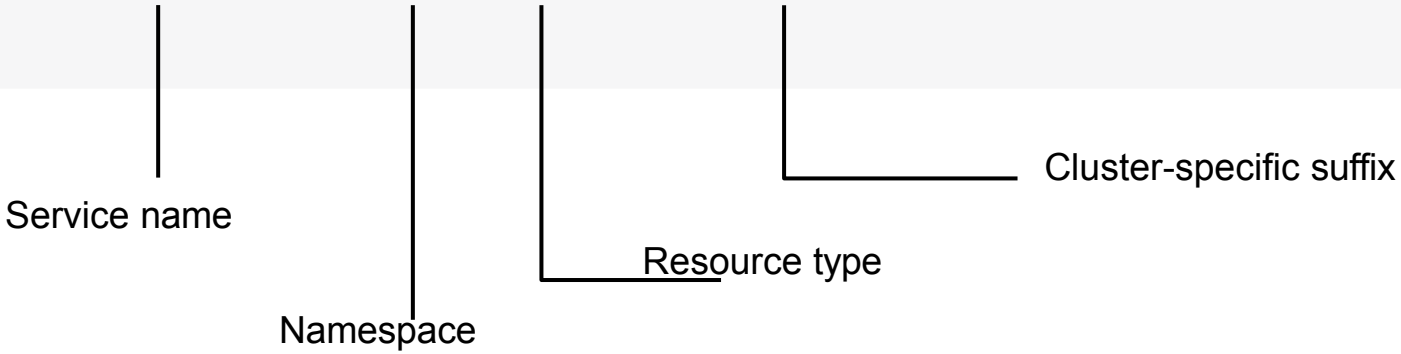
- CoreDNS gives every Service a name: <service>.<namespace>.svc.cluster.local.
- Within the same namespace the short name resolves via the Pod's search list.
- Cross-namespace you need at least <service>.<namespace>; the full FQDN always works.
- A headless Service (clusterIP: None) returns every Pod IP instead of a single virtual IP.

© 2026 Tertiary Infotech Academy Pte Ltd

Discovering the Service by DNS Lookup

Kubernetes' DNS service maps cluster IP address to Service name

```
$ kubectl run tmp --image=busybox --restart=Never -it --rm ↵
  -- wget
     echoserver.default.svc.cluster.local:8080
Connecting to echoserver.default.svc.cluster.local:8080 (10.96.254.0:8080)
...
```



Service DNS

LAB 28 Service DNS

CoreDNS gives every Service a stable DNS name: <service>.<namespace>.svc.cluster.local. CKAD tests cross-namespace resolution, Pod DNS records, headless Services, and reading /etc/resolv.conf to understand ndots:5.

You'll build

Resolve a Service by short name and FQDN, cross namespaces, read a Pod DNS record and /etc/resolv.conf, then a headless Service.

Key commands: k -n · nslookup web · cat /etc/resolv.conf · » FQDN

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 28 · COMMANDS

Service DNS

Step 1 — Resolve from the same namespace (short name)

```
k -n app run client --image=busybox --restart=Never -it --rm -- \
  sh -c 'nslookup web; wget -qO- web | head -3'
```

Step 2 — Resolve from a different namespace

```
nslookup web              # fails cross-namespace
nslookup web.app          # works
nslookup web.app.svc.cluster.local
```

Step 3 — Pod DNS record and resolv.conf

```
# Pod A-record: <dashed-ip>.<namespace>.pod.cluster.local
cat /etc/resolv.conf
# search app.svc.cluster.local svc.cluster.local cluster.local
# ndots:5
```


© 2026 Tertiary Infotech Academy Pte Ltd

Service DNS (cont.)

Step 4 — Headless Service returns all Pod IPs

```
k -n app create service clusterip headless --clusterip="None" --tcp=80:80
nslookup headless.app    # returns every Pod IP, not one VIP
```

Note FQDN = <service>.<namespace>.svc.cluster.local. Short names resolve via the search list within the namespace; cross-namespace needs <service>.<namespace>. ndots:5 is why short names check the search list before an absolute lookup. Headless (clusterIP: None) returns per-Pod IPs.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 28 · TEST IT

Service DNS

✅ Test it

nslookup web works in-namespace but needs web.app cross-namespace, and the headless Service resolves to all backing Pod IPs.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

SECTION

Recap

NETWORKING

What is an Ingress?

Route HTTP(S) traffic from the outside of the cluster to Service(s)

- ● It’s not a specific Service type, nor should it be confused with the Service type
- LoadBalancer.
- ● Defines rules for mapping a URL context path to one or many Service objects.
- ● TLS termination (support for HTTPS) needs to be configured explicitly. By default,
- only HTTP is supported.

NETWORKING

What is an Ingress Controller?

Evaluating Ingress rules

- ● An Ingress cannot work without an Ingress controller. The Ingress controller
- evaluates the collection of rules defined by an Ingress that determine traffic routing.
- ● Multiple Ingress controller can be deployed to a cluster. An Ingress can be
- configured to pick a specific one via spec.ingressClassName.

NETWORKING

Ingress Traffic Routing

The context path is mapped to a backend (Service name and port)

- /health

NETWORKING

Creating an Ingress with Imperative Approach

The rule definition requires intricate knowledge of syntax

```
$ kubectl create
 ingress corellian
 --rule="star-alliance.com/corellian/api=corellian:8080"
ingress.networking.k8s.io/corellian created
```

NETWORKING

Ingress YAML Manifest

Allows for defining multiple rules that map to backend

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: corellian
spec:
  -
  rules:
  host: star-alliance.com
  The host definition is
  http:   optional
  paths:
  - backend:
  service:
  name: corellian
```

NETWORKING

Ingress Rules

Traffic routing is controlled by rules defined on Ingress resource

- Type Example Description
- An optional host mycompany.abc.com If a host is provided, then the rules apply to that host.
- If no host is defined, then all inbound HTTP(S) traffic
- A list of paths /corellian/api
- is handled.
- The backend corellian:8080 Incoming traffic must match the host and path to
- correctly forward the traffic to a Service.
- A combination of Service name and port.

NETWORKING

Path Types

Incoming requests match based on assigned type in rule

- Path Type Rule Incoming Request
- Exact /corellian/api Matches /corellian/api but does not match
- /corellian/test or /corellian/api/.
- Prefix /corellian/api Matches /corellian/api and /corellian/api/
- but does not match /corellian/test.

NETWORKING

Listing and Describing Ingresses

Renders hosts, IP addresses, and ports

```
$ kubectl get
 ingress
NAME   CLASS   HOSTS   ADDRESS   PORTS   AGE
corellian   <none>   star-alliance.com   192.168.64.15   80   10m
$ kubectl describe ingress corellian
Name:   corellian
Namespace:   default
Address:   192.168.64.15
Default backend: default-http-backend:80 (<error:  Rules:   endpoints "default-http-backend" not
found>)
 Host   Path Backends
 ----   ---- --------
 star-alliance.com
 /corellian/api   corellian:8080 (172.17.0.5:8080)
Annotations:   <none>
```

NETWORKING

Configuring DNS for an Ingress

Making an Ingress convenient to use by end users

- ● To resolve the Ingress, you'll need to configure DNS entries to the external address.
- ● You'll need to either configure an A record, or a CNAME record. The ExternalDNS
- cluster add-on can help with managing those DNS records for you.
- ● If you have no domain or are using a local Kubernetes solution, you can add the
- mapping between IP address and domain name to /etc/hosts.

NETWORKING

Accessing an Ingress

Use host name, port, and context path

```
$ wget star-alliance.com/corellian/api --timeout=5 --tries=1
--2021-11-30 19:34:57-- http://star-alliance.com/corellian/api
Resolving star-alliance.com (star-alliance.com)... 192.168.64.15
Connecting to star-alliance.com (star-alliance.com)|192.168.64.15|:80...
 connected.
HTTP request sent, awaiting response... 200 OK
...
$ wget star-alliance.com/corellian/api/ --timeout=5 --tries=1
--2021-11-30 15:36:26-- http://star-alliance.com/corellian/api/
Resolving star-alliance.com (star-alliance.com)... 192.168.64.15
Connecting to star-alliance.com (star-alliance.com)|192.168.64.15|:80...
 connected.
HTTP request sent, awaiting response... 404 Not Found
2021-11-30 15:36:26 ERROR 404: Not Found.
```

Ingress with TLS

LAB 29

Ingress & TLS

An Ingress provides Layer-7 HTTP/HTTPS routing: hostname and path rules forwarding to backend Services. CKAD tests installing a controller, TLS Secrets, host routing and path-based routing with pathType.

You'll build

Install ingress-nginx, create a TLS Secret, write a host+TLS Ingress, test HTTPS, then add a path-based rule.

Key commands: k create · apiVersion: networking.k8s.io/v1 · curl -k · » Ingress

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 29 · COMMANDS

Ingress with TLS

Step 1 — Backend Service + TLS Secret

```
k create deployment web --image=hashicorp/http-echo --replicas=2 -- -text=hello-ingress
k expose deployment web --port=5678 --target-port=5678
k create secret tls demo-tls --cert=tls.crt --key=tls.key
```


© 2026 Tertiary Infotech Academy Pte Ltd

Ingress with TLS (cont.)

Step 2 — Ingress with TLS and host routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: {name: demo}
spec:
  ingressClassName: nginx
  tls:
  - hosts: [demo.local]
    secretName: demo-tls
  rules:
  - host: demo.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: {service: {name: web, port: {number: 5678}}}
```


© 2026 Tertiary Infotech Academy Pte Ltd

LAB 29 · COMMANDS

Ingress with TLS (cont.)

Step 3 — Test HTTPS routing

```
curl -k --resolve demo.local:$HTTPS_PORT:127.0.0.1 \
  https://demo.local:$HTTPS_PORT/        # hello-ingress
```

Step 4 — Add path-based routing

```
# add a second rule: path /v2 -> service v2:5678 (pathType: Prefix)
curl -k --resolve demo.local:$HTTPS_PORT:127.0.0.1 \
  https://demo.local:$HTTPS_PORT/v2       # hello-v2
```

Note Ingress apiVersion is networking.k8s.io/v1. ingressClassName selects the controller; tls.secretName must point at a kubernetes.io/tls Secret; pathType: Prefix matches a path and everything below it. --resolve + -k let curl test without real DNS or a valid cert.

© 2026 Tertiary Infotech Academy Pte Ltd

Ingress with TLS

✅ Test it

curl https://demo.local:$HTTPS_PORT/ returns hello-ingress over TLS, and the /v2 path returns hello-v2 after the second rule is added.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

SECTION

Network Policies Restricting Pod-to-Pod communication

Why Ingress? Services vs Ingress

Without Ingress (Services only)

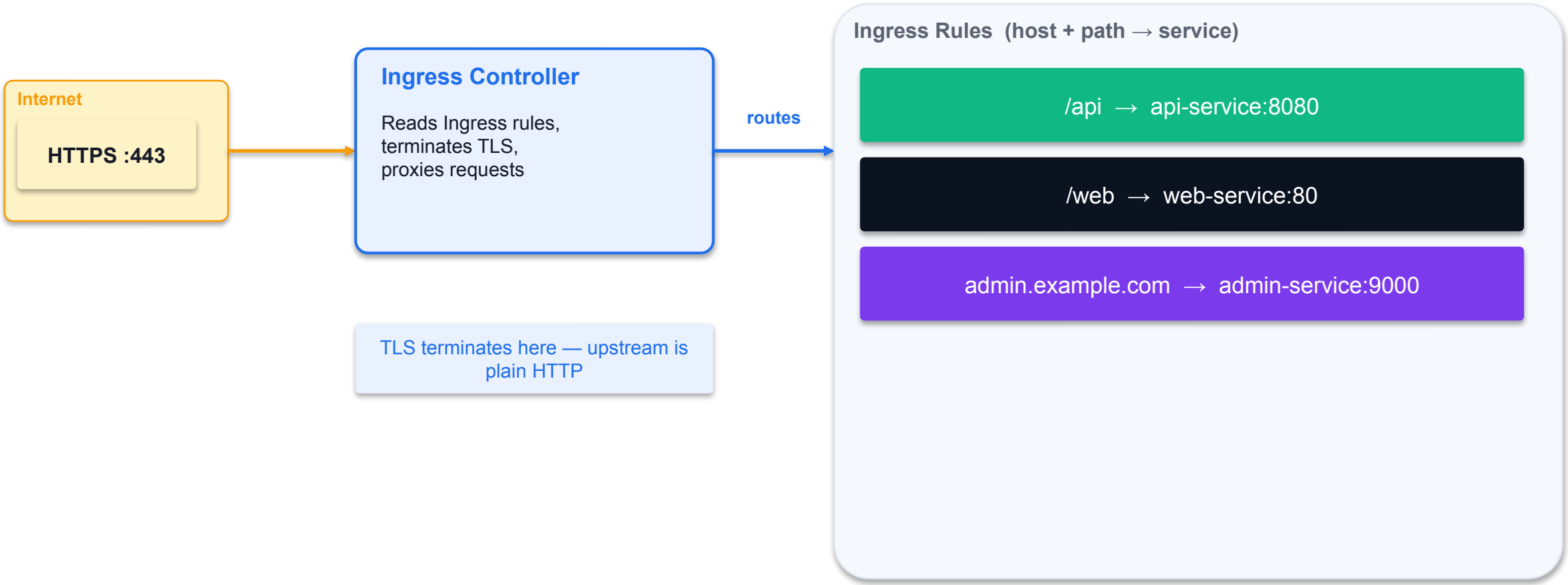
- One Service = one IP/port
- LoadBalancer Service = one cloud LB per service (expensive)
- No SSL/TLS termination at Service level
- No host/path-based routing with Services alone
- Many services = many public IPs to manage

With Ingress

- Single entry point for all HTTP(S) traffic
- L7 routing: route by hostname or URL path
  - example.com/api → api-service
  - example.com/web → web-service
- TLS termination: one cert handles all services
- Requires an IngressController (nginx, traefik, AWS ALB…)

© 2026 Tertiary Infotech Academy Pte Ltd

Ingress — L7 Routing Architecture



© 2026 Tertiary Infotech Academy Pte Ltd

Ingress YAML — Anatomy

Path-based routing for two services on the same host

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx          # which IngressController to use
  tls:
  - hosts: [example.com]
    secretName: tls-secret          # kubectl create secret tls
  rules:
  - host: example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service: {name: api-svc, port: {number: 8080}}
      - path: /web
        pathType: Prefix
        backend:
          service: {name: web-svc, port: {number: 80}}
```


© 2026 Tertiary Infotech Academy Pte Ltd

IngressClass and IngressController

- An IngressController is a running Pod (e.g. nginx, traefik) that reads Ingress rules and configures its proxy.
- An IngressClass links an Ingress resource to a specific IngressController.
  - spec.ingressClassName: nginx → handled by the nginx IngressController
- If only one IngressClass exists it can be marked as default (isDefaultClass: true).
- Check available classes: kubectl get ingressclass
- CKAD exam: IngressClass is usually pre-configured — just reference it in your Ingress spec.
- Troubleshoot: kubectl describe ingress <name> — check Events for routing errors.

© 2026 Tertiary Infotech Academy Pte Ltd

One Entry Point, Many Rules

- An Ingress defines host- and path-based HTTP/HTTPS rules that forward to backend Services.
- An Ingress controller (e.g. ingress-nginx) must be running; ingressClassName selects it.
- TLS terminates at the controller — tls.secretName points at a kubernetes.io/tls Secret.
- pathType: Prefix matches a path and everything below it; Exact requires an exact match.
- apiVersion is networking.k8s.io/v1 (extensions/v1beta1 was removed in 1.22).

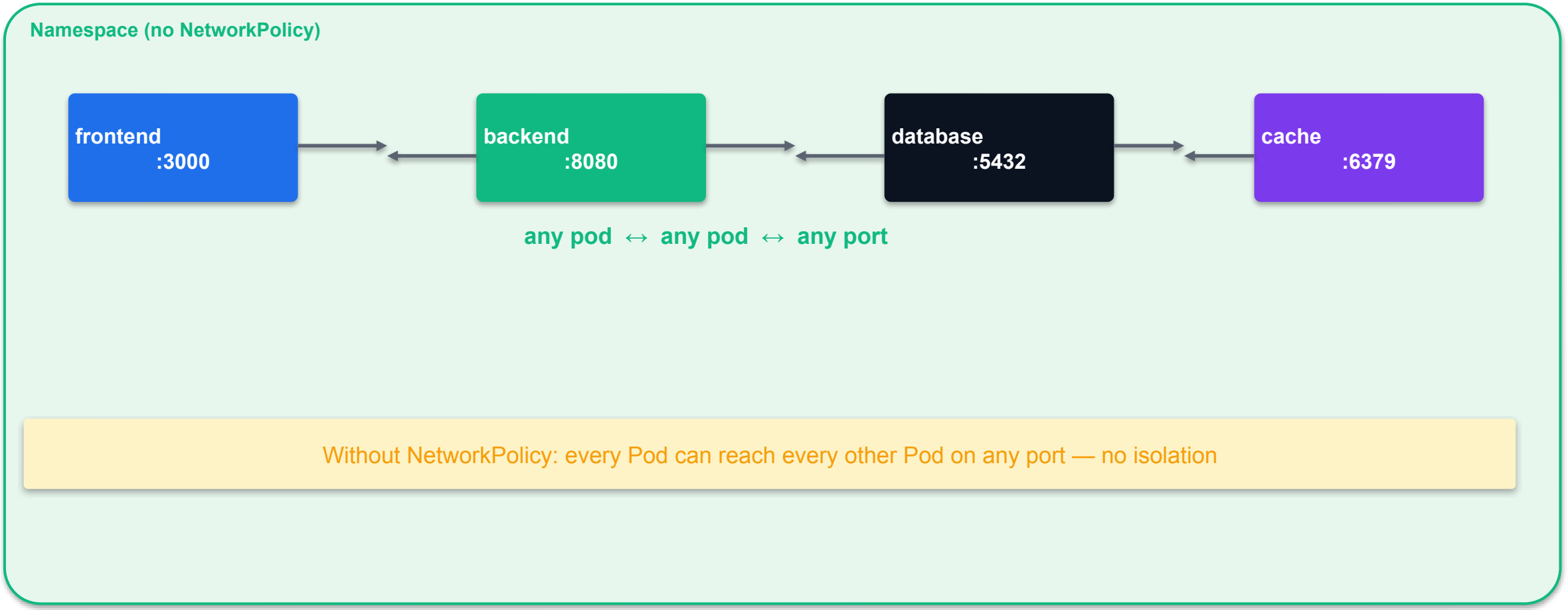
© 2026 Tertiary Infotech Academy Pte Ltd

Restrict the Default Open Network

- By default every Pod can talk to every other Pod — NetworkPolicies allow-list specific traffic.
- Default-deny: podSelector: {} with policyTypes:[Ingress] and no ingress rules blocks all inbound.
- Allow by podSelector (Pods by label) or namespaceSelector (whole namespaces) in from/to.
- Egress works the same way — the classic exam pattern is 'deny all egress except DNS (port 53)'.
- Policies are additive: multiple policies combine with logical OR on their allow rules.

© 2026 Tertiary Infotech Academy Pte Ltd

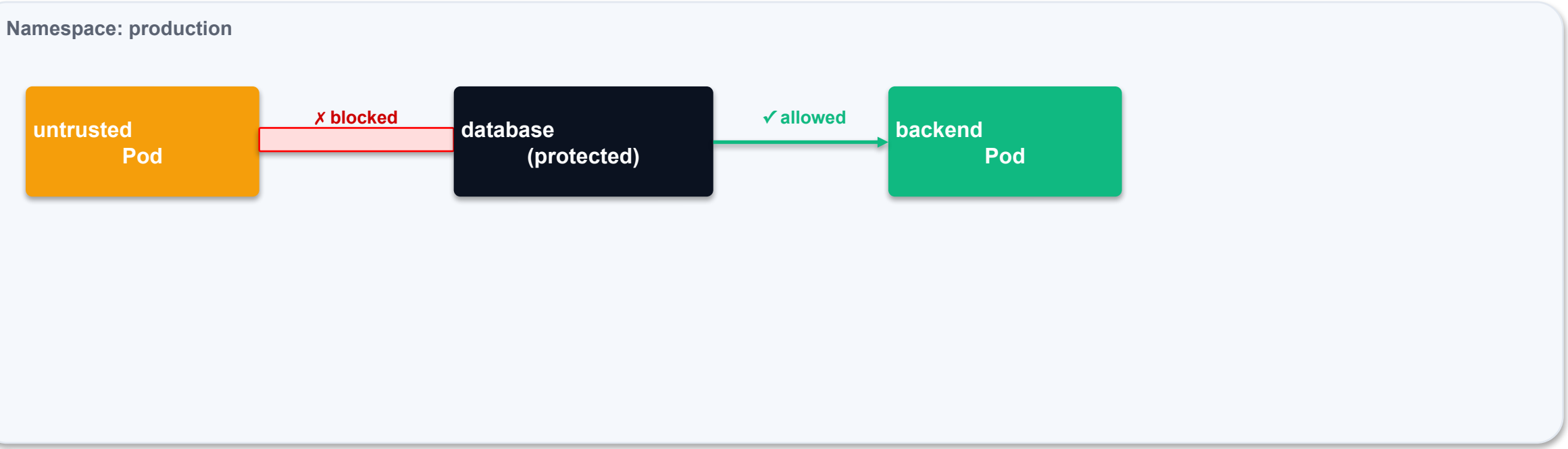
Default: All Pods Can Communicate



© 2026 Tertiary Infotech Academy Pte Ltd

NETWORKPOLICY · INGRESS RULES

NetworkPolicy — Restrict Ingress Traffic




```
kind: NetworkPolicy
spec:
  podSelector: {matchLabels: {app: database}}
  policyTypes: [Ingress]
  ingress:
  - from:
    - podSelector: {matchLabels: {role: backend}}
    ports:
    - port: 5432
```

NetworkPolicy — Restrict Egress Traffic

Allow backend to reach only the database (port 5432) — block everything else

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-egress
  namespace: production
spec:
  podSelector:
    matchLabels:
      role: backend
  policyTypes:
  - Egress
  egress:
  - to:
    - podSelector:
        matchLabels:
          app: database
    ports:
    - protocol: TCP
      port: 5432
  - ports:                    # allow DNS
    - protocol: UDP
      port: 53
```


© 2026 Tertiary Infotech Academy Pte Ltd

NetworkPolicy Patterns

- Default-deny ingress: select all pods, no ingress rules → blocks all incoming traffic.
  - kind: NetworkPolicy spec: podSelector: {} policyTypes: [Ingress] ingress: []
- Default-deny egress: select all pods, no egress rules → blocks all outgoing traffic.
  - kind: NetworkPolicy spec: podSelector: {} policyTypes: [Egress] egress: []
- Always allow DNS (port 53 UDP) in egress rules, or pods cannot resolve service names.
- namespaceSelector: target all pods in a specific namespace (e.g. monitoring).
- ipBlock: allow/deny traffic to/from external CIDRs (e.g. 192.168.0.0/16).

© 2026 Tertiary Infotech Academy Pte Ltd

NETWORKING

What is a Network Policy?

Firewall rules for Pod-to-Pod communication

- ● The installed Container Network Interface (CNI) plugin leases and assigns a new
- virtual IP address to every Pod when it is created.
- ● Communication between Pods within the same cluster is unrestricted. You can use a
- Pod’s IP address to communicate with it.
- ● Network policies implement the principle of least privilege. Only allow Pods to
- communicate if it is needed to fulfill the requirements of architecture.
- ● You can disallow and allow network communication with coarse-grained and
- fine-grained rules.

NETWORKING

What is an Network Policy Controller?

Evaluating Network Policy rules

- ● A Network Policy cannot work without a Network Policy controller. The Network
- Policy controller evaluates the collection of rules defined by a Network Policy.
- ● One option for such a Network Policy controller is Cilium.

NETWORKING

Network Policy YAML Manifest

Key-value pairs can be parsed from different sources

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: api-allow
spec:
 podSelector:   Selects the Pod the policy should
 matchLabels:   apply to by label selection
 app: payment-processor
 role: api
 Allows incoming traffic from the
 ingress:
 - from:   Pod with matching labels within
 - podSelector:   the same namespace
 matchLabels:
```

NETWORKING

Network Policy Rules

Attributes that form the rules

- Attribute Description
- podSelector Selects the Pods in the namespace to apply the network
- policy to.
- policyTypes Defines the type of traffic (i.e., ingress and/or egress) the
- network policy applies to.
- ingress Lists the rules for incoming traffic. Each rule can define from
- and ports sections.
- egress Lists the rules for outgoing traffic. Each rule can define to
- and ports sections.

NETWORKING

Behavior of to and from Selectors

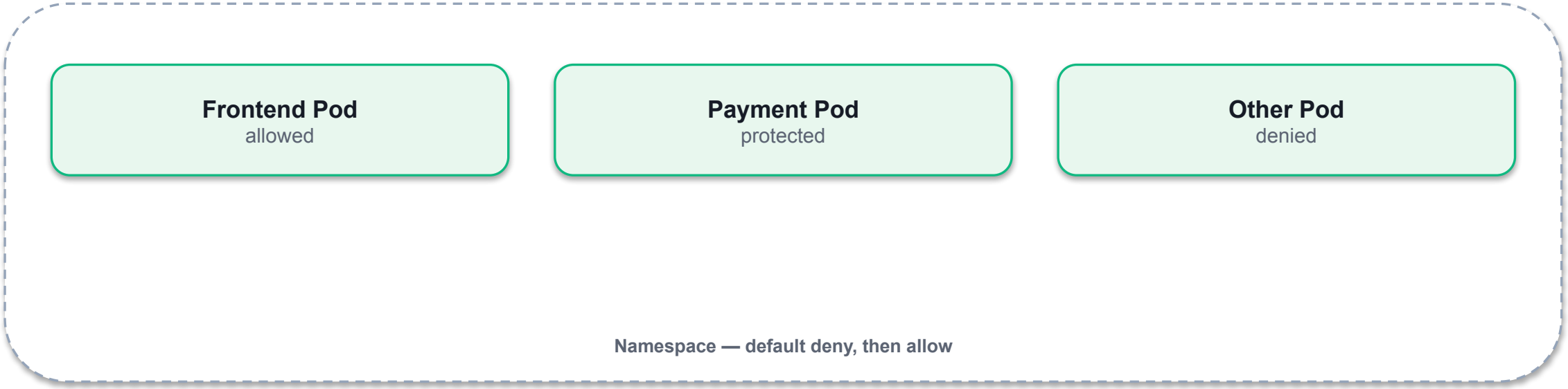
Ingress and egress sections define the same attributes

- Attribute Description
- podSelector Selects Pods by label(s) in the same namespace as the
- Network Policy which should be allowed as ingress sources or
- egress destinations.
- namespaceSelector Selects namespaces by label(s) for which all Pods should be
- allowed as ingress sources or egress destinations.
- namespaceSelector and Selects Pods by label(s) within namespaces by label(s).
- podSelector

NETWORKING

Example Network Policy

Restricting access to the payment processor Pod from other Pods



NETWORKING

Listing Network Policies

Doesn’t provide much details apart from Pod selector labels

```
$ kubectl get networkpolicy
NAME POD-SELECTOR AGE
api-allow app=payment-processor,role=api   83m
```

NETWORKING

Rendering Network Policy Details

Ingress and egress rules can be inspected in details

```
$ kubectl describe networkpolicy api-allow
Name:   api-allow
Namespace:   default
Created on:   2020-09-26 18:02:57 -0600 MDT
Labels:   <none>
Annotations: <none>
Spec:
 PodSelector:   app=payment-processor,role=api
 Allowing ingress traffic:
 To Port: <any> (traffic allowed to all ports)
 From:
 PodSelector: app=coffeeshop
 Not affecting egress traffic
 Policy Types: Ingress
```

NETWORKING

Verifying the Runtime Behavior

Communication from Pods not matching with labels will be blocked

```
$ kubectl run grocery-store --rm -it --image=busybox
 -l app=grocery-store,role=backend -- /bin/sh
/ # wget --spider --timeout=1 10.0.0.51
Connecting to 10.0.0.51 (10.0.0.51:80)
wget: download timed out
/ # exit
pod "grocery-store" deleted
$ kubectl run coffeeshop --rm -it --image=busybox
 -l app=coffeeshop,role=backend -- /bin/sh
/ # wget --spider --timeout=1 10.0.0.51
Connecting to 10.0.0.51 (10.0.0.51:80)
remote file exists
/ # exit
pod "coffeeshop" deleted
```

NETWORKING

Default Deny Network Policies

Network policies are additive

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: default-deny-all
spec:
 Selects all Pods in
 podSelector: {}
 policyTypes:   the namespace
 - Ingress
 - Egress
 Applies in incoming
 and outgoing traffic
```

NETWORKING

Restricting Access to Ports

By default, all ports are accessible

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
 name: port-allow
spec:
 podSelector:
 matchLabels:
 app: backend
 ingress:
 - from:
 - podSelector:
 ports:
 matchLabels:
 Only allow incoming
```

NETWORKING

Scenario-Based Learning Resource

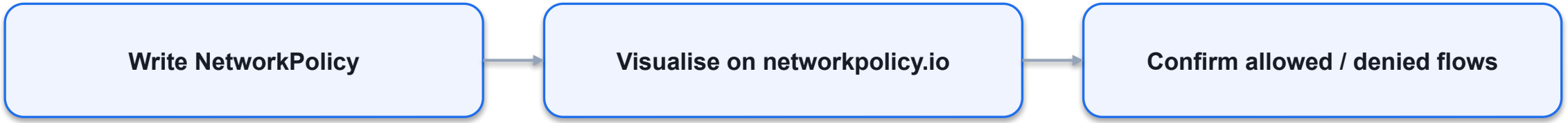
Exercising use cases for Network Policies is helpful with understanding concept

```
https://github.com/ahmetb/kubernetes-network-policy-recipes
```

NETWORKING

Visualizing and Analysing a Network Policy

The page networkpolicy.io provides policy editor



NetworkPolicy

LAB 30 NetworkPolicy

By default every Pod can talk to every other Pod. NetworkPolicies allow-list specific sources and destinations. CKAD regularly includes a NetworkPolicy question — you must write a default-deny and a selective-allow policy from scratch.

You'll build

Apply default-deny ingress, a selective podSelector allow, an egress DNS-only lockdown, then a cross-namespace allow.

Key commands: spec: · ingress: · » Default-deny

https://killercoda.com/playgrounds/course/kubernetes-playgrounds/two-node

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 30 · COMMANDS

NetworkPolicy (cont.)

Step 3 — Egress lockdown: allow DNS only

```
spec:
  podSelector: {matchLabels: {role: blocked}}
  policyTypes: [Egress]
  egress:
  - to:
    - namespaceSelector: {}
      podSelector: {matchLabels: {k8s-app: kube-dns}}
    ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
```

Step 4 — Cross-namespace allow (namespaceSelector)

```
ingress:
- from:
  - namespaceSelector:
      matchLabels: {purpose: trusted}
```


© 2026 Tertiary Infotech Academy Pte Ltd

NetworkPolicy (cont.)

Note Default-deny = podSelector: {} + policyTypes: [Ingress] with no ingress list. podSelector in from matches Pods by label in the same namespace; namespaceSelector matches whole namespaces. Policies are additive — multiple policies combine with logical OR.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 30 · TEST IT

NetworkPolicy

✅ Test it

After the allow policy, client-ok (role=allowed) reaches the backend while client-bad stays blocked, proving the NetworkPolicy is enforced.

Killercoda: https://killercoda.com/playgrounds/scenario/kubernetes

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE COMPLETE

All five CKAD domains complete.

Design & build · Deployment · Observability · Config & Security · Services & Networking.

© 2026 Tertiary Infotech Academy Pte Ltd

MOCK EXAM

Mock-Exam Practice — from 1:00 PM

Practise on killer.sh / Killercoda before the graded assessment · ask questions freely

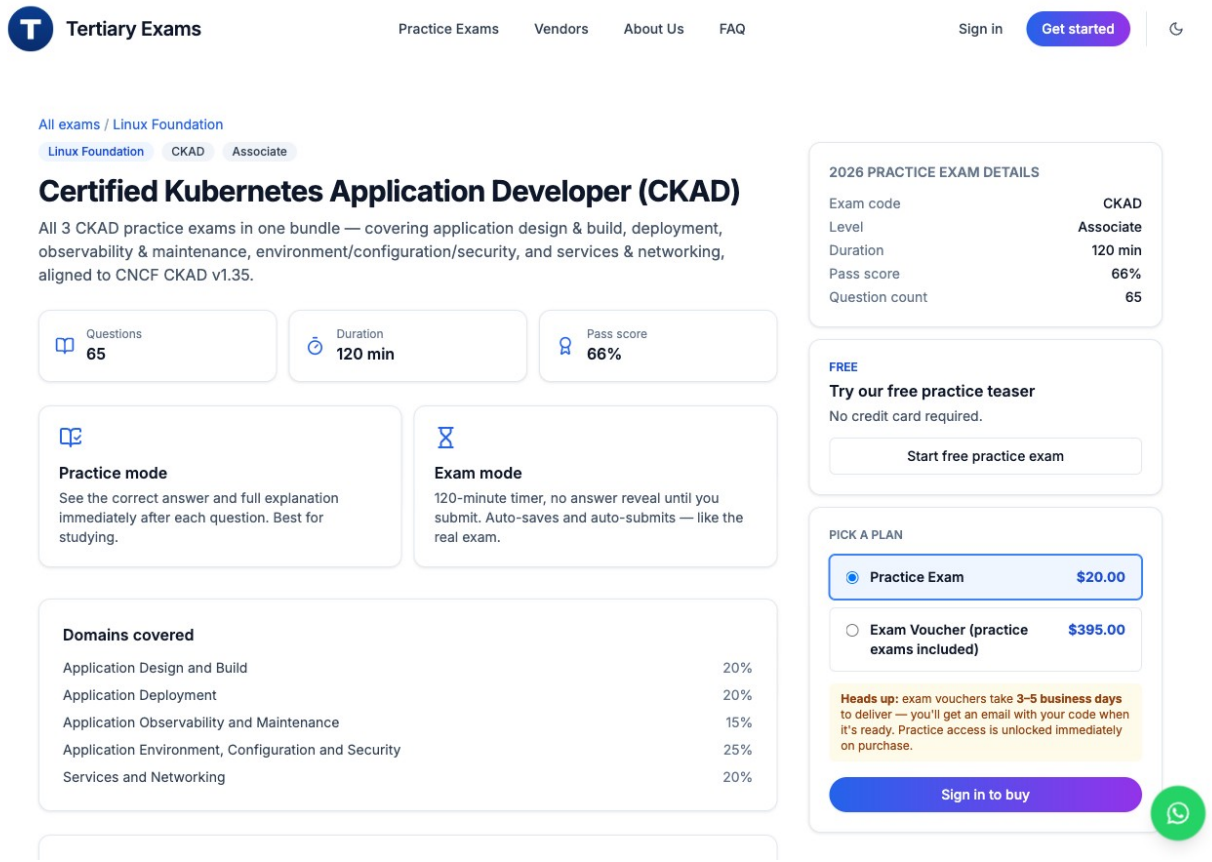
© 2026 Tertiary Infotech Academy Pte Ltd

PRACTICE EXAM

Take the Online CKAD Practice Exams

- Reinforce all five CKAD domains with realistic, exam-style questions before exam day.
- 3 full practice exams · 65 questions · 120-minute timer · 66% pass mark — aligned to CNCF CKAD v1.35.
- Practice mode reveals the answer and a full explanation after each question; Exam mode mirrors the real timed test.
- Start free with the practice teaser — no credit card required; sign in to unlock the full bundle.

Sign up and practise at:
exams.tertiaryinfotech.com/practice-exams/linuxfoundation/linuxfoundation-ckad



© 2026 Tertiary Infotech Academy Pte Ltd

THANK YOU

Thank you — and good luck with the CKAD exam!

Keep practising on killer.sh and Killercoda; the exam is 2 hours, 100% hands-on.

© 2026 Tertiary Infotech Academy Pte Ltd

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer displays the digital attendance QR code from the SSG portal.
- Scan the QR code with your phone camera and submit your attendance.
- A minimum of 75% attendance is required for assessment and funding.



Minimum 75% attendance required

© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT RULES

Briefing for Assessment

Clear your space

Keep only approved materials on the table.

No recording

No photos of the assessment.

Work individually

No discussion during the assessment.

Use the LMS

The assessment is delivered online.

Open book

Slides, Learner Guide & kubernetes.io are allowed.

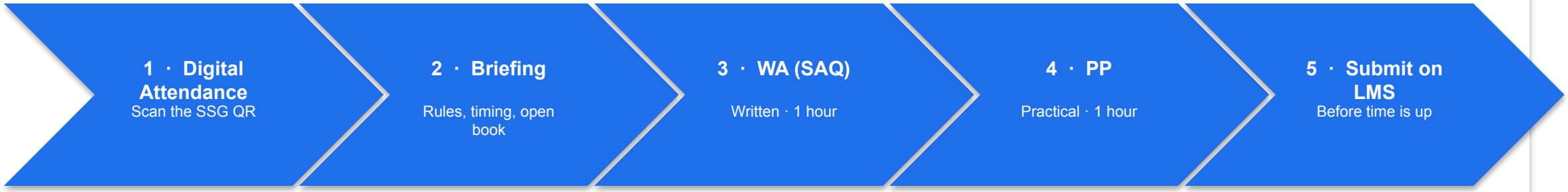
Time's up

Submit when time is up.

© 2026 Tertiary Infotech Academy Pte Ltd

ASSESSMENT FLOW

How the Assessment Runs



SIGN-OFF PATH

Trainer marks

Against the WSQ marking guide for every K and A criterion.

Result recorded

Competent / Not Yet Competent entered on the LMS.

Learner signs

You sign the Assessment Summary Record.

Assessor signs off

Trainer signs and submits the record to SSG.

Assessed as Competent = minimum 75% attendance · pass the WA (SAQ) · pass the Practical Performance

© 2026 Tertiary Infotech Academy Pte Ltd

FINAL ASSESSMENT

Assessment

Final Assessment

- Written Assessment (SAQ) 1 hour + Practical Performance 1 hour
- Mock-exam practice from 1:30 PM; final assessment from 2:00 PM
- Covers all five CKAD domains and the hands-on labs
- Open book — slides, Learner Guide & kubernetes.io
- Must be attempted for SSG funding

Funding & Competency

Minimum 75% attendance

Based on SSG digital attendance records.

Assessed as Competent

Pass the Written Assessment (SAQ) and the Practical Performance.

© 2026 Tertiary Infotech Academy Pte Ltd

THANK YOU

Thank you — and good luck with the CKAD exam!

Keep practising on killer.sh and Killercoda; the exam is 2 hours, 100% hands-on.